

尚硅谷大数据项目之湖仓一体 （电商数据仓库系统）

（作者：尚硅谷大数据研发部）

版本：V1.0

第 1 章 离线数仓、实时数仓与 Data Lakehouse

1.1 什么是数据仓库

数据仓库是一个为数据分析而设计的企业级数据管理系统。数据仓库可集中、整合多个信息源的大量数据，借助数据仓库的分析能力，企业可从数据中获得宝贵的信息进而改进决策。同时，随着时间的推移，数据仓库中积累的大量历史数据对于数据科学家和业务分析师也是十分宝贵的。

1.2 什么是数据湖

数据湖（Data Lake）和数据库、数据仓库一样，都是数据存储的设计模式。数据库和数据仓库会以关系型的方式来设计存储、处理数据。但数据湖的设计理念是相反的，数据仓库是为了保障数据的质量、数据的一致性、数据的重用性等对数据进行结构化处理。

数据湖是一个数据存储库，可以使用数据湖来存储大量的原始数据。现在企业的数据仓库都会通过分层的方式将数据存储到文件夹、文件中，而数据湖使用的是平面架构来存储数据。我们需要做的只是给每个数据元素分配一个唯一的标识符，并通过元数据标签来进行标注。当企业中出现业务问题时，可以从数据湖中查询数据，然后分析业务对应的那一小部分数据集来解决业务问题。

了解过 Hadoop 的同学知道，基于 Hadoop 可以存储任意形式的数据。所以，很多时候数据湖会和 Hadoop 关联到一起。例如：把数据加载到 Hadoop 中，然后将数据分析、和数据挖掘的工具基于 Hadoop 进行处理。数据湖越来越多的用于描述任何的大型数据池，数据都是以原始数据方式存储，知道需要查询应用数据的时候才会开始分析数据需求和应用架构。

数据湖是描述数据存储策略的方式，并不与具体的某个技术框架关联。数据库、数据仓库也一样。它们都是数据的管理策略。

数据湖是专注于原始数据保真以及低成本长期存储的存储设计模式，它相当于是对数据
更多 [Java - 大数据 - 前端 - python 人工智能资料下载](#)，可百度访问：[尚硅谷官网](#)

仓库的补充。数据湖是用于长期存储数据容器的集合，通过数据湖可以大规模的捕获、加工、探索任何形式的原始数据。通过使用一些低成本的技术，可以让下游设施可以更好地利用，下游设施包括像数据集市、数据仓库或者是机器学习模型。

1.3 数据湖的优点

- （1）提供不限数据类型的存储
- （2）开发人员和数据科学家可以快速动态建立数据模型、构建应用、查询数据，非常灵活。
- （3）因为数据湖没有固定的结构，所以更易于访问
- （4）长期存储数据的成本低廉，数据湖可以安装在低成本的硬件在，例如：在一般的 X86 机器上部署 Hadoop
- （5）因为数据湖是非常灵活的，它允许使用多种不同的处理、分析方式来让数据发挥价值，例如：数据分析、实时分析、机器学习以及 SQL 查询都可以。

1.4 数据湖与数据仓库对比

数据湖和数据仓库是用于存储大数据的两种不同策略，最大区别是：数据仓库是提前设计好模式（schema）的，因为数据仓库中存储的都是结构化数据。而在数据湖中，不一定是这样的。数据湖中可以存储结构化和非结构化的数据，是无法预先定义好结构的。

我们来进一步进行对比：

（1）数据的存储位置不同

数据仓库因为是要有结构的，在企业中很多都是基于关系型模型。而数据湖通常位于分布式存储例如 Hadoop 或者类似的大数据存储中。

（2）数据源不同

数据仓库的数据来源很多时候来自于 OLTP 应用的结构化数据库中提取的，用于支持内部的业务部门（例如：销售、市场、运营等部门）进行业务分析。而数据湖的数据来源可以是结构化的、也可以是非结构化的，例如：业务系统数据库、IOT 设备、社交媒体、移动 APP 等。

（3）用户不同

数据仓库主要是业务系统的大量业务数据进行统计分析，所以会应用数据分析的部门是数据仓库的主要用户，例如：销售部、市场部、运营部、总裁办等等。而当需要一个大型的

存储，而当前没有明确的数据应用用户或者是目标，将来想要使用这些数据的人可以在使用时开始设计架构，此时，数据湖更适合。

但数据湖中的数据都是原始数据，是未经整理的，这对于普通的用户几乎是不可用的。数据湖更适合数据科学家，因为数据科学家可以应用模型、技术发觉数据中的价值，去解决企业中的业务问题。

（4）数据质量

数据仓库是非常重数据质量的，大家现在经常听说的数据中台，其中有一大块是数据质量管理、数据资产管理等。数据仓库中的数据都是经过处理的。而数据湖中的数据可靠性是较差的，这些数据可能是任意状态、形态的数据。

（5）数据模式

数据仓库在数据写入之前就要定义好模式（`schema`），例如：我们会先建立模型、建立表结构，然后导入数据。我们可以把它称之为 `write-schema`。而数据湖中的数据是没有模式的，直到有用户要访问数据、使用数据才会建立 `schema`。我们可以把它称之为 `read-schema`。

（6）敏捷扩展性

数据仓库的模式一旦建立，要重新调整模式，往往代价很大，牵一发而动全身，所有相关的 ETL 程序可能都需要调整。而数据湖是非常灵活的，可以根据需要重新配置结构或者模式。

基于上述内容，我们可以了解到，数据湖和数据仓库的应用点是不一样的。他们是两种相对独立的数据设计模式。在一些企业中，可能会既有数据湖、又有数据仓库。数据湖并不是要替代数据仓库，而是对企业的管理模式进行补充。

（7）应用

数据仓库一般用于做批处理报告、BI、可视化。而数据湖主要用于机器学习、预测分析、数据探索和分析。

1.5 数据湖应用

数据湖是用于数据存储的设计模式，但最终数据肯定是需要一种介质存储下来的。我们可以自己来选择数据湖的物理存储引擎。例如：使用 Hadoop 作为数据湖的物理存储引擎、或者使用 AWS 的 S3 作为存储引擎等。

但架构数据湖时，需要注意几点原则，这几点原则也将和其他数据存储方法区别开来。

可以加载各种源系统中的数据，并存储。

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

任意类型的数据都可以存储。

数据是以原始状态保存在数据湖中的，是几乎不需要做任何转换的。

数据可以根据应用、分析的要求，进行转换成适合分析的模式

构建数据湖时，为了方便数据的管理。我们可以建立一些管理办法，例如：

将数据进行合理分类，例如：按照数据类型分类、按照业务内容分类、按照应用场景分类或者按照可能的用户来分类。

为了方便数据湖的数据存取，要提前定义好命名规则和固定的文件目录结构。

如果出现数据质量问题也可以解决掉。

建立数据访问标准，可以追踪到哪些用户正在访问数据。

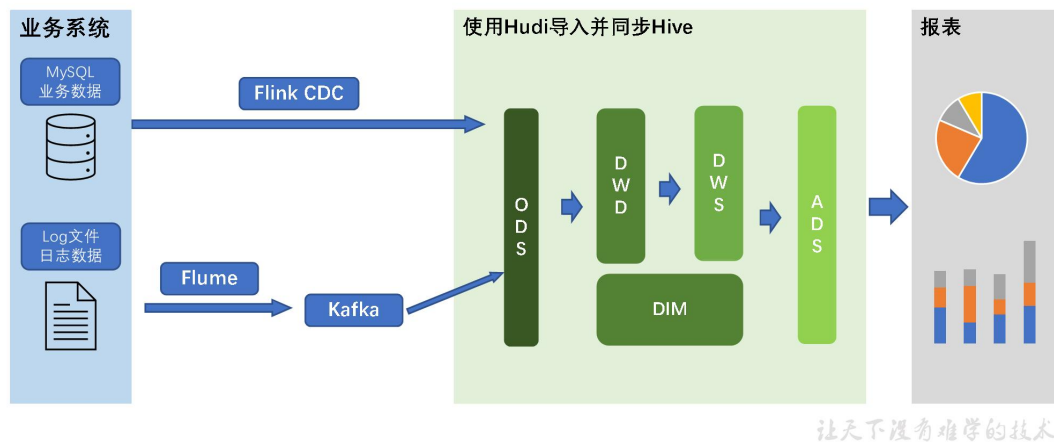
让数据目录可以被检索到。

提供一些加密、监控、授权、警报等功能。

1.6 湖仓一体核心架构



湖仓一体核心架构



第 2 章 建模概述

数据仓库中的建模方式同样适用于湖仓一体建模。

2.1 建模的意义

如果把数据看作图书馆里的书，我们希望看到它们在书架上分门别类地放置；如果把数据看作城市的建筑，我们希望城市规划布局合理；如果把数据看作电脑文件和文件夹，我们希望按照自己的习惯有很好的文件夹组织方式，而不是糟糕混乱的桌面，经常为找一个文件而不知所措。

数据模型就是数据组织和存储方法，它强调从业务、数据存取和使用角度合理存储数据。只有将数据有序的组织 and 存储起来之后，数据才能得到高性能、低成本、高效率、高质量的使用。

高性能：良好的数据模型能够帮助我们快速查询所需要的数据。

低成本：良好的数据模型能减少重复计算，实现计算结果的复用，降低计算成本。

高效率：良好的数据模型能极大的改善用户使用数据的体验，提高使用数据的效率。

高质量：良好的数据模型能改善数据统计口径的混乱，减少计算错误的可能性。

2.2 建模方法论

2.2.1 ER 模型

数据仓库之父 Bill Inmon 提出的建模方法是从全企业的高度，用实体关系（Entity Relationship, ER）模型来描述企业业务，并用规范化的方式表示出来，在范式理论上符合 3NF。

1) 实体关系模型

实体关系模型将复杂的数据抽象为两个概念——实体和关系。实体表示一个对象，例如学生、班级，关系是指两个实体之间的关系，例如学生和班级之间的从属关系。

2) 数据库规范化

数据库规范化是使用一系列范式设计数据库（通常是关系型数据库）的过程，其目的是减少数据冗余，增强数据的一致性。

这一系列范式就是指在设计关系型数据库时，需要遵从的不同的规范。关系型数据库的范式一共有六种，分别是第一范式（1NF）、第二范式（2NF）、第三范式（3NF）、巴斯-

科德范式（BCNF）、第四范式(4NF)和第五范式（5NF）。遵循的范式级别越高，数据冗余性就越低。

3) 三范式

(1) 函数依赖

函数依赖

学号	姓名	系名	系主任	课程	分数
1022211101	李小明	经济系	王强	高等数学	95
1022211101	李小明	经济系	王强	大学英语	87
1022211101	李小明	经济系	王强	普通化学	76
1022211102	张莉莉	经济系	王强	高等数学	72
1022211102	张莉莉	经济系	王强	大学英语	98
1022211102	张莉莉	经济系	王强	计算机基础	88
1022511101	高芳芳	法律系	刘玲	高等数学	82
1022511101	高芳芳	法律系	刘玲	法学基础	82

1、完全函数依赖：

设X, Y是关系R的两个属性集合，X'是X的真子集，存在 $X \rightarrow Y$ ，但对每一个X'都有 $X' \not\rightarrow Y$ ，则称Y完全函数依赖于X。记做： $X \twoheadrightarrow Y$

人类语言：

比如通过，(学号，课程)推出分数，但是单独用学号推断不出来分数，那么可以说：分数完全依赖于(学号，课程)。

即：通过AB能得出C，但是AB单独得不出C，那么说C完全依赖于AB。

2、部分函数依赖

假如Y函数依赖于X，但同时Y并不完全函数依赖于X，

那么我们就称Y部分函数依赖于X，记做： $X \rightharpoonup Y$

人类语言：

比如通过，(学号，课程)推出姓名，因为其实直接可以通过，学号推出姓名，所以：姓名部分依赖于(学号，课程)

即：通过AB能得出C，通过A也能得出C，或者通过B也能得出C，那么说C部分依赖于AB。

3、传递函数依赖

传递函数依赖：设X, Y, Z是关系R中互不相同的属性集合，存在 $X \rightarrow Y (Y \not\rightarrow X), Y \rightarrow Z$ ，则称Z传递函数依赖于X。记做： $X \twoheadrightarrow Z$

人类语言：

比如：学号推出系名，系名推出系主任，但是，系主任推不出学号，系主任主要依赖于系名。这种情况可以说：系主任传递依赖于学号

通过A得到B，通过B得到C，但是C得不到A，那么说C传递依赖于A。
让天下没有难学的技术

(2) 第一范式

第一范式

1、第一范式1NF核心原则就是：属性不可分割

表 不符合一范式的表格设计

ID	商品	商家ID	用户ID
001	5 台电脑	XXX旗舰店	00001

很明显上图所示的表格设计是不符合第一范式的，商品列中的数据不是原子数据项，是可以进行分割的，因此对表格进行修改，让表格符合第一范式的要求，修改结果如下图所示：

表 符合一范式的表格设计

ID	商品	数量	商家ID	用户ID
001	电脑	5	XXX旗舰店	00001

实际上，1NF是所有关系型数据库的最基本要求，你在关系型数据库管理系统（RDBMS），例如SQL Server, Oracle, MySQL中创建数据表的时候，如果数据表的设计不符合这个最基本的要求，那么操作一定是不能成功的。也就是说，只要在RDBMS中已经存在的数据表，一定是符合1NF的。

让天下没有难学的技术

(3) 第二范式

2、第二范式2NF核心原则：不能存在“部分函数依赖”

学号	姓名	系名	系主任	课名	分数
1022211101	李小明	经济系	王强	高等数学	95
1022211101	李小明	经济系	王强	大学英语	87
1022211101	李小明	经济系	王强	普通化学	76
1022211102	张莉莉	经济系	王强	高等数学	72
1022211102	张莉莉	经济系	王强	大学英语	98
1022211102	张莉莉	经济系	王强	计算机基础	88
1022511101	高芳芳	法律系	刘玲	高等数学	82
1022511101	高芳芳	法律系	刘玲	法学基础	82

以上表格明显存在，部分依赖。比如，这张表的主键是（学号，课名），分数确实完全依赖于（学号，课名），但是姓名并不完全依赖于（学号，课名）

学号	课名	分数
1022211101	高等数学	95
1022211101	大学英语	87
1022211101	普通化学	76
1022211102	高等数学	72
1022211102	大学英语	98
1022211102	计算机基础	88
1022511101	高等数学	82
1022511101	法学基础	82

学号	姓名	系名	系主任
1022211101	李小明	经济系	王强
1022211102	张莉莉	经济系	王强
1022511101	高芳芳	法律系	刘玲

以上符合第二范式，去掉部分函数依赖依赖

让天下没有难学的技术

(4) 第三范式

3、第三范式3NF核心原则：不能存在传递函数依赖

在下面这张表中，存在传递函数依赖：学号->系名->系主任，但是系主任推不出学号。

学号	姓名	系名	系主任
1022211101	李小明	经济系	王强
1022211102	张莉莉	经济系	王强
1022511101	高芳芳	法律系	刘玲

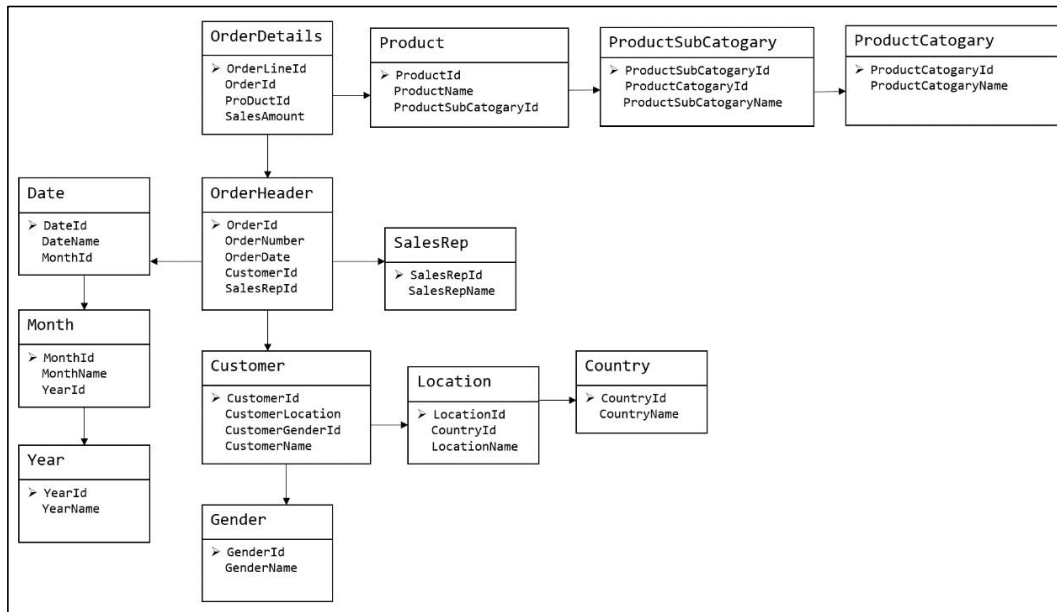
上面表需要再次拆解：

学号	姓名	系名
1022211101	李小明	经济系
1022211102	张莉莉	经济系
1022511101	高芳芳	法律系

系名	系主任
经济系	王强
法律系	刘玲

让天下没有难学的技术

下图为一个采用 Bill Inmon 倡导的建模方法构建的模型，从图中可以看出，较为松散、零碎，物理表数量多。



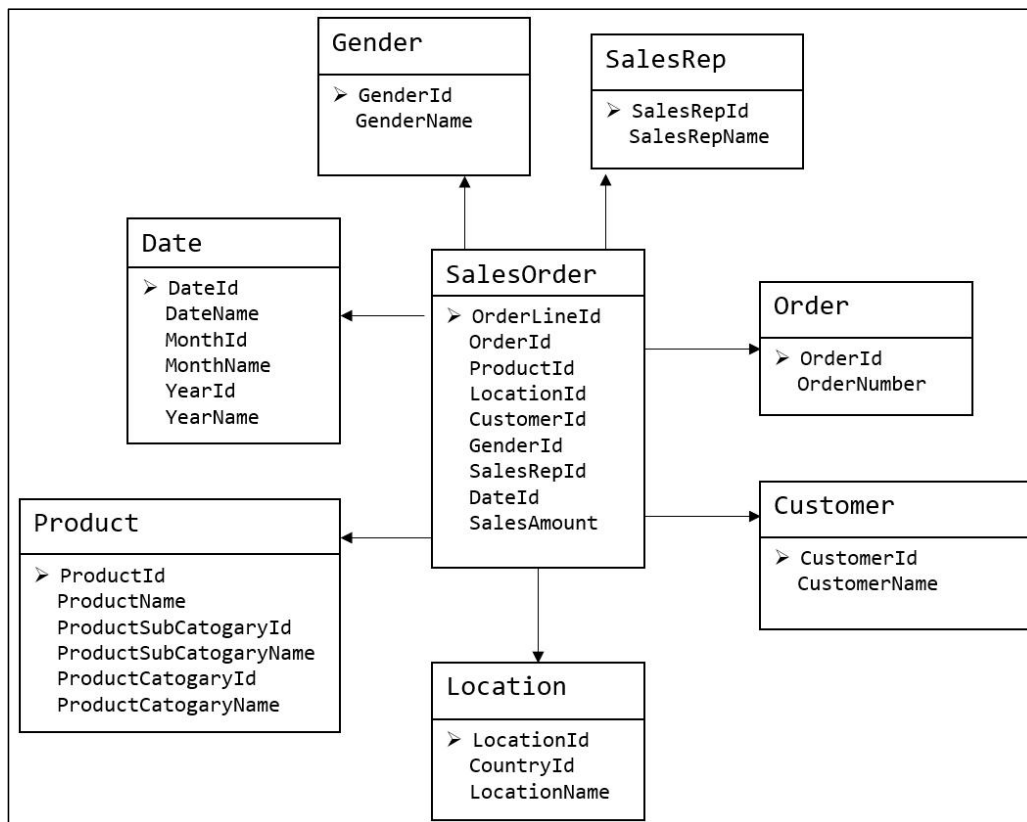
这种建模方法的出发点是整合数据，其目的是将整个企业的数据进行组合和合并，并进行规范处理，减少数据冗余性，保证数据的一致性。这种模型并不适合直接用于分析统计。

2.2.2 维度模型

数据仓库领域的另一位大师——Ralph Kimball 倡导的建模方法为维度建模。维度模型将复杂的业务通过**事实**和**维度**两个概念进行呈现。事实通常对应业务过程，而维度通常对应业务过程发生时所处的环境。

注：业务过程可以概括为一个个不可拆分的行为事件，例如电商交易中的下单，取消订单，付款，退单等，都是业务过程。

下图为一个典型的维度模型，其中位于中心的 **SalesOrder** 为事实表，其中保存的是下单这个业务过程的所有记录。位于周围每张表都是维度表，包括 **Date**（日期），**Customer**（顾客），**Product**（产品），**Location**（地区）等，这些维度表就组成了每个订单发生时所处的环境，即何人、何时、在何地下单了何种产品。从图中可以看出，模型相对清晰、简洁。



维度建模以数据分析作为出发点，为数据分析服务，因此它关注的重点的用户如何更快的完成需求分析以及如何实现较好的大规模复杂查询的响应性能。

第 3 章 维度建模理论之事实表

3.1 事实表概述

事实表作为数据仓库维度建模的核心，紧紧围绕着业务过程来设计。其包含与该业务过程有关的维度引用（维度表外键）以及该业务过程的度量（通常是可累加的数字类型字段）。

3.1.1 事实表特点

事实表通常比较“细长”，即列较少，但行较多，且行的增速快。

3.1.2 事实表分类

事实表有三种类型：分别是事务事实表、周期快照事实表和累积快照事实表，每种事实表都具有不同的特点和适用场景，下面逐个介绍。

3.2 事务型事实表

3.2.1 概述

事务事实表用来记录各业务过程，它保存的是各业务过程的原子操作事件，即最细粒度的操作事件。粒度是指事实表中一行数据所表达的业务细节程度。

事务型事实表可用于分析与各业务过程相关的各项统计指标，由于其保存了最细粒度的记录，可以提供最大限度的灵活性，可以支持无法预期的各种细节层次的统计需求。

3.2.2 设计流程

设计事务事实表时一般可遵循以下四个步骤：

选择业务过程→声明粒度→确认维度→确认事实

1) 选择业务过程

在业务系统中，挑选我们**感兴趣**的业务过程，业务过程可以概括为一个个不可拆分的行为事件，例如电商交易中的下单，取消订单，付款，退单等，都是业务过程。通常情况下，一个业务过程对应一张事务型事实表。

2) 声明粒度

业务过程确定后，需要为每个业务过程声明粒度。即精确定义每张事务型事实表的每行数据表示什么，应该尽可能选择最细粒度，以此来应各种细节程度的需求。

典型的粒度声明如下：

订单事实表中一行数据表示的是一个订单中的一个商品项。

3) 确定维度

确定维度具体是指，确定与每张事务型事实表相关的维度有哪些。

确定维度时应尽量多的选择与业务过程相关的环境信息。因为维度的丰富程度就决定了维度模型能够支持的指标丰富程度。

4) 确定事实

此处的“事实”一词，指的是每个业务过程的度量值（通常是可累加的数字类型的值，例如：次数、个数、件数、金额等）。

经过上述四个步骤，事务型事实表就基本设计完成了。第一步选择业务过程可以确定有哪些事务型事实表，第二步可以确定每张事务型事实表的每行数据是什么，第三步可以确定每张事务型事实表的维度外键，第四步可以确定每张事务型事实表的度量值字段。

3.2.3 不足

事务型事实表可以保存所有业务过程的最细粒度的操作事件，故理论上其可以支撑与各业务过程相关的各种统计粒度的需求。但对于某些特定类型的需求，其逻辑可能会比较复杂，或者效率会比较低下。例如：

1) 存量型指标

例如商品库存，账户余额等。此处以电商中的虚拟货币为例，虚拟货币业务包含的业务过程主要包括获取货币和使用货币，两个业务过程各自对应一张事务型事实表，一张存储所有的获取货币的原子操作事件，另一张存储所有使用货币的原子操作事件。

假定现有一个需求，要求统计截至当日的各用户虚拟货币余额。由于获取货币和使用货币均会影响到余额，故需要对两张事务型事实表进行聚合，且需要区分两者对余额的影响（加或减），另外需要对两张表的全表数据聚合才能得到统计结果。

可以看到，不论是从逻辑上还是效率上考虑，这都不是一个好的方案。

2) 多事务关联统计

例如，现需要统计最近 30 天，用户下单到支付的时间间隔的平均值。统计思路应该是找到下单事务事实表和支付事务事实表，过滤出最近 30 天的记录，然后按照订单 id 对两张事实表进行关联，之后用支付时间减去下单时间，然后再求平均值。

逻辑上虽然并不复杂，但是其效率较低，应为下单事务事实表和支付事务事实表均为大

表，大表 join 大表的操作应尽量避免。

可以看到，在上述两种场景下事务型事实表的表现并不理想。下面要介绍的另外两种类型的事实表就是为了弥补事务型事实表的不足的。

3.3 周期型快照事实表

3.3.1 概述

周期快照事实表以具有规律性的、可预见的时间间隔来记录事实，主要用于分析一些存量型（例如商品库存，账户余额）或者状态型（空气温度，行驶速度）指标。

对于商品库存、账户余额这些存量型指标，业务系统中通常就会计算并保存最新结果，所以定期同步一份全量数据到数据仓库，构建周期型快照事实表，就能轻松应对此类统计需求，而无需再对事务型事实表中大量的历史记录进行聚合了。

对于空气温度、行驶速度这些状态型指标，由于它们的值往往是连续的，我们无法捕获其变动的原子事务操作，所以无法使用事务型事实表统计此类需求。而只能定期对其进行采样，构建周期型快照事实表。

3.3.2 设计流程

1) 确定粒度

周期型快照事实表的粒度可由采样周期和维度描述，故确定采样周期和维度后即可确定粒度。

采样周期通常选择每日。

维度可根据统计指标决定，例如指标为统计每个仓库中每种商品的库存，则可确定维度为仓库和商品。

确定完采样周期和维度后，即可确定该表粒度为每日-仓库-商品。

2) 确认事实

事实也可根据统计指标决定，例如指标为统计每个仓库中每种商品的库存，则事实为商品库存。

3.3.3 事实类型

此处的事实类型是指度量值的类型，而非事实表的类型。事实（度量值）共分为三类，分别是可加事实，半可加事实和不可加事实。

1) 可加事实

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

可加事实是指可以按照与事实表相关的所有维度进行累加，例如事务型事实表中的事实。

2) 半可加事实

半可加事实是指只能按照与事实表相关的一部分维度进行累加，例如周期型快照事实表中的事实。以上述各仓库中各商品的库存每天快照事实表为例，这张表中的库存事实可以按照仓库或者商品维度进行累加，但是不能按照时间维度进行累加，因为将每天的库存累加起来是没有任何意义的。

3) 不可加事实

不可加事实是指完全不具备可加性，例如比率型事实。不可加事实通常需要转化为可加事实，例如比率可转化为分子和分母。

3.4 累积型快照事实表

3.4.1 概述

累计快照事实表是基于一个业务流程中的多个关键业务过程联合处理而构建的事实表，如交易流程中的下单、支付、发货、确认收货业务过程。

累积型快照事实表通常具有多个日期字段，每个日期对应业务流程中的一个关键业务过程（里程碑）。

订单 id	用户 id	下单日期	支付日期	发货日期	确认收货日期	订单金额	支付金额
1001	1234	2020-06-14	2020-06-15	2020-06-16	2020-06-17	1000	1000

累积型快照事实表主要用于分析业务过程（里程碑）之间的时间间隔等需求。例如前文提到的用户下单到支付的平均时间间隔，使用累积型快照事实表进行统计，就能避免两个事务事实表的关联操作，从而变得十分简单高效。

3.4.2 设计流程

累积型快照事实表的设计流程同事务型事实表类似，也可采用以下四个步骤，下面重点描述与事务型事实表的不同之处。

选择业务过程→声明粒度→确认维度→确认事实。

1) 选择业务过程

选择一个业务流程中需要关联分析的多个关键业务过程，多个业务过程对应一张累积型快照事实表。

2) 声明粒度

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

精确定义每行数据表示的是什么，尽量选择最小粒度。

3) 确认维度

选择与各业务过程相关的维度，需要注意的是，每各业务过程均需要一个日期维度。

4) 确认事实

选择各业务过程的度量值。

第 4 章 维度建模理论之维度表

4.1 维度表概述

维度表是维度建模的基础和灵魂。前文提到，事实表紧紧围绕业务过程进行设计，而维度表则围绕业务过程所处的环境进行设计。维度表主要包含一个主键和各种维度字段，维度字段称为维度属性。

4.2 维度表设计步骤

1) 确定维度（表）

在设计事实表时，已经确定了与每个事实表相关的维度，理论上每个相关维度均需对应一张维度表。需要注意到，可能存在多个事实表与同一个维度都相关的情况，这种情况需保证维度的唯一性，即只创建一张维度表。另外，如果某些维度表的维度属性很少，例如只有一个**名称，则可不创建该维度表，而把该表的维度属性直接增加到与之相关的事实表中，这个操作称为**维度退化**。

2) 确定主维表和相关维表

此处的主维表和相关维表均指**业务系统**中与某维度相关的表。例如业务系统中与商品相关的表有 sku_info, spu_info, base_trademark, base_category3, base_category2, base_category1 等，其中 sku_info 就称为商品维度的主维表，其余表称为商品维度的相关维表。维度表的粒度通常与主维表相同。

3) 确定维度属性

确定维度属性即确定维度表字段。维度属性主要来自于业务系统中与该维度对应的主维表和相关维表。维度属性可直接从主维表或相关维表中选择，也可通过进一步加工得到。

确定维度属性时，需要遵循以下要求：

（1）尽可能生成丰富的维度属性

维度属性是后续做分析统计时的查询约束条件、分组字段的基本来源，是数据易用性的关键。维度属性的丰富程度直接影响到数据模型能够支持的指标的丰富程度。

（2）尽量不使用编码，而使用明确的文字说明，一般可以编码和文字共存。

（3）尽量沉淀出通用的维度属性

有些维度属性的获取需要进行比较复杂的逻辑处理，例如需要通过多个字段拼接得到。为避免后续每次使用时的重复处理，可将这些维度属性沉淀到维度表中。

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

4.3 维度设计要点

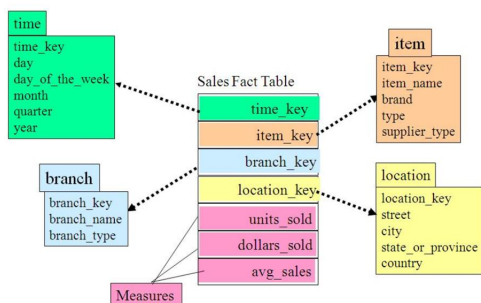
4.3.1 规范化与反规范化

规范化是指使用一系列范式设计数据库的过程，其目的是减少数据冗余，增强数据的一致性。通常情况下，规范化之后，一张表的字段会拆分到多张表。

反规范化是指将多张表的数据冗余到一张表，其目的是减少 join 操作，提高查询性能。在设计维度表时，如果对其进行规范化，得到的维度模型称为雪花模型，如果对其进行反规范化，得到的模型称为星型模型。

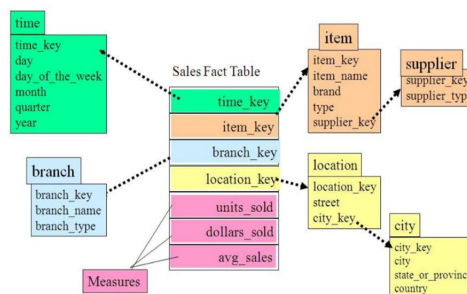
星型模型、雪花模型

1、星型模型



雪花模型与星型模型的区别主要在于维度表是否进行规范化。

2、雪花模型



雪花模型，比较靠近3NF，但是无法完全遵守，因为遵循3NF的性能成本太高。

让天下没有难学的技术

数据仓库系统的主要目的是用于数据分析和统计，所以是否方便用户进行统计分析决定了模型的优劣。采用雪花模型，用户在统计分析的过程中需要大量的关联操作，使用复杂度高，同时查询性能很差，而采用星型模型，则方便、易用且性能好。所以出于易用性和性能的考虑，维度表一般是很不规范化的。

4.3.2 维度变化

维度属性通常不是静态的，而是会随时间变化的，数据仓库的一个重要特点就是反映历史的变化，所以如何保存维度的历史状态是维度设计的重要工作之一。保存维度数据的历史状态，通常有以下两种做法，分别是全量快照表和拉链表。

1) 全量快照表

离线数据仓库的计算周期通常为每天一次，所以可以每天保存一份全量的维度数据。这种方式的优点和缺点都很明显。

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

优点是简单而有效，开发和维护成本低，且方便理解和使用。

缺点是浪费存储空间，尤其是当数据的变化比例比较低时。

2) 拉链表

拉链表的意义就在于能够更加高效的保存维度信息的历史状态。

(1) 什么是拉链表



拉链表，记录每条信息的生命周期，一旦一条记录的生命周期结束，就重新开始一条新的记录，并把当前日期放入生效开始日期。

如果当前信息至今有效，在生效结束日期中填入一个极大值（如9999-12-31）。

用户ID	姓名	手机号码	开始日期	结束日期
1	张三	136****9090	2019-01-01	2019-01-01
1	张三	137****8989	2019-01-02	2019-01-09
1	张三	147****1234	2019-01-10	9999-12-31

让天下没有难学的技术

(2) 为什么要做拉链表



拉链表适合于：数据会发生变化，但是变化频率并不高的维度（即：缓慢变化维）。

比如：用户信息会发生变化，但是每天变化的比例不高。如果数据量有一定规模，按照每日全量的方式保存效率很低。比如：1亿用户*365天，每天一份用户信息。（做每日全量效率低）

每日全量表

用户ID	姓名	手机号码	dt
1	张三	136****9090	2019-01-01
1	张三	136****9090	2019-01-02
1	张三	136****9090	2019-01-03
...	...	...	...
1	张三	136****9090	2019-05-11
1	张三	155****1212	2019-05-12

拉链表

用户ID	姓名	手机号码	开始日期	结束日期
1	张三	136****9090	2019-01-01	2019-05-11
1	张三	155****1212	2019-05-12	9999-12-31

让天下没有难学的技术

(3) 如何使用拉链表



通过，生效开始日期<=某个日期 且 生效结束日期>=某个日期，能够得到某个时间点的数据全量切片。

1) 拉链表数据

用户ID	姓名	开始时间	结束时间
1	张三	2019-01-01	9999-12-31
2	李四	2019-01-01	2019-01-02
2	李小四	2019-01-03	9999-12-31
3	王五	2019-01-01	9999-12-31
4	赵六	2019-01-02	9999-12-31

2) 例如获取2019-01-01的历史切片：select * from user_info where start_date<='2019-01-01' and end_date>='2019-01-01'

用户ID	姓名	开始时间	结束时间
1	张三	2019-01-01	9999-12-31
2	李四	2019-01-01	2019-01-02
3	王五	2019-01-01	9999-12-31

3) 例如获取2019-01-02的历史切片：select * from order_info where start_date<='2019-01-02' and end_date>='2019-01-02'

用户ID	姓名	开始时间	结束时间
1	张三	2019-01-01	9999-12-31
2	李四	2019-01-01	2019-01-02
3	王五	2019-01-01	9999-12-31
4	赵六	2019-01-02	9999-12-31

让天下没有难学的技术

4.3.3 多值维度

如果事实表中一条记录在某个维度表中有多条记录与之对应，称为多值维度。例如，下单事实表中的一条记录为一个订单，一个订单可能包含多个商品，所以商品维度表中就可能有多条数据与之对应。

针对这种情况，通常采用以下两种方案解决。

第一种：降低事实表的粒度，例如将订单事实表的粒度由一个订单降低为一个订单中的一个商品项。

第二种：在事实表中采用多字段保存多个维度值，每个字段保存一个维度 id。这种方案只适用于多值维度个数固定的情况。

建议尽量采用第一种方案解决多值维度问题。

4.3.4 多值属性

维表中的某个属性同时有多个值，称之为“多值属性”，例如商品维度的平台属性和销售属性，每个商品均有多个属性值。

针对这种情况，通常有可以采用以下两种方案。

第一种：将多值属性放到一个字段，该字段内容为 key1:value1, key2:value2 的形式，例如一个手机商品的平台属性值为“品牌:华为，系统:鸿蒙，CPU:麒麟 990”。

第二种：将多值属性放到多个字段，每个字段对应一个属性。这种方案只适用于多值属性个数固定的情况。

第 5 章 湖仓一体化设计

5.1 湖仓一体分层规划

数据湖仓一体化同样需要良好的数据分层结构。

合理的分层，能够使数据体系更加清晰，使复杂问题得以简化。以下是该项目的分层规划。



分层规划

数据应用层（ADS）

存放各项统计指标结果。

汇总数据层（DWS）

基于上层的指标需求，以分析的主题对象作为建模驱动，构建公共统计粒度的汇总表。

明细数据层（DWD）

基于维度建模理论进行构建，存放维度模型中的事实表，保存各业务过程最小粒度的操作记录。

原始数据层（ODS）

存放未经过处理的原始数据，结构上与源系统保持一致，是数据仓库的数据准备区。

公共维度层（DIM）

基于维度建模理论进行构建，存放维度模型中的维度表，保存一致性维度信息。

分层简称	全称
ODS	Operation Data Store
DWD	Data Warehouse Detail
DIM	Dimension
DWS	Data Warehouse Summary
ADS	Application Data Service

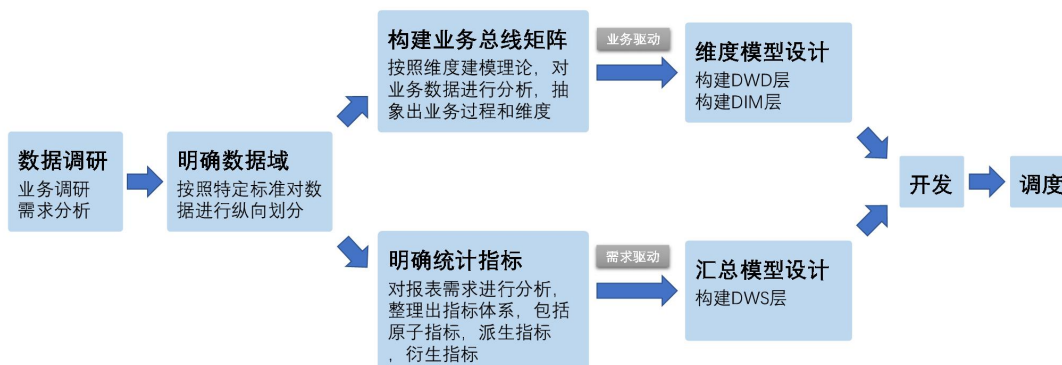
让天下没有难学的技术

5.2 湖仓一体分层构建流程

以下是湖仓一体分层构建的完整流程。



数据仓库构建流程



让天下没有难学的技术

5.2.1 数据调研

数据调研重点要做两项工作，分别是业务调研和需求分析。这两项工作做的是否充分，直接影响着数据仓库的质量。

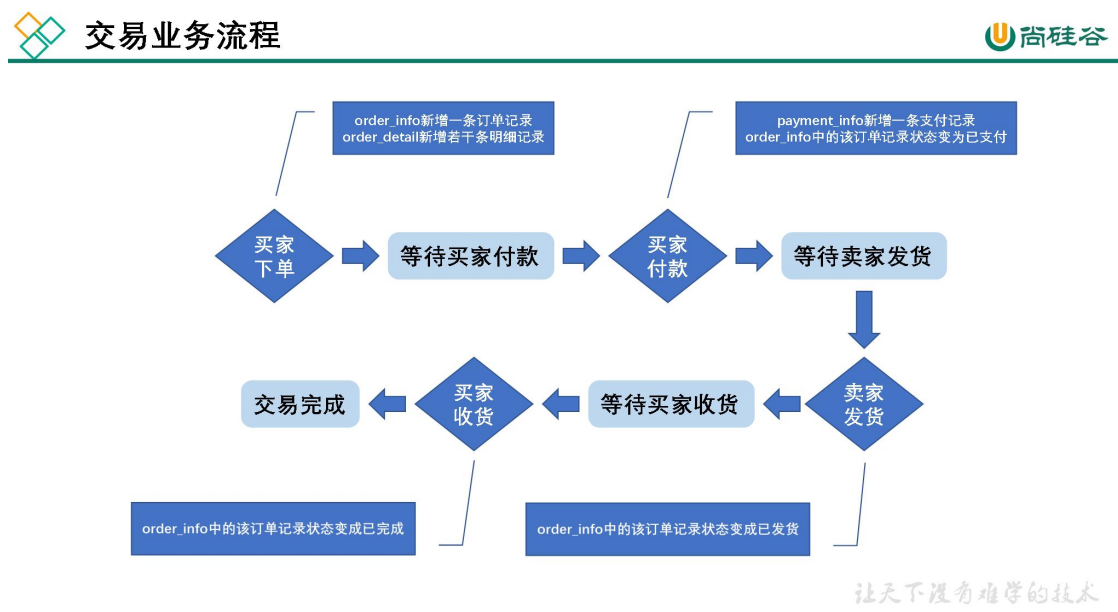
1) 业务调研

业务调研的主要目标是**熟悉业务流程、熟悉业务数据**。

熟悉业务流程要求做到，明确每个业务的具体流程，需要将该业务所包含的每个**业务过程**一一列举出来。

熟悉业务数据要求做到，将数据（包括埋点日志和业务数据表）与业务过程对应起来，明确每个业务过程会对哪些表的数据产生影响，以及产生什么影响。产生的影响，需要具体到，是新增一条数据，还是修改一条数据，并且需要明确新增的内容或者是修改的逻辑。

下面业务电商中的交易为例进行演示，交易业务涉及到的业务过程有买家下单、买家支付、卖家发货，买家收货，具体流程如下图。



2) 需求分析

典型的需求指标如，最近一天各省份手机品类订单总额。

分析需求时，需要明确需求所需的**业务过程及维度**，例如该需求所需的业务过程就是买家下单，所需的维度有日期，省份，商品品类。

3) 总结

做完业务分析和需求分析之后，要保证每个需求都能找到与之对应的业务过程及维度。

更多 [Java](#) -[大数据](#) -[前端](#) -[python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

若现有数据无法满足需求，则需要和业务方进行沟通，例如某个页面需要新增某个行为的埋点。

5.2.2 明确数据域

数据仓库模型设计除横向的分层外，通常也需要根据业务情况进行纵向划分数据域。

划分数据域的意义是**便于数据的管理和应用**。

通常可以根据业务过程或者部门进行划分，本项目根据业务过程进行划分，需要注意的是一个业务过程只能属于一个数据域。

下面是本项目所有业务过程及数据域划分详情。

数据域	业务过程
交易域	加购物车、减购物车、下单、支付完成、确认收货、退单、退款完成
流量域	页面浏览、启动应用、动作、曝光、错误
用户域	注册、登录
互动域	收藏、评价
工具域	优惠券领取、优惠券使用（下单）、优惠券使用（支付）

5.2.3 构建业务总线矩阵

业务总线矩阵中包含维度模型所需的所有事实（业务过程）以及维度，以及各业务过程与各维度的关系。矩阵的行是一个个业务过程，矩阵的列是一个个的维度，行列的交点表示业务过程与维度的关系。



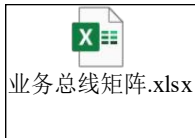
业务总线矩阵

数据域	业务过程	维度						
		时间	用户	商品	地区	支付方式	退单类型	退单原因类型
交易域	加购物车	√	√	√				
	下单	√	√	√	√			
	取消订单	√	√	√	√			
	支付成功	√	√	√	√	√		
	退单	√	√	√	√		√	√
	退款成功	√	√	√	√	√		
用户域	注册	√	√					
	登录	√	√		√			
工具域	领取优惠券	√	√					
	使用优惠券(下单)	√	√					
	使用优惠券(支付)	√	√					
互动域	收藏商品	√	√	√				
	评价	√	√	√				

让天下没有难学的技术

一个业务过程对应维度模型中一张事务型事实表，一个维度则对应维度模型中的一张维度表。所以构建业务总线矩阵的过程就是设计维度模型的过程。但是需要注意的是，总线矩阵中通常只包含事务型事实表，另外两种类型的事实表需单独设计。

按照事务型事实表的设计流程，**选择业务过程→声明粒度→确认维度→确认事实**，得到的最终的业务总线矩阵见以下表格。



后续的 DWD 层以及 DIM 层的搭建需参考业务总线矩阵。

5.2.4 明确统计指标

明确统计指标具体的工作是，深入分析需求，构建指标体系。构建指标体系的主要意义就是指标定义标准化。所有指标的定义，都必须遵循同一套标准，这样能有效的避免指标定义存在歧义，指标定义重复等问题。

1) 指标体系相关概念

(1) 原子指标

原子指标基于某一**业务过程**的**度量值**，是业务定义中不可再拆解的指标，原子指标的核心功能就是对指标的**聚合逻辑**进行了定义。我们可以得出结论，原子指标包含三要素，分别是业务过程、度量值和聚合逻辑。

例如**订单总额**就是一个典型的原子指标，其中的业务过程为用户下单、度量值为订单金额，聚合逻辑为 sum() 求和。需要注意的是原子指标只是用来辅助定义指标一个概念，通常不会对应有实际统计需求与之对应。

(2) 派生指标

派生指标基于原子指标，其与原子指标的关系如下图所示。

派生指标



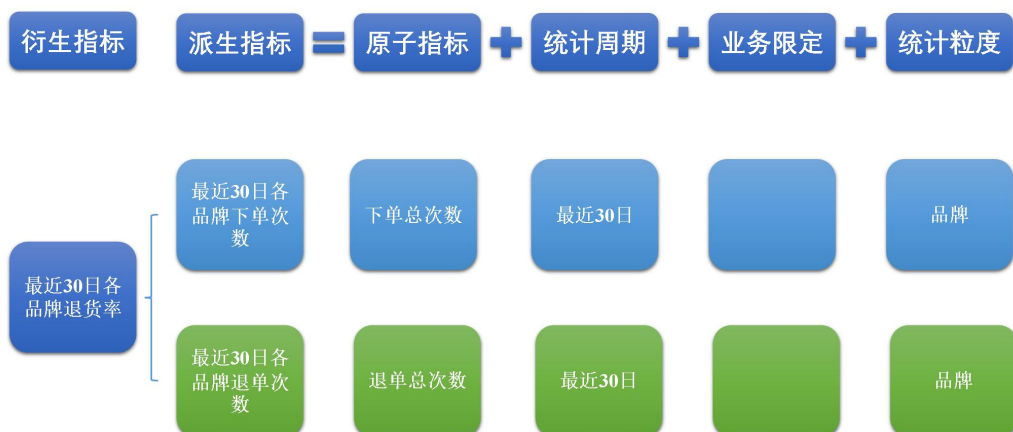
让天下没有难学的技术

与原子指标不同，派生指标通常会对应实际的统计需求。请从图中的例子中，体会指标定义标准化的含义。

（3）衍生指标

衍生指标是在一个或多个派生指标的基础上，通过各种逻辑运算复合而成的。例如比率、比例等类型的指标。衍生指标也会对应实际的统计需求。

衍生指标



让天下没有难学的技术

2) 指标体系对于数仓建模的意义

通过上述两个具体的案例可以看出，绝大多数的统计需求，都可以使用原子指标、派生指标以及衍生指标这套标准去定义。同时能够发现这些统计需求都直接的或间接的对应一个或者是多个派生指标。

当统计需求足够多时，必然会出现部分统计需求对应的派生指标相同的情况。这种情况下，我们就可以考虑将这些公共的派生指标保存下来，这样做的主要目的就是减少重复计算，提高数据的复用性。

这些公共的派生指标统一保存在数据仓库的 DWS 层。因此 DWS 层设计，就可以参考我们根据现有的统计需求整理出的派生指标。

按照上述标准整理出的指标体系如下：

（1）思维导图版



指标体系.xmind

（2）PDF 版



指标体系.pdf

从上述指标体系中抽取出来的所有派生指标见如下表格。



派生指标.xlsx

5.2.4 维度模型设计

维度模型的设计参照上述得到的业务总线矩阵即可。事实表存储在 DWD 层，维度表存储在 DIM 层。

5.2.5 汇总模型设计

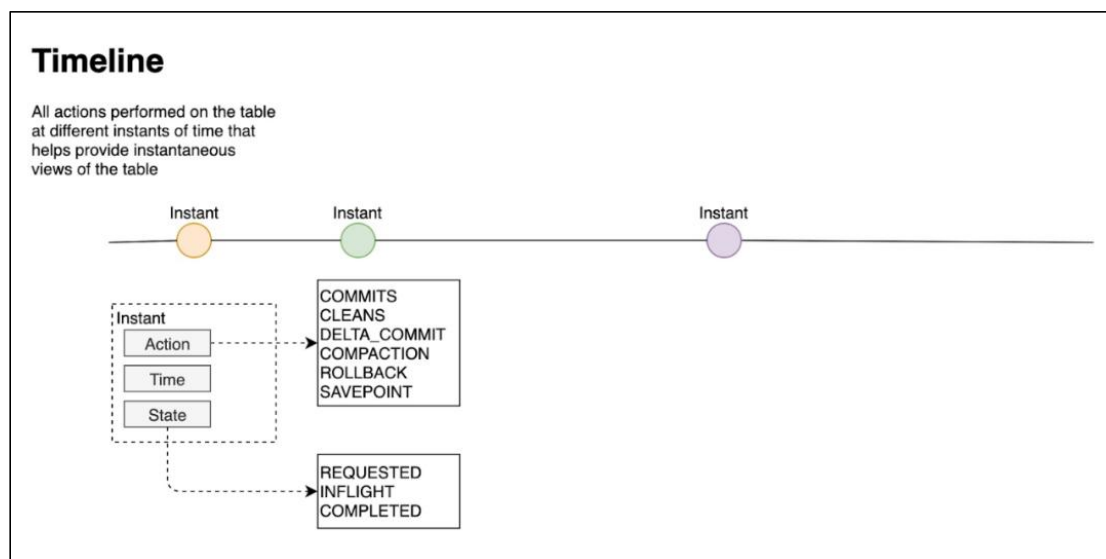
汇总模型的设计参考上述整理出的指标体系（主要是派生指标）即可。汇总表与派生指标的对应关系是，**一张汇总表通常包含**业务过程相同、统计周期相同、统计粒度相同的**多个派生指标**。请思考：汇总表与事实表的对应关系是？

第 6 章 Hudi 核心概念介绍

更为详细的使用文档请参考《尚硅谷大数据之 Hudi》。

6.1 基本概念

6.1.1 时间轴（TimeLine）



Hudi 的核心是维护表上在不同的**即时时间（instants）**执行的所有操作的**时间轴（timeline）**，这有助于提供表的即时视图，同时还有效地支持按到达顺序检索数据。一个 instant 由以下三个部分组成：

1) Instant action: 在表上执行的操作类型

- **COMMITTS**: 一次 commit 表示将一批数据原子性地写入一个表。
- **CLEANS**: 清除表中不再需要的旧版本文件的后台活动。
- **DELTA_COMMIT**: 增量提交指的是将一批数据原子性地写入一个 MergeOnRead 类型的表，其中部分或所有数据可以写入增量日志。
- **COMPACTION**: 合并 Hudi 内部差异数据结构的后台活动，例如:将更新操作从基于行的 log 日志文件合并到列式存储的数据文件。在内部，COMPACTION 体现为 timeline 上的特殊提交。
- **ROLLBACK**: 表示当 commit/delta_commit 不成功时进行回滚，其会删除在写入过程中产生的部分文件。
- **SAVEPOINT**: 将某些文件组标记为已保存，以便其不会被删除。在发生灾难需要

恢复数据的情况下，它有助于将数据集还原到时间轴上的某个点。

2) Instant time

通常是一个时间戳（例如：20190117010349），它按照动作开始时间的顺序单调增加。

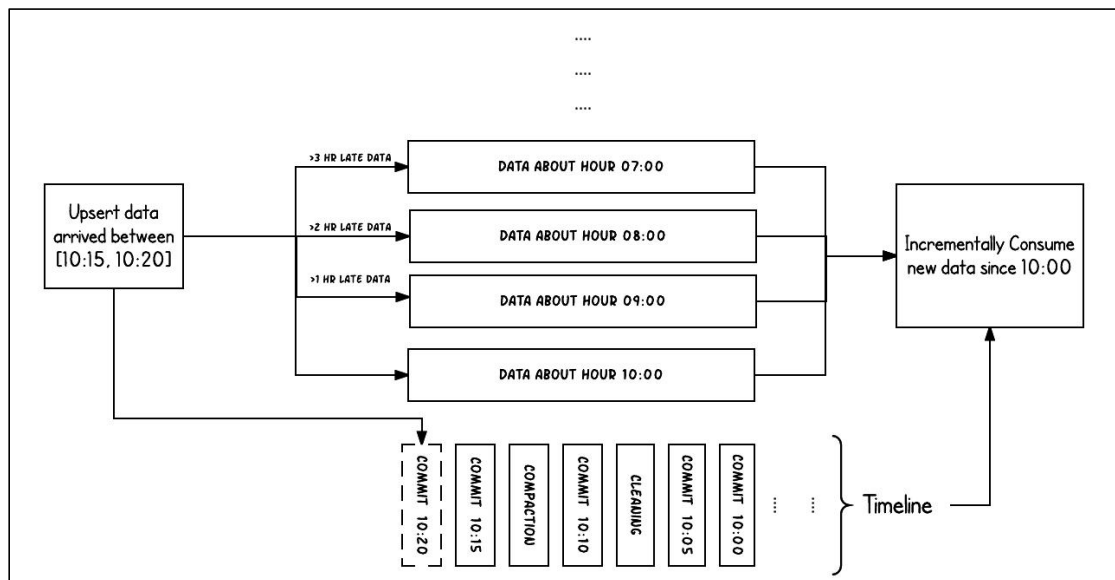
3) State

- REQUESTED: 表示某个 action 已经调度，但尚未执行。
- INFLIGHT: 表示 action 当前正在执行。
- COMPLETED: 表示 timeline 上的 action 已经完成。

4) 两个时间概念

区分两个重要的时间概念：

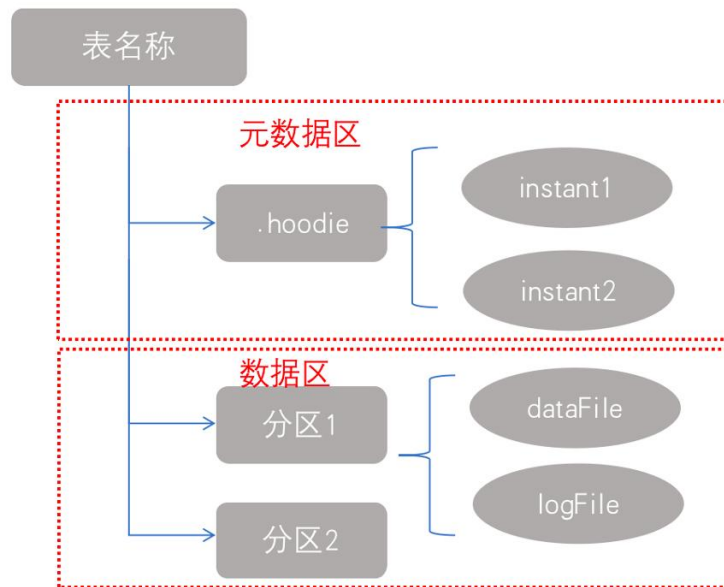
- Arrival time: 数据到达 Hudi 的时间，commit time。
- Event time: record 中记录的时间。



上图中采用时间（小时）作为分区字段，从 10:00 开始陆续产生各种 commits，10:20 来了一条 9:00 的数据，根据 event time 该数据仍然可以落到 9:00 对应的分区，通过 timeline 直接消费 10:00（commit time）之后的增量更新（只消费有新 commits 的 group），那么这条延迟的数据仍然可以被消费到。

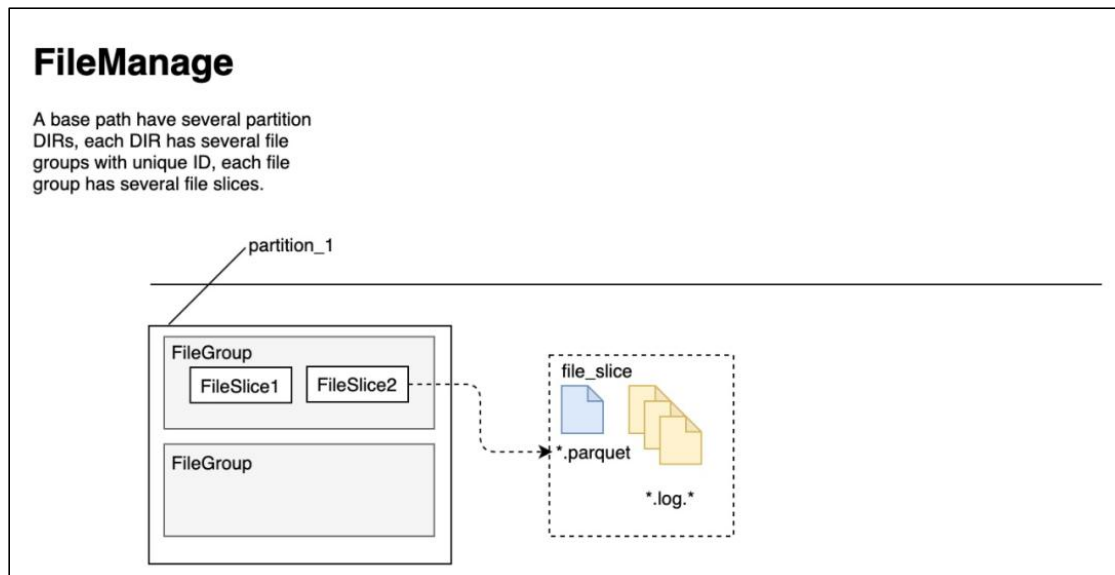
6.1.2 文件布局（File Layout）

Hudi 将一个表映射为如下文件结构



Hudi 存储分为两个部分：

- （1）元数据：.hoodie 目录对应着表的元数据信息，包括表的版本管理（Timeline）、归档目录（存放过时的 instant 也就是版本），一个 instant 记录了一次提交（commit）的行为、时间戳和状态，Hudi 以时间轴的形式维护了在数据集上执行的所有操作的元数据；
- （2）数据：和 hive 一样，以分区方式存放数据；分区里面存放着 Base File（.parquet）和 Log File（.log.*）；



- （1）Hudi 将数据表组织成分布式文件系统基本路径（basepath）下的目录结构
- （2）表被划分为多个分区，这些分区是包含该分区的数据文件的文件夹，非常类似于 Hive 表
- （3）在每个分区中，文件被组织成文件组，由文件 ID 唯一标识

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

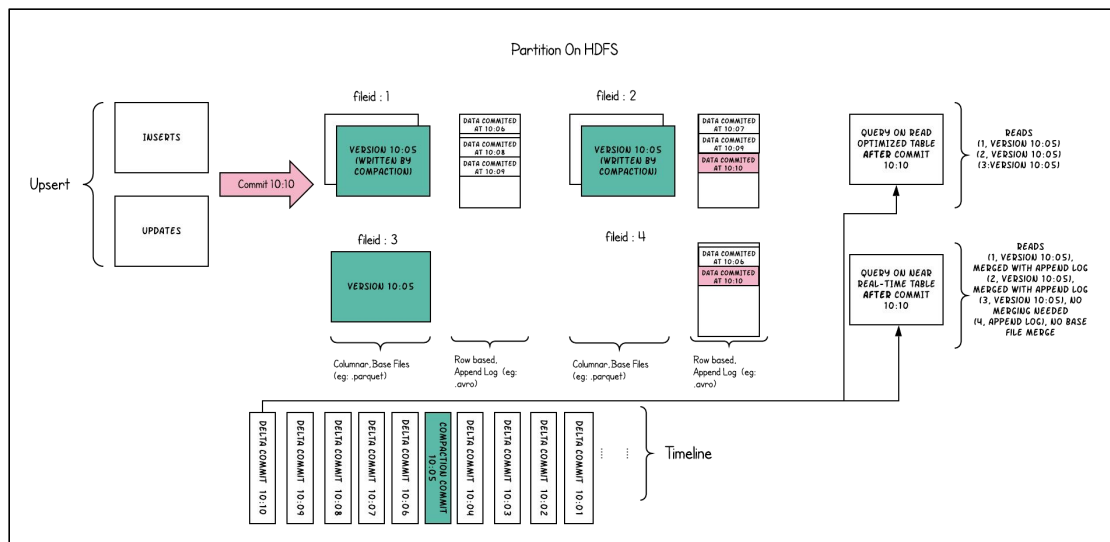
(4) 每个文件组包含几个文件片（FileSlice）

(5) 每个文件片包含：

- 一个基本文件(.parquet)：在某个 commit/compaction 即时时间（instant time）生成的（MOR 可能没有）
- 多个日志文件(.log.*), 这些日志文件包含自生成基本文件以来对基本文件的插入/更新（COW 没有）

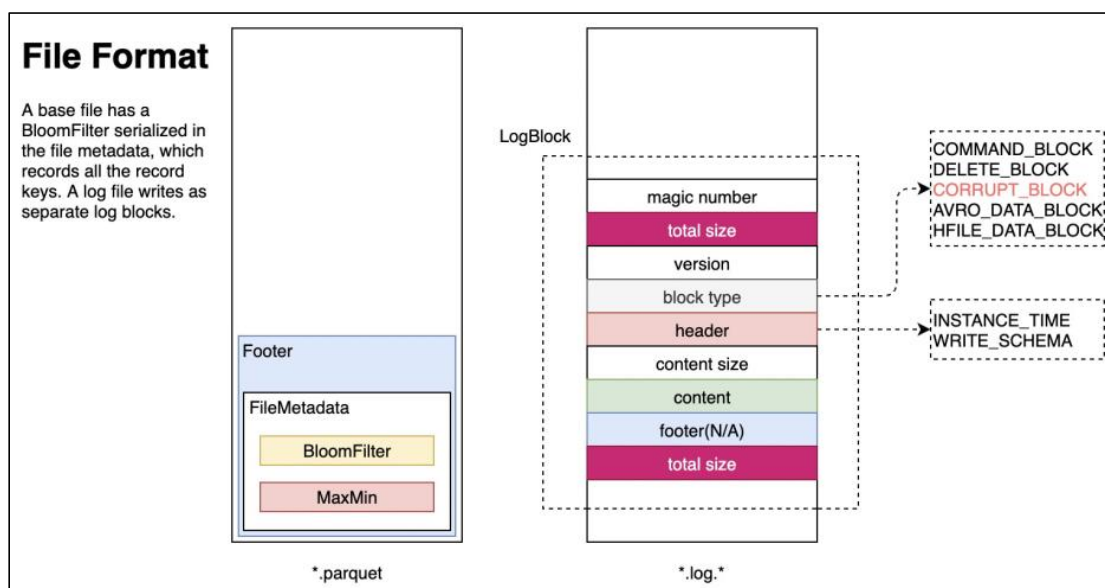
(6) Hudi 采用了多版本并发控制(Multiversion Concurrency Control, MVCC)

- compaction 操作：合并日志和基本文件以产生新的文件片
- clean 操作：清除不使用的/旧的文件片以回收文件系统上的空间



(7) Hudi 的 base file（parquet 文件）在 footer 的 meta 去记录了 record key 组成的 BloomFilter，用于在 file based index 的实现中实现高效率的 key contains 检测。只有不在 BloomFilter 的 key 才需要扫描整个文件消灭假阳。

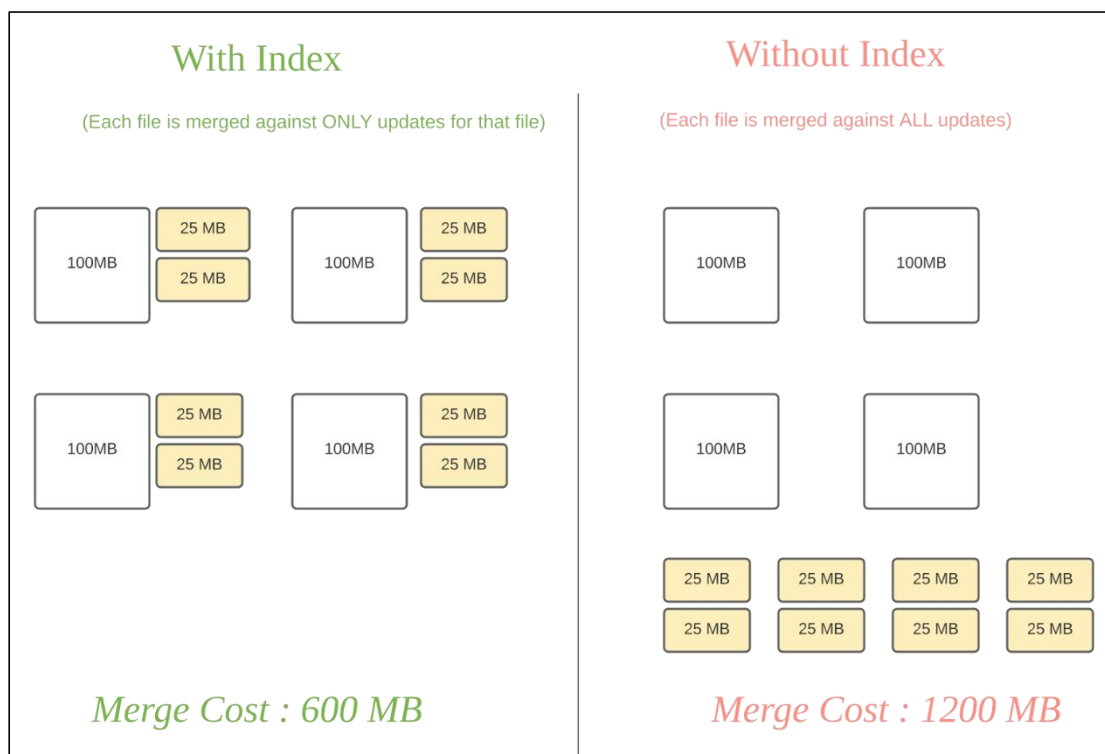
(8) Hudi 的 log（avro 文件）是自己编码的，通过积攒数据 buffer 以 LogBlock 为单位写出，每个 LogBlock 包含 magic number、size、content、footer 等信息，用于数据读、校验和过滤。



6.1.3 索引（Index）

1) 原理

Hudi 通过索引机制提供高效的 upserts, 具体是将给定的 hoodie key(record key + partition path)与文件 id（文件组）建立唯一映射。这种映射关系，数据第一次写入文件后保持不变，所以，一个 FileGroup 包含了一批 record 的所有版本记录。Index 用于区分消息是 INSERT 还是 UPDATE。



Hudi 为了消除不必要的读写，引入了索引的实现。在有了索引之后，更新的数据可以快速被定位到对应的 File Group。上图为例，白色是基本文件，黄色是更新数据，有了索引机制，可以做到：避免读取不需要的文件、避免更新不必要的文件、无需将更新数据与历史数据做分布式关联，只需要在 File Group 内做合并。

2) 索引选项

Index 类型	原理	优点	缺点
Bloom Index	默认配置，使用布隆过滤器来判断记录存在与否，也可选使用 record key 的范围裁剪需要的文件	效率高，不依赖外部系统，数据和索引保持一致性	因假阳性问题，还需回溯原文件再查找一遍
Simple Index	把 update/delete 操作的新数据和老数据进行 join	实现最简单，无需额外的资源	性能比较差
HBase Index	把 index 存放在 HBase 里面。在插入 File Group 定位阶段所有 task 向 HBase 发送 Batch Get 请求，获取 Record Key 的 Mapping 信息	对于小批次的 keys，查询效率高	需要外部的系统，增加了运维压力
Flink State-based Index	HUDI 在 0.8.0 版本中实现的 Flink witer，采用了 Flink 的 state 作为底层的 index 存储，每个 records 在写入之前都会先计算目标 bucket ID。	不同于 BloomFilter Index，避免了每次重复的文件 index 查找	

注意：Flink 只有一种 state based index，其他 index 是 Spark 可选配置。

3) 全局索引与非全局索引

global index 里面存放了一张表里所有 record 的 key，而 non-global index 是每个 partition 都有一个对应的 index，里面只存放了本 partition 的 key。所以如果用户使用 non-global index，就必须保证同一个 key 的 record 不会出现在多个 partition 里面。

从 index 的维护成本和写入性能的角度考虑，维护一个 global index 的难度更大，对写入性能的影响也更大，所以需要 non-global index。

bloom 和 simple index 都有全局选项：

- hoodie.index.type=GLOBAL_BLOOM
- hoodie.index.type=GLOBAL_SIMPLE
- HBase 索引本质上是一个全局索引

4) 索引的选择

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

（1）普通索引：主要用于非分区表和分区不会发生分区列值变更的表。当然如果你不关心多分区主键重复的情况也是可以使用的。他的优势是只会加载 `upsert` 数据中的分区下的每个文件中的索引，相对于全局索引需要扫描的文件少。并且索引只会命中当前分区的 `fileid` 文件，需要重写的快照也少相对全局索引高效。但是某些情况下我们的设置的分区列的值就是会变那么必须要使用全局索引保证数据不重复，这样 `upsert` 写入速度就会慢一些。其实对于非分区表他就是个分区列值不会变且只有一个分区的表，很适合普通索引，如果非分区表硬要用全局索引其实和普通索引性能和效果是一样的。

（2）全局索引：分区表场景要考虑分区值变更，需要加载所有分区文件的索引比普通索引慢。

（3）布隆索引：加载 `fileid` 文件页脚布隆过滤器，加载少量数据数据就能判断数据是否存在文件存在。缺点是有一定的误判，但是 `merge` 机制可以避免重复数据写入。`parquet` 文件多会影响索引加载速度。适合没有分区变更和非分区表。主键如果是类似自增的主键布隆索引可以提供更高的性能，因为布隆索引记录的有最大 `key` 和最小 `key` 加速索引查找。

（4）全局布隆索引：解决分区变更场景，原理和布隆索引一样，在分区表中比普通布隆索引慢。

（5）简易索引：直接加载文件里的数据不会像布隆索引一样误判，但是加载的数据要比布隆索引要多，`left join` 关联的条数也要比布隆索引多。大多数场景没布隆索引高效，但是极端情况命中所有的 `parquet` 文件，那么此时还不如使用简易索引加载所有文件里的数据进行判断。

（6）全局简易索引：解决分区变更场景，原理和简易索引一样，在分区表中比普通简易索引慢。建议优先使用全局布隆索引。

（7）HBase 索引：不受分区变更场景的影响，操作算子要比布隆索引少，在大量的分区和文件的场景中比布隆全局索引高效。因为每条数据都要查询 `hbase`，`upsert` 数据量很大会对 `hbase` 有负载的压力需要考虑 `hbase` 集群承受压力，适合微批分区表的写入场景。在非分区表中数量不大文件也少，速度和布隆索引差不多，这种情况建议用布隆索引。

（8）内存索引：用于测试不适合生产环境。

6.1.4 表类型（Table Types）

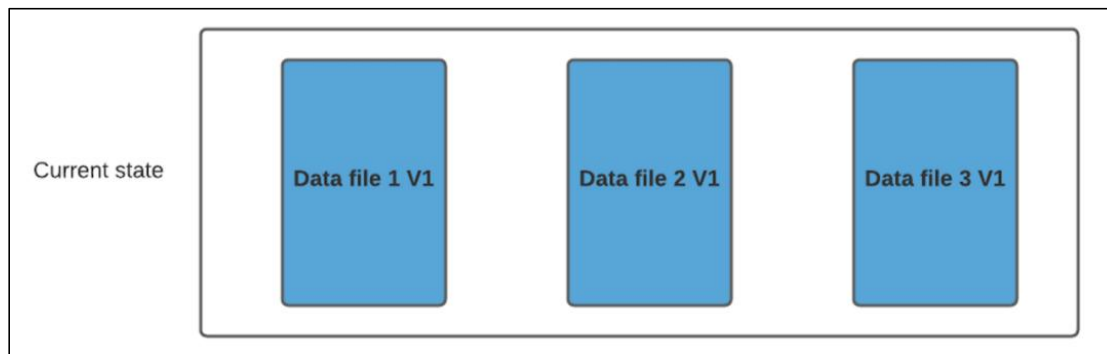
1) Copy On Write

在 COW 表中，只有数据文件/基本文件（`.parquet`），没有增量日志文件（`.log.*`）。

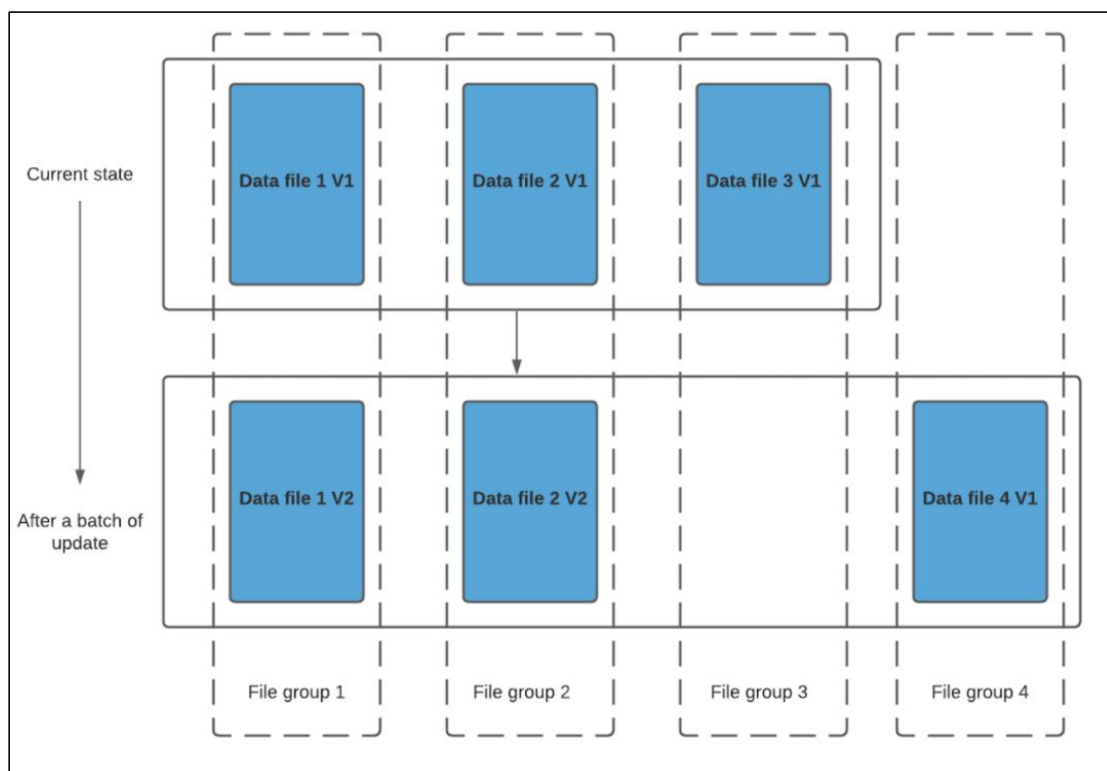
更多 [Java - 大数据 - 前端 - python 人工智能资料下载](#)，可百度访问：[尚硅谷官网](#)

对每一个新批次写入都将创建相应数据文件的新版本（新的 FileSlice），新版本文件包括旧版本文件的记录以及来自传入批次的记录（全量最新）。

假设我们有 3 个文件组，其中包含如下数据文件。



我们进行一批新的写入，在索引后，我们发现这些记录与 File group 1 和 File group 2 匹配，然后有新的插入，我们将为其创建一个新的文件组（File group 4）。



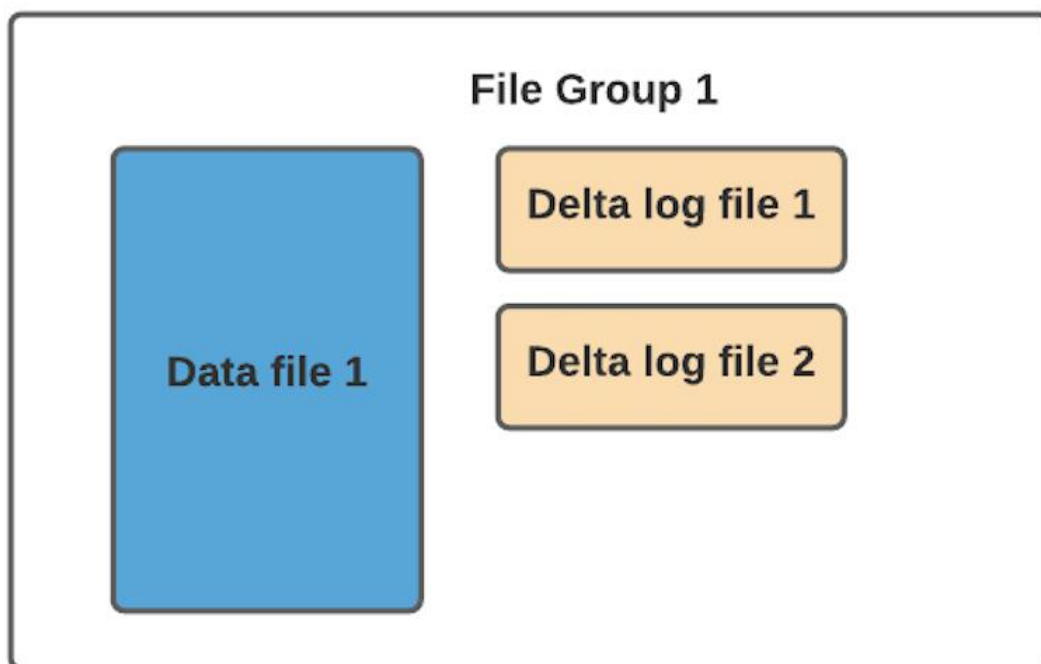
因此 data_file1 和 data_file2 都将创建更新的版本，data_file1 V2 是 data_file1 V1 的内容与 data_file1 中传入批次匹配记录的记录合并。

由于在写入期间进行合并，COW 会产生一些写入延迟。但是 COW 的优势在于它的简单性，不需要其他表服务（如压缩），也相对容易调试。

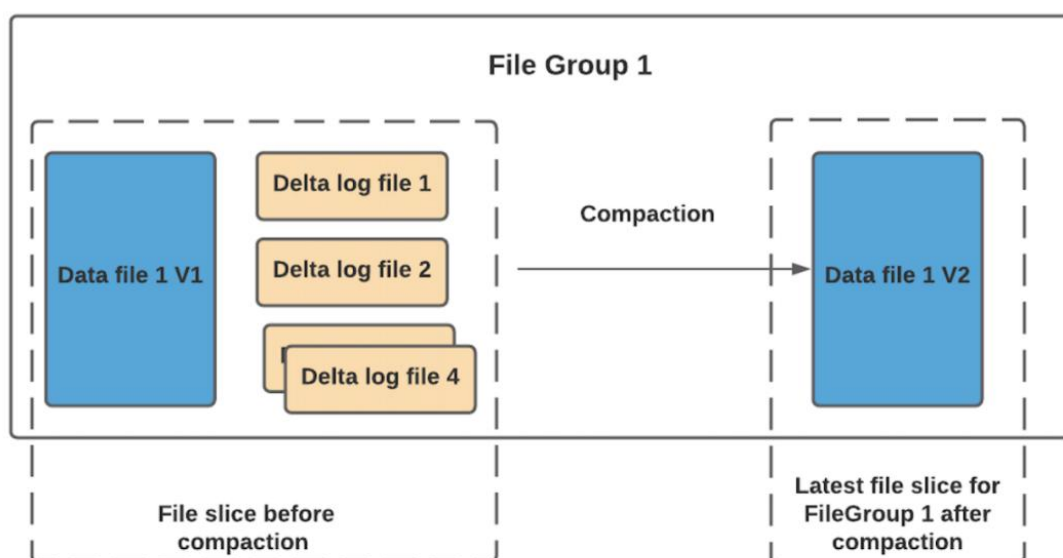
2) Merge On Read

MOR 表中，包含 列存的基本文件（.parquet）和行存的增量日志文件（基于行的 avro 格式，.log.*）。

顾名思义，MOR 表的合并成本在读取端。因此在写入期间我们不会合并或创建较新的数据文件版本。标记/索引完成后，对于具有要更新记录的现有数据文件，Hudi 创建增量日志文件并适当命名它们，以便它们都属于一个文件组。



读取端将实时合并基本文件及其各自的增量日志文件。每次的读取延迟都比较高（因为查询时进行合并），所以 Hudi 使用压缩机制来将数据文件和日志文件合并在一起并创建更新版本的数据文件。



用户可以选择内联或异步模式运行压缩。Hudi 也提供了不同的压缩策略供用户选择，最常用的一种是基于提交的数量。例如可以将压缩的最大增量日志配置为 4。这意味着在进行 4 次增量写入后，将对数据文件进行压缩并创建更新版本的数据文件。压缩完成后，读取端只需要读取最新的数据文件，而不必关心旧版本文件。

MOR 表的写入行为，依据 index 的不同会有细微的差别：

- 对于 BloomFilter 这种无法对 log file 生成 index 的索引方案，对于 INSERT 消息仍然会写 base file（parquet format），只有 UPDATE 消息会 append log 文件（因为 base file 已经记录了该 UPDATE 消息的 FileGroup ID）。
- 对于可以对 log file 生成 index 的索引方案，例如 Flink writer 中基于 state 的索引，每次写入都是 log format，并且会不断追加和 roll over。

3) COW 与 MOR 的对比

	CopyOnWrite	MergeOnRead
数据延迟	高	低
查询延迟	低	高
Update(I/O) 更新成本	高（重写整个 Parquet 文件）	低（追加到增量日志）
Parquet 文件大小	低（更新成本 I/O 高）	较大（低更新成本）
写放大	大	低（取决于压缩策略）

6.1.5 查询类型（Query Types）

Hudi 支持如下三种查询类型：

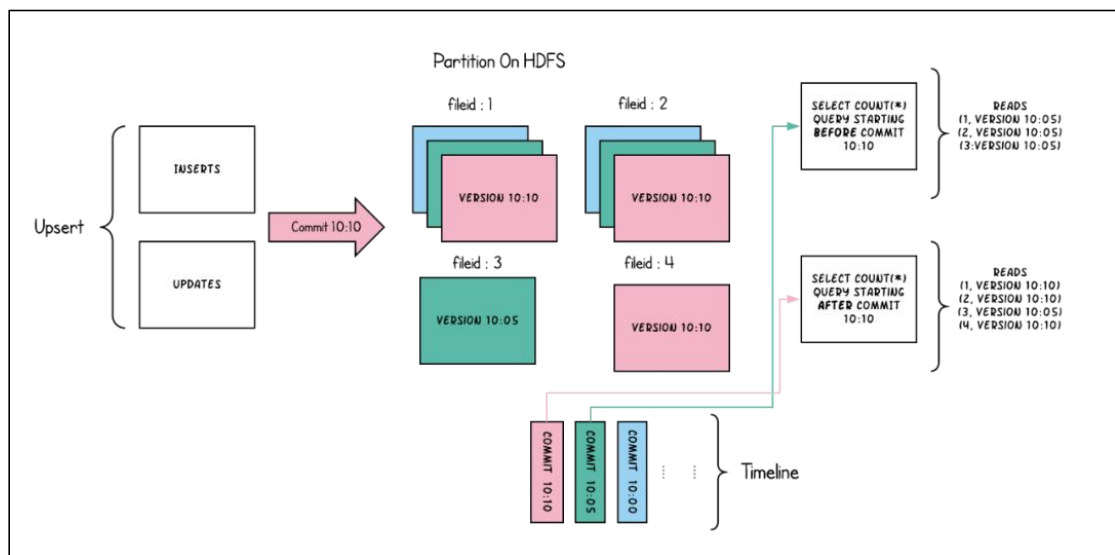
1) Snapshot Queries

快照查询，可以查询指定 commit/delta commit 即时操作后表的最新快照。

在读时合并（MOR）表的情况下，它通过即时合并最新文件片的基本文件和增量文件来提供近实时表（几分钟）。

对于写时复制（COW），它可以替代现有的 parquet 表（或相同基本文件类型的表），同时提供 upsert/delete 和其他写入方面的功能，可以理解为查询最新版本的 Parquet 数据文件。

下图是 COW 的快照查询：



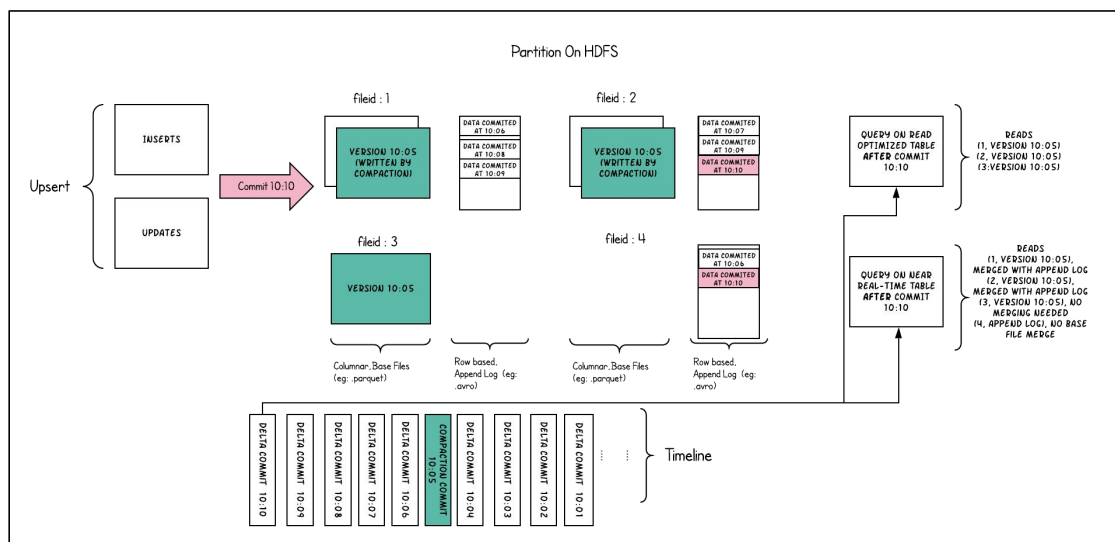
2) Incremental Queries

增量查询，可以查询给定 commit/delta commit 即时操作以来新写入的数据。有效的提供变更流来启用增量数据管道。

3) Read Optimized Queries

读优化查询，可查看给定的 commit/compact 即时操作的表的最新快照。仅将最新文件片的基本/列文件暴露给查询，并保证与非 Hudi 表相同的列查询性能。

下图是 MOR 表的快照查询与读优化查询的对比：



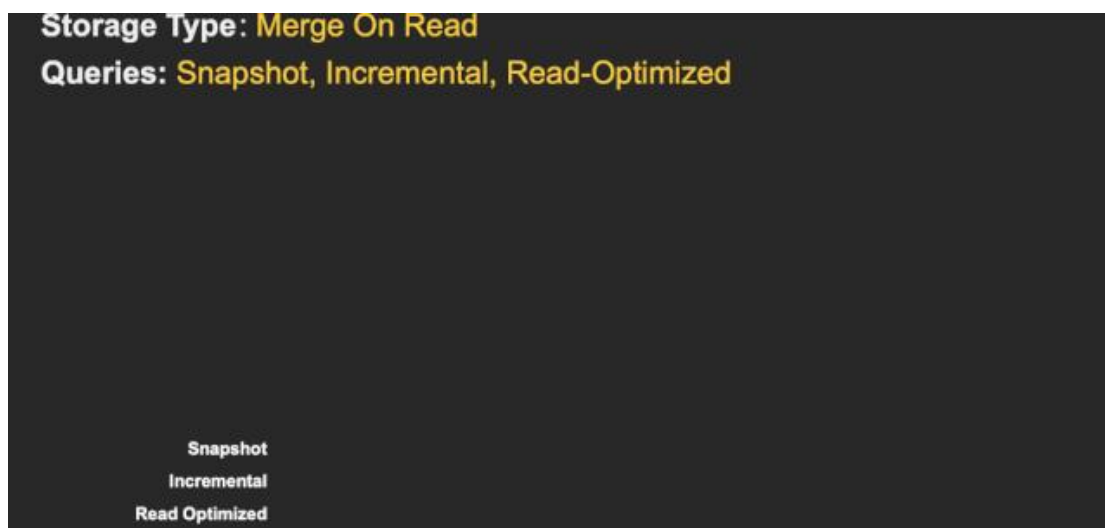
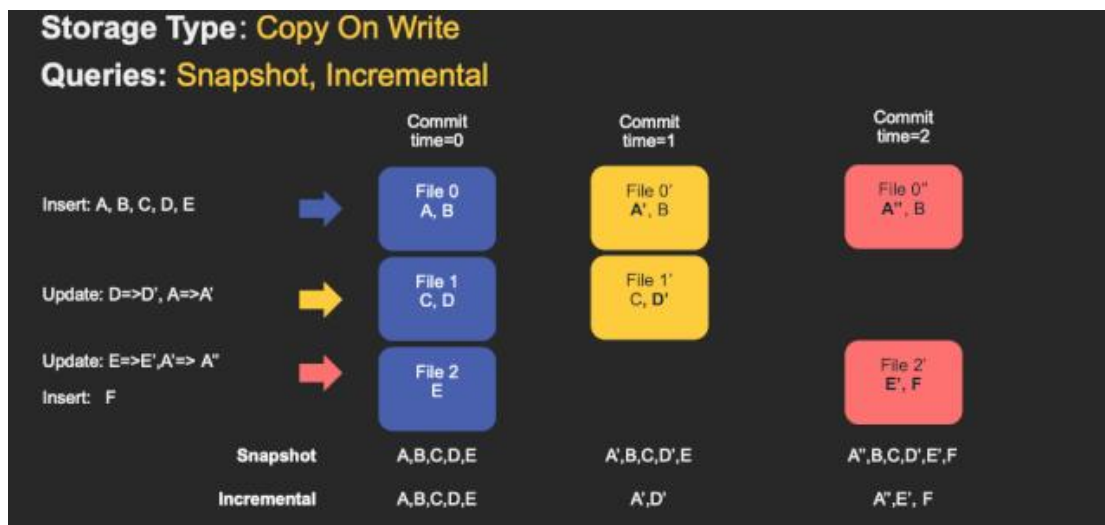
Read Optimized Queries 是对 Merge On Read 表类型快照查询的优化。

	Snapshot	Read Optimized
数据延迟	低	高

查询延迟	高（合并列式基础文件+行式增量日志文件）	低(原始列式基础文件)
------	----------------------	-------------

4) 不同表支持的查询类型

Table Type	Supported Query types
Copy On Write	Snapshot Queries + Incremental Queries
Merge On Read	Snapshot Queries + Incremental Queries + Read Optimized Queries



6.2 数据写

6.2.1 写操作

(1) UPSERT: 默认行为，数据先通过 index 打标(INSERT/UPDATE)，有一些启发式算法决定消息的组织以优化文件的大小 => CDC 导入

(2) INSERT: 跳过 index, 写入效率更高 => Log Deduplication

(3) BULK_INSERT: 写排序, 对大数据量的 Hudi 表初始化友好, 对文件大小的限制 best effort (写 HFile)

6.2.2 写流程 (UPSERT)

1) Copy On Write

(1) 先对 records 按照 record key 去重

(2) 首先对这批数据创建索引 (HoodieKey => HoodieRecordLocation); 通过索引区分哪些 records 是 update, 哪些 records 是 insert (key 第一次写入)

(3) 对于 update 消息, 会直接找到对应 key 所在的最新 FileSlice 的 base 文件, 并做 merge 后写新的 base file (新的 FileSlice)

(4) 对于 insert 消息, 会扫描当前 partition 的所有 SmallFile (小于一定大小的 base file), 然后 merge 写新的 FileSlice; 如果没有 SmallFile, 直接写新的 FileGroup + FileSlice

2) Merge On Read

(1) 先对 records 按照 record key 去重 (可选)

(2) 首先对这批数据创建索引 (HoodieKey => HoodieRecordLocation); 通过索引区分哪些 records 是 update, 哪些 records 是 insert (key 第一次写入)

(3) 如果是 insert 消息, 如果 log file 不可建索引 (默认), 会尝试 merge 分区内最小的 base file (不包含 log file 的 FileSlice), 生成新的 FileSlice; 如果没有 base file 就新写一个 FileGroup + FileSlice + base file; 如果 log file 可建索引, 尝试 append 小的 log file, 如果没有就新写一个 FileGroup + FileSlice + base file

(4) 如果是 update 消息, 写对应的 file group + file slice, 直接 append 最新的 log file (如果碰巧是当前最小的小文件, 会 merge base file, 生成新的 file slice)

(5) log file 大小达到阈值会 roll over 一个新的

6.2.3 写流程 (INSERT)

1) Copy On Write

(1) 先对 records 按照 record key 去重 (可选)

(2) 不会创建 Index

(3) 如果有小的 base file 文件, merge base file, 生成新的 FileSlice + base file, 否则

直接写新的 FileSlice + base file

2) Merge On Read

- (1) 先对 records 按照 record key 去重（可选）
- (2) 不会创建 Index
- (3) 如果 log file 可索引，并且有小的 FileSlice，尝试追加或写最新的 log file；如果 log file 不可索引，写一个新的 FileSlice + base file

6.2.4 写流程（INSERT OVERWRITE）

在同一分区中创建新的文件组集。现有的文件组被标记为 "删除"。根据新记录的数量创建新的文件组

1) COW

在插入分区之前	插入相同数量的记录覆盖	插入覆盖更多的记录	插入重写 1 条记录
分区包含 file1-t0.parquet, file2-t0.parquet。	分区将添加 file3-t1.parquet, file4-t1.parquet。 file1, file2 在 t1 后的元数据中被标记为无效。	分区将添加 file3-t1.parquet, file4-t1.parquet, file5-t1.parquet, ..., fileN-t1.parquet。 file1, file2 在 t1 后的元数据中被标记为无效	分区将添加 file3-t1.parquet。 file1, file2 在 t1 后的元数据中被标记为无效。

2) MOR

在插入分区之前	插入相同数量的记录覆盖	插入覆盖更多的记录	插入重写 1 条记录
分区包含 file1-t0.parquet, file2-t0.parquet。 .file1-t00.log	file3-t1.parquet, file4-t1.parquet。 file1, file2 在 t1 后的元数据中被标记为无效。	file3-t1.parquet, file4-t1.parquet ... fileN-t1.parquet file1, file2 在 t1 后的元数据中被标记为无效	分区将添加 file3-t1.parquet。 file1, file2 在 t1 后的元数据中被标记为无效。

3) 优点

- (1) COW 和 MOR 在执行方面非常相似。不干扰 MOR 的 compaction。
- (2) 减少 parquet 文件大小。
- (3) 不需要更新关键路径中的外部索引。索引实现可以检查文件组是否无效（类似于在 HBaseIndex 中检查 commit 是否无效的方式）。
- (4) 可以扩展清理策略，在一定的时间窗口后删除旧文件组。

更多 [Java - 大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

4) 缺点

(1) 需要转发以前提交的元数据。

- 在 t1, 比如 file1 被标记为无效, 我们在 t1.commit 中存储 "invalidFiles=file1"(或者在 MOR 中存储 deltacommit)
- 在 t2, 比如 file2 也被标记为无效。我们转发之前的文件, 并在 t2.commit 中标记 "invalidFiles=file1, file2" (或 MOR 的 deltacommit)

(2) 忽略磁盘中存在的 parquet 文件也是 Hudi 的一个新行为, 可能容易出错, 我们必须认识到新的行为, 并更新文件系统的所有视图来忽略它们。这一点可能会在实现其他功能时造成问题。

6.2.5 Key 生成策略

用来生成 HoodieKey (record key + partition path), 目前支持以下策略:

- 支持多个字段组合 record keys
- 支持多个字段组合的 partition path (可定制时间格式, Hive style path name)
- 非分区表

6.2.6 删除策略

1) 逻辑删: 将 value 字段全部标记为 null

2) 物理删:

(1) 通过 OPERATION_OPT_KEY 删除所有的输入记录

(2) 配置 PAYLOAD_CLASS_OPT_KEY = org.apache.hudi.EmptyHoodieRecordPayload

删除所有的输入记录

(3) 在输入记录添加字段: _hoodie_is_deleted

6.2.7 总结

通过对写流程的梳理可以了解到 Apache Hudi 相对于其他数据湖方案的核心优势:

(1) 写入过程充分优化了文件存储的小文件问题, Copy On Write 写会一直将一个 bucket (FileGroup) 的 base 文件写到设定的阈值大小才会划分新的 bucket; Merge On Read 写在同一个 bucket 中, log file 也是一直 append 直到大小超过设定的阈值 roll over。

(2) 对 UPDATE 和 DELETE 的支持非常高效, 一条 record 的整个生命周期操作都发生在同一个 bucket, 不仅减少小文件数量, 也提升了数据读取的效率 (不必要的 join 和

更多 [Java - 大数据 - 前端 - python 人工智能资料下载](#), 可百度访问: [尚硅谷官网](#)

merge)。

6.3 数据读

6.3.1 Snapshot 读

读取所有 partiiton 下每个 FileGroup 最新的 FileSlice 中的文件，Copy On Write 表读 parquet 文件，Merge On Read 表读 parquet + log 文件

6.3.2 Incremantal 读

https://hudi.apache.org/docs/querying_data.html#spark-incr-query

当前的 Spark data source 可以指定消费的起始和结束 commit 时间，读取 commit 增量的数据集。但是内部的实现不够高效：拉取每个 commit 的全部目标文件再按照系统字段 `_hoodie_commit_time_` apply 过滤条件。

6.3.3 Streaming 读

0.8.0 版本的 HUDI Flink writer 支持实时的增量订阅，可用于同步 CDC 数据，日常的数据同步 ETL pipeline。Flink 的 streaming 读做到了真正的流式读取，source 定期监控新增的改动文件，将读取任务下派给读 task。

6.4 Compaction

- (1) 没有 base file: 走 copy on write insert 流程，直接 merge 所有的 log file 并写 base file
- (2) 有 base file: 走 copy on write upsert 流程，先读 log file 建 index，再读 base file，最后读 log file 写新的 base file

Flink 和 Spark streaming 的 writer 都可以 apply 异步的 compaction 策略，按照间隔 commits 数或者时间来触发 compaction 任务，在独立的 pipeline 中执行。

第 7 章 湖仓一体项目的优势

7.1 使用 Hudi 搭建数仓

从上一节我们对 Hudi 的简单介绍中可以知道，使用 Hudi 构建湖仓一体，有如下优势：

- （1）沿用传统数仓中的分层思想
- （2）不需要做复杂的拉链表
- （3）不需要做全量增量表，延迟低，近似为实时的离线数仓
- （4）Hudi 自动解决小文件问题
- （5）所有层的数据均可在 Hive 中通过 Hive 的同步表进行查询（底层只有一份数据）

7.2 使用 HiveCatalog 持久化元数据

HiveCatalog 作为表元数据持久化的介质，在生产环境我们一般采用 HiveCatalog 来管理元数据，这样的好处是不需要重复使用 DDL 创建表，只需要关心业务逻辑的 SQL，简化了开发的流程，可以节省很多时间。

在 Flink 中创建表，表的元数据可以保存到 hive 的元数据中，元数据不会丢失。

Flink 将表的元数据保存在 hive 的元数据中，在 hive 中可以看到 Flink 的表，但是不能对 flink 的表进行查询。

需要在启动 Flink SQL Client 时候显式指定配置文件，启动命令见 6.3.3。

将 sql-client-init.sql 文件放在/opt/module/flink/conf 目录下。内容如下：

```
#sql-client 配置 sql 文件
vim conf/sql-client-init.sql

CREATE CATALOG hive_catalog WITH (
    'type' = 'hive',
    'default-database' = 'default',
    'hive-conf-dir' = '/opt/module/hive/conf',
    'hadoop-conf-dir' = '/opt/module/hadoop/etc/hadoop'
);
-- set the HiveCatalog as the current catalog of the session
USE CATALOG hive_catalog;
load module hive with ('hive-version'='3.1.2');
```

7.3 使用 Flink SQL 进行开发

基于 yarn-session 的 SQL-client 可以在提交任务时候动态申请 TM 个数。

```
cd /opt/module/flink
#基于 yarn 模式的 sql-client (TM 个数动态申请)，启动命令：
bin/yarn-session.sh -d
bin/sql-client.sh    embedded    -i    conf/sql-client-init.sql    -s
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
yarn-session
```

#进入 SQL client 之后可以设置展示模式，分别有 table, tableau, changelog 三种模式。

```
SET sql-client.execution.result-mode=tableau;
```

7.4 引入 Hive 的系统函数作为 Flink 的内置函数

在 Flink SQL Client 中执行

```
load module hive with ('hive-version'='3.1.2');
```

可以将 Hive 的 UDF 加载为 Flink 的一个 module。

使用

```
SHOW MODULES;
```

命令可以查看所有模块的状态

这样做的好处就是，原本 Flink 中不支持 Hive 的语法，例如 split，在加载 hive 模块之后就可以进行使用了。

有的同学会有如下疑问，如果一个函数（例如 concat）在 Hive 和 Flink 中都有，那么在加载时候会解析为哪个模块的呢？

注意：Flink 只会解析已经启用了的 Module。那么当两个 Module 中出现两个同名的函数时，会有以下三种情况：

1. 如果两个 Module 都启用的话，Flink 会根据加载 Module 的顺序进行解析，结果就是会使用顺序为第一个的 Module 的 UDF

2. 如果只有一个 Module 启用的话，Flink 就只会从启用的 Module 解析 UDF

3. 如果两个 Module 都没有启用，Flink 就无法解析这个 UDF

需要补充说明的是：

1. 如果出现第一种情况时，用户也可以改变使用 Module 的顺序。比如用户可以使用

```
USE MODULE hive,core
```

将 Hive Module 设为第一个使用及解析的 Module。

2. 另外，用户可以使用

```
USE MODULES hive
```

去禁用默认的 core Module，注意，禁用不是卸载 Module，用户之后还可以再次启用 Module，并且使用

```
USE MODULES core
```

命令去将 core Module 设置为启用的。

3. 如果使用未加载的 Module，则会直接抛出异常。

4. 禁用和卸载 Module 的区别在于禁用依然会在 TableEnvironment 保留 Module，用户

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

依然可以使用 list 命令看到禁用的 Module。

7.5 集成 StreamX 进行 Flink SQL 调度

后期可以集成 StreamX 对 Flink SQL 进行管理和调度

第 8 章 湖仓一体环境准备

8.1 编译 Hudi 源码（了解）

本次项目我们采用 Hudi 进行开发，目前，Hudi 的最新版本是 0.12.0（发版日期为 2022-08-16）。我们在 Hudi 官网下载到的是 Hudi 的源码包，这就需要我们针对我们特定版本的 Hadoop 和 Hive 等组件进行编译。

数据采集部分的组件安装请参考

组件	版本
Java	1.8.212
Maven	3.6.3
Hadoop	3.1.3
Flume	1.9.0
Kafka	3.0.0
Hudi	0.12.0
Hive	3.1.2

8.1.1 安装 Maven

（1）上传 apache-maven-3.6.3-bin.tar.gz 到 /opt/software 目录，并解压更名

```
tar -zxvf apache-maven-3.6.3-bin.tar.gz -C /opt/module/  
mv apache-maven-3.6.3 maven
```

（2）添加环境变量到 /etc/profile 中

```
sudo vim /etc/profile  
#MAVEN_HOME  
export MAVEN_HOME=/opt/module/maven  
export PATH=$PATH:$MAVEN_HOME/bin
```

（3）测试安装结果

```
source /etc/profile  
mvn -v
```

2) 修改为阿里镜像

（1）修改 setting.xml，指定为阿里仓库地址

```
vim /opt/module/maven/conf/settings.xml  
  
<!-- 添加阿里云镜像 -->  
<mirror>  
  <id>nexus-aliyun</id>  
  <mirrorOf>central</mirrorOf>  
  <name>Nexus aliyun</name>  
  
<url>http://maven.aliyun.com/nexus/content/groups/public</url>  
</mirror>
```

8.1.2 编译 Hudi 源码

源码下载地址：<https://github.com/apache/hudi/releases/tag/release-0.12.0>



hudi-0.12.0.tar.g
z

解压 hudi 源码包到/opt/software 文件夹下

```
cd /opt/software
tar -zxvf hudi-release-0.12.0.tar.gz
```

8.1.3 修改 pom 文件

```
vim /opt/software/hudi-0.12.0/pom.xml
```

1) 新增 repository 加速依赖下载

```
<repository>
  <id>nexus-aliyun</id>
  <name>nexus-aliyun</name>

  <url>http://maven.aliyun.com/nexus/content/groups/public/</url>
  <releases>
    <enabled>true</enabled>
  </releases>
  <snapshots>
    <enabled>false</enabled>
  </snapshots>
</repository>
```

2) 修改依赖的组件版本

```
<hadoop.version>3.1.3</hadoop.version>
<hive.version>3.1.2</hive.version>

112      <joda.version>2.9.9</joda.version>
113      <!-- <hadoop.version>2.10.1</hadoop.version> -->
114      <hadoop.version>3.1.3</hadoop.version>
115      <hive.groupid>org.apache.hive</hive.groupid>
116      <!-- <hive.version>2.3.1</hive.version> -->
117      <hive.version>3.1.2</hive.version>
118      <presto.version>0.273</presto.version>
```

修改源码使其兼容 Hadoop3

Hudi 默认依赖的 hadoop2，要兼容 hadoop3，除了修改版本，还需要修改如下代码：

```
vim
/opt/software/hudi-0.12.0/hudi-common/src/main/java/org/apache/hu
di/common/table/log/block/HoodieParquetDataBlock.java
```

修改第 110 行，原先只有一个参数，添加第二个参数 `null`：

```
108     ByteArrayOutputStream baos = new ByteArrayOutputStream();
109
110     try (FSDDataOutputStream outputStream = new FSDDataOutputStream(baos, null)) {
111         try (HoodieParquetStreamWriter<IndexedRecord> parquetWriter = new HoodieParquetStreamWriter<
112             (outputStream, avroParquetConfig)) {
113             for (IndexedRecord record : records) {
114                 String recordKey = getRecordKey(record).orElse(null);
115                 parquetWriter.writeAvro(recordKey, record);
116             }
117             outputStream.flush();
118         }
119     }
```

8.1.4 手动安装 Kafka 依赖

有几个 kafka 的依赖需要手动安装，否则编译报错如下：

```
[ERROR] Failed to execute goal on project hudi-utilities_2.12: Could not resolve dependencies for project org.apache.hudi:hudi-utilities_2.12:jar:0.12.0: The following artifacts could not be resolved: io.confluent:kafka-avro-serializer:jar:5.3.4, io.confluent:common-config:jar:5.3.4, io.confluent:common-utils:jar:5.3.4, io.confluent:kafka-schema-registry-client:jar:5.3.4: Failure to find io.confluent:kafka-avro-serializer:jar:5.3.4 in https://maven.aliyun.com/repository/public was cached in the local repository, resolution will not be reattempted until the update interval of aliyunmaven has elapsed or updates are forced -> [Help 1]
```

下载 jar 包

通过网址下载：<http://packages.confluent.io/archive/5.3/confluent-5.3.4-2.12.zip>

解压后找到以下 jar 包，上传服务器 hadoop102 任意位置

jar 包放在了本课程的资料包中。

- common-config-5.3.4.jar
- common-utils-5.3.4.jar
- kafka-avro-serializer-5.3.4.jar
- kafka-schema-registry-client-5.3.4.jar

install 到 maven 本地仓库

```
mvn install:install-file -DgroupId=io.confluent -DartifactId=common-config -Dversion=5.3.4 -Dpackaging=jar -Dfile=./common-config-5.3.4.jar
mvn install:install-file -DgroupId=io.confluent -DartifactId=common-utils -Dversion=5.3.4 -Dpackaging=jar -Dfile=./common-utils-5.3.4.jar
mvn install:install-file -DgroupId=io.confluent
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

```
-DartifactId=kafka-avro-serializer -Dversion=5.3.4 -Dpackaging=jar
-Dfile=./kafka-avro-serializer-5.3.4.jar
mvn install:install-file -DgroupId=io.confluent
-DartifactId=kafka-schema-registry-client -Dversion=5.3.4
-Dpackaging=jar -Dfile=./kafka-schema-registry-client-5.3.4.jar
```

8.1.5 解决 spark 模块依赖冲突

修改了 Hive 版本为 3.1.2，其携带的 jetty 是 0.9.3，hudi 本身用的 0.9.4，存在依赖冲突，修改 hudi-spark-bundle 的 pom 文件，排除低版本 jetty，添加 hudi 指定版本的 jetty

```
vim /opt/software/hudi-0.12.0/packaging/hudi-spark-bundle/pom.xml
```

在 388 行的位置，修改如下（红色部分）：

```
<!-- Hive -->
<dependency>
  <groupId>${hive.groupid}</groupId>
  <artifactId>hive-service</artifactId>
  <version>${hive.version}</version>
  <scope>${spark.bundle.hive.scope}</scope>
  <exclusions>
    <exclusion>
      <artifactId>guava</artifactId>
      <groupId>com.google.guava</groupId>
    </exclusion>
    <exclusion>
      <groupId>org.eclipse.jetty</groupId>
      <artifactId>*</artifactId>
    </exclusion>
    <exclusion>
      <groupId>org.pentaho</groupId>
      <artifactId>*</artifactId>
    </exclusion>
  </exclusions>
</dependency>

<dependency>
  <groupId>${hive.groupid}</groupId>
  <artifactId>hive-service-rpc</artifactId>
  <version>${hive.version}</version>
  <scope>${spark.bundle.hive.scope}</scope>
</dependency>

<dependency>
  <groupId>${hive.groupid}</groupId>
  <artifactId>hive-jdbc</artifactId>
  <version>${hive.version}</version>
  <scope>${spark.bundle.hive.scope}</scope>
  <exclusions>
    <exclusion>
      <groupId>javax.servlet</groupId>
      <artifactId>*</artifactId>
    </exclusion>
    <exclusion>
      <groupId>javax.servlet.jsp</groupId>
```

```
        <artifactId>*</artifactId>
      </exclusion>
    </exclusion>
    <groupId>org.eclipse.jetty</groupId>
    <artifactId>*</artifactId>
  </exclusion>
</exclusions>
</dependency>

<dependency>
  <groupId>${hive.groupid}</groupId>
  <artifactId>hive-metastore</artifactId>
  <version>${hive.version}</version>
  <scope>${spark.bundle.hive.scope}</scope>
  <exclusions>
    <exclusion>
      <groupId>javax.servlet</groupId>
      <artifactId>*</artifactId>
    </exclusion>
    <exclusion>
      <groupId>org.datanucleus</groupId>
      <artifactId>datanucleus-core</artifactId>
    </exclusion>
    <exclusion>
      <groupId>javax.servlet.jsp</groupId>
      <artifactId>*</artifactId>
    </exclusion>
    <exclusion>
      <artifactId>guava</artifactId>
      <groupId>com.google.guava</groupId>
    </exclusion>
  </exclusions>
</dependency>

<dependency>
  <groupId>${hive.groupid}</groupId>
  <artifactId>hive-common</artifactId>
  <version>${hive.version}</version>
  <scope>${spark.bundle.hive.scope}</scope>
  <exclusions>
    <exclusion>
      <groupId>org.eclipse.jetty.orbit</groupId>
      <artifactId>javax.servlet</artifactId>
    </exclusion>
    <exclusion>
      <groupId>org.eclipse.jetty</groupId>
      <artifactId>*</artifactId>
    </exclusion>
  </exclusions>
</dependency>

<!-- 增加 hudi 配置版本的 jetty -->
<dependency>
  <groupId>org.eclipse.jetty</groupId>
  <artifactId>jetty-server</artifactId>
  <version>${jetty.version}</version>
</dependency>
```

```
<dependency>
  <groupId>org.eclipse.jetty</groupId>
  <artifactId>jetty-util</artifactId>
  <version>${jetty.version}</version>
</dependency>
<dependency>
  <groupId>org.eclipse.jetty</groupId>
  <artifactId>jetty-webapp</artifactId>
  <version>${jetty.version}</version>
</dependency>
<dependency>
  <groupId>org.eclipse.jetty</groupId>
  <artifactId>jetty-http</artifactId>
  <version>${jetty.version}</version>
</dependency>
```

否则在使用 spark 向 hudi 表插入数据时，会报错如下：

```
java.lang.NoSuchMethodError:
org.apache.hudi.org.apache.jetty.server.session.SessionHandler.setHttpOnly(Z)V
```

8.1.6 执行编译命令

```
cd /opt/software/hudi-0.12.0/
mvn clean package -DskipTests -Dspark3.2 -Dflink1.13 -Dscala-2.12
-Dhadoop.version=3.1.3 -Pflink-bundle-shade-hive3
```

我们最后实际用到的几个包为：

```
/opt/software/hudi-0.12.0/packaging/hudi-flink-bundle/target/hudi-
flink1.13-bundle-0.12.0.jar
/opt/software/hudi-0.12.0/packaging/hudi-hadoop-mr-bundle/target/
hudi-hadoop-mr-bundle-0.12.0.jar
/opt/software/hudi-0.12.0/packaging/hudi-hive-sync-bundle/target/
hudi-hive-sync-bundle-0.12.0.jar
```

这几个编译好的 jar 包放在本课程的资料包中。

8.2 Hudi 集成 Flink

8.2.1 解压 Flink 并配置环境变量

```
cd /opt/software
tar -zxvf flink-1.13.6-bin-scala_2.12.tgz -C /opt/module/
mv /opt/module/flink-1.13.6 /opt/module/flink
```

```
sudo vim /etc/profile.d/my_env.sh

export HADOOP_CLASSPATH=`hadoop classpath`
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop

sudo source /etc/profile.d/my_env.sh
```


8.2.2 修改 Flink 的配置文件

修改 flink-conf.yaml

```
vim /opt/module/flink/conf/flink-conf.yaml

classloader.check-leaked-classloader: false
#根据机器性能进行 tm 的 slot 设置
taskmanager.numberOfTaskSlots: 4

state.backend: rocksdb
# checkpoint 的时间根据集群性能和数据量确定
execution.checkpointing.interval: 30000
state.checkpoints.dir: hdfs://hadoop102:8020/ckps
state.backend.incremental: true
```

8.2.3 将 Hudi 集成至 Flink

我们将编译好的 hudi-flink1.13-bundle_2.12-0.11.0.jar 放到 Flink 的 lib 目录下

```
cp
/opt/software/hudi-0.11.0/packaging/hudi-flink-bundle/target/hudi-
flink1.13-bundle_2.12-0.12.0.jar /opt/module/flink/lib/
```

8.2.4 解决 guava 依赖冲突

```
cp /opt/module/hadoop/share/hadoop/common/lib/guava-27.0-jre.jar
/opt/module/flink/lib/
```

8.2.5 将项目用到的 connector 的 jar 包放入 flink 的 lib 中

```
#需要下载的 jar 放入 flink 的 lib
https://repo.maven.apache.org/maven2/org/apache/flink/flink-sql-c
onnectors-kafka_2.12/1.13.6/flink-sql-connectors-kafka_2.12-1.13.6.
jar
https://repo1.maven.org/maven2/com/ververica/flink-sql-connectors-
mysql-cdc/2.2.1/flink-sql-connectors-mysql-cdc-2.2.1.jar
https://repo.maven.apache.org/maven2/org/apache/flink/flink-sql-c
onnectors-hive-3.1.2_2.12/1.13.6/flink-sql-connectors-hive-3.1.2_2.
12-1.13.6.jar
```

需要注意：

(1) hive-connector 必须解决 guava 冲突。使用压缩软件打开 jar, 删除 com 目录下的 google 文件夹



(2) 解决找不到 `hadoop` 的依赖问题

```
cp
/opt/module/hadoop-3.1.3/share/hadoop/mapreduce/hadoop-mapreduce-
client-core-3.1.3.jar flink/lib
```

8.2.6 将元数据集成至 HiveCatalog

需要在 `flink/conf` 目录下提前准备配置元数据的配置文件

将 `sql-client-init.sql` 文件放在 `/opt/module/flink/conf` 目录下。内容如下：

```
#sql-client 配置 sql 文件
vim conf/sql-client-init.sql

CREATE CATALOG hive_catalog WITH (
    'type' = 'hive',
    'default-database' = 'default',
    'hive-conf-dir' = '/opt/module/hive/conf',
    'hadoop-conf-dir' = '/opt/module/hadoop/etc/hadoop'
);
-- set the HiveCatalog as the current catalog of the session
USE CATALOG hive_catalog;
load module hive with ('hive-version'='3.1.2');
```

在启动 `sql client` 时候显式指定

```
bin/sql-client.sh embedded -i conf/sql-client-init.sql -s
yarn-session
```

8.3 Hudi 集成 Hive

8.3.1 解压 Hive

把 apache-hive-3.1.2-bin.tar.gz 上传到 Linux 的/opt/software 目录下

解压 apache-hive-3.1.2-bin.tar.gz 到/opt/module/目录下面

```
tar -zxvf /opt/software/apache-hive-3.1.3-bin.tar.gz -C /opt/module/
```

修改 apache-hive-3.1.2-bin.tar.gz 的名称为 hive

```
mv /opt/module/apache-hive-3.1.2-bin/ /opt/module/hive
```

8.3.2 将 Hudi 集成至 Hive

将 hudi-hadoop-mr-bundle-0.12.0.jar 和 hudi-hive-sync-bundle-0.12.0.jar 放到 hive 节点的 lib 目录下;

```
cp
/opt/software/hudi-0.12.0/packaging/hudi-hadoop-mr-bundle/target/
hudi-hadoop-mr-bundle-0.12.0.jar /opt/module/hive/lib/
```

```
cp
/opt/software/hudi-0.12.0/packaging/hudi-hive-sync-bundle/target/
hudi-hive-sync-bundle-0.12.0.jar /opt/module/hive/lib/
```

8.3.3 配置 Hive 与环境变量

修改/etc/profile.d/my_env.sh, 添加环境变量

```
sudo vim /etc/profile.d/my_env.sh
```

添加内容

```
#HIVE_HOME
export HIVE_HOME=/opt/module/hive
export PATH=$PATH:$HIVE_HOME/bin
```

source 操作

```
source /etc/profile.d/my_env.sh
```

将 MySQL 的 JDBC 驱动拷贝到 Hive 的 lib 目录下

```
cp /opt/software/mysql-connector-java-5.1.37.jar $HIVE_HOME/lib
```

在\$HIVE_HOME/conf 目录下新建 hive-site.xml 文件

```
[atguigu@hadoop102 software]$ vim $HIVE_HOME/conf/hive-site.xml
```

添加如下内容:

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>

<configuration>
  <!-- jdbc 连接的 URL -->
  <property>
    <name>javax.jdo.option.ConnectionURL</name>
```

```
<value>jdbc:mysql://hadoop102:3306/metastore?useSSL=false&useUnicode=true&characterEncoding=UTF-8</value>
</property>
<!-- jdbc 连接的 Driver-->
<property>
  <name>javax.jdo.option.ConnectionDriverName</name>
  <value>com.mysql.jdbc.Driver</value>
</property>
<!-- jdbc 连接的 username-->
<property>
  <name>javax.jdo.option.ConnectionUserName</name>
  <value>root</value>
</property>
<!-- jdbc 连接的 password -->
<property>
  <name>javax.jdo.option.ConnectionPassword</name>
  <value>123456</value>
</property>
<!-- Hive 默认在 HDFS 的工作目录 -->
<property>
  <name>hive.metastore.warehouse.dir</name>
  <value>/user/hive/warehouse</value>
</property>
<!-- Hive 元数据存储的验证 -->
<property>
  <name>hive.metastore.schema.validation</name>
  <value>>false</value>
</property>
<!-- 元数据存储授权 -->
<property>
  <name>hive.metastore.event.db.notification.api.auth</name>
  <value>>false</value>
</property>
<!-- 指定 hiveserver2 连接的 host -->
<property>
  <name>hive.server2.thrift.bind.host</name>
  <value>hadoop102</value>
</property>
<!-- 指定 hiveserver2 连接的端口号 -->
<property>
  <name>hive.server2.thrift.port</name>
  <value>10000</value>
</property>
<!-- hiveserver2 高可用参数, 开启此参数可以提高 hiveserver2 启动速度 -->
<property>
  <name>hive.server2.active.passive.ha.enable</name>
  <value>true</value>
</property>
<!-- 指定 metastore 服务的地址 -->
<property>
  <name>hive.metastore.uris</name>
  <value>thrift://hadoop102:9083</value>
</property>
<!-- 打印表名 -->
<property>
```

```
<name>hive.cli.print.header</name>
<value>true</value>
</property>
<!-- 打印库名 -->
<property>
  <name>hive.cli.print.current.db</name>
  <value>true</value>
</property></configuration>
```

8.3.4 初始化 Hive 元数据库

登录 MySQL

```
mysql -uroot -p123456
```

新建 Hive 元数据库

```
mysql> create database metastore;
mysql> quit;
```

初始化 Hive 元数据库（修改为采用 MySQL 存储元数据）

```
bin/schematool -dbType mysql -initSchema -verbose
```

8.3.5 启动 Hive Metastore 和 Hiveserver2 服务（附脚本）

启动 hiveserver2 和 metastore 服务的命令如下：

```
bin/hive --service hiveserver2
bin/hive --service metastore
```

也可编写脚本进行一键启停

```
vim $HIVE_HOME/bin/hiveservices.sh

#!/bin/bash

HIVE_LOG_DIR=$HIVE_HOME/logs
if [ ! -d $HIVE_LOG_DIR ]
then
  mkdir -p $HIVE_LOG_DIR
fi

#检查进程是否运行正常，参数 1 为进程名，参数 2 为进程端口
function check_process()
{
  pid=$(ps -ef 2>/dev/null | grep -v grep | grep -i $1 | awk '{print $2}')
  ppid=$(netstat -nltp 2>/dev/null | grep $2 | awk '{print $7}' | cut -d '/' -f 1)
  echo $pid
  [[ "$pid" =~ "$ppid" ]] && [ "$ppid" ] && return 0 || return 1
}

function hive_start()
{
  metapid=$(check_process HiveMetastore 9083)
  cmd="nohup hive --service metastore >$HIVE_LOG_DIR/metastore.log 2>&1 &"
}
```

```
[ -z "$metapid" ] && eval $cmd || echo "Metastore 服务已启动"
server2pid=$(check_process HiveServer2 10000)
cmd="nohup                                hive                                --service
hiveserver2 >$HIVE_LOG_DIR/hiveServer2.log 2>&1 &"
[ -z "$server2pid" ] && eval $cmd || echo "HiveServer2 服务已启动"
"
}

function hive_stop()
{
metapid=$(check_process HiveMetastore 9083)
[ "$metapid" ] && kill $metapid || echo "Metastore 服务未启动"
server2pid=$(check_process HiveServer2 10000)
[ "$server2pid" ] && kill $server2pid || echo "HiveServer2 服务
未启动"
}

case $1 in
"start")
hive_start
;;
"stop")
hive_stop
;;
"restart")
hive_stop
sleep 2
hive_start
;;
"status")
check_process HiveMetastore 9083 >/dev/null && echo "Metastore
服务运行正常" || echo "Metastore 服务运行异常"
check_process HiveServer2 10000 >/dev/null && echo "HiveServer2
服务运行正常" || echo "HiveServer2 服务运行异常"
;;
*)
echo Invalid Args!
echo 'Usage: '$(basename $0)' start|stop|restart|status'
;;
esac
```

给脚本执行权限

```
chmod u+x $HIVE_HOME/bin/hiveservices.sh
```

启动 hiveserver2 和 metastore 服务

```
hiveservices.sh start
```

8.3.6 使用外部工具连接 Hive（可选）

可使用 Navicat/Datagrip 等外部数据库工具进行连接，端口为 10000。

8.4 模拟数据准备

通常企业在开始搭建数仓时，业务系统中会存在历史数据，一般是业务数据库存在历史数据，而用户行为日志无历史数据。假定数仓上线的日期为 2020-06-14，为模拟真实场景，需准备以下数据。

8.4.1 日志数据

用户行为日志，一般是没有历史数据的，故日志只需要准备 2020-06-14 一天的数据。具体操作如下：

（1）生成模拟数据

- （1）启动日志采集通道，包括 Flume、Kafka 等
- （2）修改两个日志服务器（hadoop102、hadoop103）中的 `/opt/module/applog/application.yml` 配置文件，将 `mock.date` 参数改为 2020-06-14。
- （3）执行日志生成脚本 `lg.sh`。
- （4）观察 Kafka 的 topic 中是否出现相应数据

（2）使用 Flink SQL 将 Kafka 中的数据映射为表

库为 `kafka_log`

```
CREATE TABLE `kafka_log`.`kafka_topic_log` (  
  `common` ROW<ar STRING,ba STRING,ch STRING,is_new STRING,md  
  STRING,mid STRING,os STRING,uid STRING,vc STRING>,  
  `page` ROW<during_time STRING,item STRING,item_type  
  STRING,last_page_id STRING,page_id STRING,source_type STRING> ,  
  `actions` ARRAY<ROW<action_id STRING,item STRING,item_type  
  STRING,ts BIGINT>>,  
  `displays` ARRAY<ROW<display_type STRING,item STRING,item_type  
  STRING,`order` STRING,pos_id STRING>>,  
  `start` ROW<entry STRING,loading_time BIGINT,open_ad_id  
  BIGINT,open_ad_ms BIGINT,open_ad_skip_ms BIGINT>,  
  `err` ROW<error_code BIGINT,msg STRING>,  
  `ts` BIGINT  
) WITH (  
  'connector' = 'kafka',  
  'topic' = 'topic_log',  
  'properties.bootstrap.servers' =  
'hadoop102:9092,hadoop103:9092,hadoop104:9092',  
  'properties.group.id' = 'hudi_source',  
  'scan.startup.mode' = 'latest-offset',  
  'format' = 'json',  
  'json.fail-on-missing-field'='false',  
  'json.ignore-parse-errors' = 'true'  
) ;
```


8.4.2 业务数据

业务数据一般存在历史数据，此处需准备 2020-06-10 至 2020-06-14 的数据。具体操作如下。

(1) 生成模拟数据

① 修改 hadoop102 节点上的/opt/module/db_log/application.properties 文件,将 mock.date、mock.clear, mock.clear.user 三个参数调整为如图所示的值。

```
#业务日期
mock.date=2020-06-10
#是否重置
mock.clear=1
#是否重置用户
mock.clear.user=1
```

② 执行模拟生成业务数据的命令，生成第一天 2020-06-10 的历史数据。

```
[atguigu@hadoop102 db_log]$ java -jar
gmall2020-mock-db-2021-01-22.jar
```

③ 修改 /opt/module/db_log/application.properties 文件，将 mock.date、mock.clear, mock.clear.user 三个参数调整为如图所示的值。

```
#业务日期
mock.date=2020-06-11
#是否重置
mock.clear=0
#是否重置用户
mock.clear.user=0
```

④ 执行模拟生成业务数据的命令，生成第二天 2020-06-11 的历史数据。

```
[atguigu@hadoop102 db_log]$ java -jar
gmall2020-mock-db-2021-10-10.jar
```

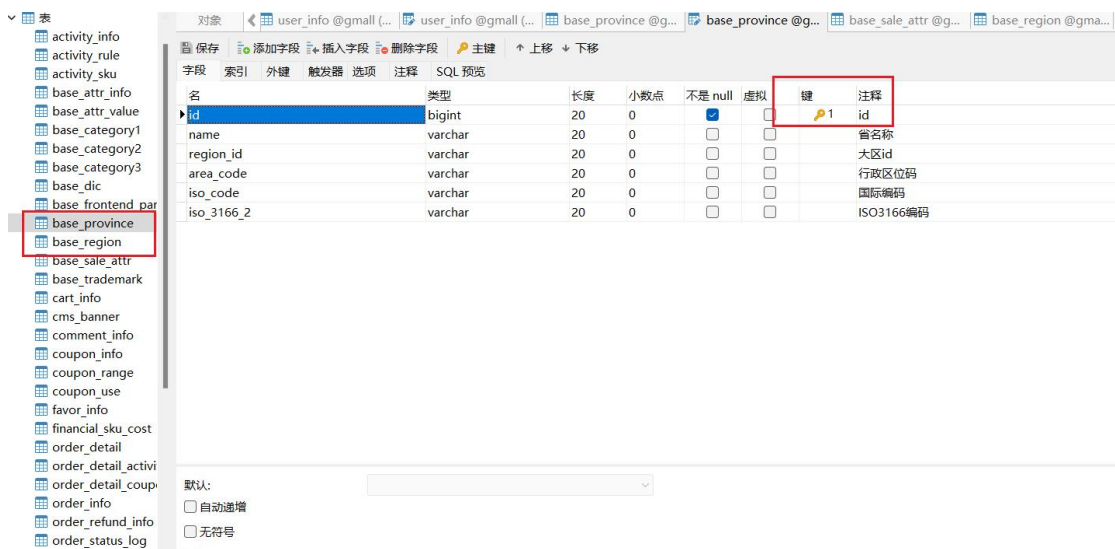
⑤ 之后只修改/opt/module/db_log/application.properties 文件中的 mock.date 参数，依次改为 2020-06-12, 2020-06-13, 2020-06-14，并分别生成对应日期的数据。

(2) 使用 Flink CDC 将 MySQL 的数据映射为 Flink 中的表

库为 flink_cdc

表中除了有业务数据还有维度数据

注意:CDC 在使用表格数据的时候要求数据必须要有主键，所有需要修改 mysql 中的表格信息：



The screenshot shows a database management interface with a table schema for 'base_province'. The table has the following fields:

名	类型	长度	小数点	不是 null	虚拟	键	注释
id	bigint	20	0	☒	☐	1	id
name	varchar	20	0	☐	☐		省名称
region_id	varchar	20	0	☐	☐		大区id
area_code	varchar	20	0	☐	☐		行政区代码
iso_code	varchar	20	0	☐	☐		国际编码
iso_3166_2	varchar	20	0	☐	☐		ISO3166编码

第 9 章 湖仓一体之 ODS 层

ODS 层的设计要点如下：

- 1) ODS 层的表结构设计依托于从业务系统同步过来的数据结构。
- 2) 先使用 Flink SQL 建表，然后将数据源（也就是 8.4 中的数据）进行 insert 操作，并补齐缺失的字段。

9.1 日志表建表及数据装载

1) 建表数据

```
CREATE TABLE `hudi_ods`.`ods_log` (
  `uuid` STRING,
  `common` ROW<ar STRING,ba STRING,ch STRING,is_new STRING,md STRING,mid STRING,os STRING,uid STRING,vc STRING>,
  `page` ROW<during_time STRING,item STRING,item_type STRING,last_page_id STRING,page_id STRING,source_type STRING>,
  `actions` ARRAY<ROW<action_id STRING,item STRING,item_type STRING,ts BIGINT>>,
  `displays` ARRAY<ROW<display_type STRING,item STRING,item_type STRING,`order` STRING,pos_id STRING>>,
  `start` ROW<entry STRING,loading_time BIGINT,open_ad_id BIGINT,open_ad_ms BIGINT,open_ad_skip_ms BIGINT>,
  `err` ROW<error_code BIGINT,msg STRING>,
  `ts` BIGINT,
  `dt` STRING,
  `t` as TO_TIMESTAMP(FROM_UNIXTIME(ts/1000,'yyyy-MM-dd HH:mm:ss')),
  WATERMARK FOR `t` AS `t` - INTERVAL '5' SECOND
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
```

```
'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_ods/ods_log',
'table.type'='MERGE_ON_READ',
'hoodie.datasource.write.recordkey.field' = 'uuid',
'hoodie.datasource.write.precombine.field' = 'ts',
'write.bucket_assign.tasks'='1',
'write.tasks' = '1',
'compaction.tasks' = '1',
'compaction.async.enabled' = 'true',
'compaction.schedule.enabled' = 'true',
'compaction.trigger.strategy' = 'num_commits',
'compaction.delta_commits' = '5',
'read.streaming.enabled' = 'true',
'read.streaming.skip_compaction' = 'true',
'hive_sync.enable'='true',
'hive_sync.mode' = 'hms',
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
'hive_sync.db'='hive_ods',
'hive_sync.table'='ods_log'
);
```

2) 数据装载

```
set table.dynamic-table-options.enabled=true;
set table.exec.sink.not-null-enforcer=drop;
INSERT INTO hudi_ods.ods_log
select uuid() as uuid
      ,*
      ,date_format(from_utc_timestamp(ts,'GMT+8'),'yyyy-MM-dd')
dt
from `kafka_log`.`kafka_topic_log`;
```

9.2 业务表建表及数据装载

9.2.1 活动信息表 ods_activity_info

```
CREATE TABLE `hudi_ods`.`ods_activity_info` (
  `id` bigint,
  `activity_name` varchar(200),
  `activity_type` varchar(10),
  `activity_desc` varchar(2000),
  `start_time` timestamp(0),
  `end_time` timestamp(0),
  `create_time` timestamp(0),
  `ts` timestamp(3),
  `dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_activity_i
nfo',
'table.type'='MERGE_ON_READ',
'hoodie.datasource.write.recordkey.field' = 'id',
'hoodie.datasource.write.precombine.field' = 'ts',
'write.bucket_assign.tasks'='1',
'write.tasks' = '4',
```

```
'compaction.tasks' = '1',
'compaction.async.enabled' = 'true',
'compaction.schedule.enabled' = 'true',
'compaction.trigger.strategy' = 'num_commits',
'compaction.delta_commits' = '5',
'read.streaming.enabled' = 'true',
'changelog.enabled' = 'true',
'read.streaming.skip_compaction' = 'true',
'hive_sync.enable'='true',
'hive_sync.mode' = 'hms',
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
'hive_sync.db'='hive_ods',
'hive_sync.table'='ods_activity_info');
```

9.2.2 活动规则表 ods_activity_rule

```
CREATE TABLE `hudi_ods`.`ods_activity_rule` (
  `id` int,
  `activity_id` int,
  `activity_type` varchar(20),
  `condition_amount` decimal(16, 2),
  `condition_num` bigint,
  `benefit_amount` decimal(16, 2),
  `benefit_discount` decimal(10, 2),
  `benefit_level` bigint,
  `ts` timestamp(3),
  `dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_activity_r
ule',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_ods',
  'hive_sync.table'='ods_activity_rule');
```

9.2.3 一级品类表 ods_base_category1

```
CREATE TABLE `hudi_ods`.`ods_base_category1` (
  `id` bigint,
```

```
`name` varchar(10),
`ts` timestamp(3),
`dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_base_category1',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_ods',
  'hive_sync.table'='ods_base_category1');
```

9.2.4 二级品类表 ods_base_category2

```
CREATE TABLE `hudi_ods`.`ods_base_category2` (
  `id` bigint,
  `name` varchar(200),
  `category1_id` bigint,
  `ts` timestamp(3),
  `dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_base_category2',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
```

```
'hive_sync.enable'='true',
'hive_sync.mode' = 'hms',
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
'hive_sync.db'='hive_ods',
'hive_sync.table'='ods_base_category2');
```

9.2.5 三级品类表 ods_base_category3

```
CREATE TABLE `hudi_ods`.`ods_base_category3` (
  `id` bigint,
  `name` varchar(200),
  `category2_id` bigint,
  `ts` timestamp(3),
  `dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_base_category3',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_ods',
  'hive_sync.table'='ods_base_category3');
```

9.2.6 编码字典表 ods_base_dic

```
CREATE TABLE `hudi_ods`.`ods_base_dic` (
  `dic_code` varchar(10),
  `dic_name` varchar(100),
  `parent_code` varchar(10),
  `create_time` timestamp(0),
  `operate_time` timestamp(0),
  `ts` timestamp(3),
  `dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_base_dic',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'dic_code',
```

```
'hoodie.datasource.write.precombine.field' = 'ts',
'write.bucket_assign.tasks'='1',
'write.tasks' = '4',
'compaction.tasks' = '1',
'compaction.async.enabled' = 'true',
'compaction.schedule.enabled' = 'true',
'compaction.trigger.strategy' = 'num_commits',
'compaction.delta_commits' = '5',
'read.streaming.enabled' = 'true',
'changelog.enabled' = 'true',
'read.streaming.skip_compaction' = 'true',
'hive_sync.enable'='true',
'hive_sync.mode' = 'hms',
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
'hive_sync.db'='hive_ods',
'hive_sync.table'='ods_base_dic');
```

9.2.7 省份表 ods_base_province

```
CREATE TABLE `hudi_ods`.`ods_base_province` (
  `id` bigint,
  `name` varchar(20),
  `region_id` varchar(20),
  `area_code` varchar(20),
  `iso_code` varchar(20),
  `iso_3166_2` varchar(20),
  `ts` timestamp(3),
  `dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_base_province',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_ods',
  'hive_sync.table'='ods_base_province');
```

9.2.8 地区表 ods_base_region

```
CREATE TABLE `hudi_ods`.`ods_base_region` (
  `id` varchar(20),
```



```
`region_name` varchar(20),
`ts` timestamp(3),
`dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_base_region',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_ods',
  'hive_sync.table'='ods_base_region');
```

9.2.9 品牌表 ods_base_trademark

```
CREATE TABLE `hudi_ods`.`ods_base_trademark` (
  `id` bigint,
  `tm_name` varchar(100),
  `logo_url` varchar(200),
  `ts` timestamp(3),
  `dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_base_trademark',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
```

```
'hive_sync.enable'='true',  
'hive_sync.mode' = 'hms',  
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',  
'hive_sync.db'='hive_ods',  
'hive_sync.table'='ods_base_trademark');
```

9.2.10 购物车表 ods_cart_info

```
CREATE TABLE `hudi_ods`.`ods_cart_info` (  
  `id` bigint,  
  `user_id` varchar(200),  
  `sku_id` bigint,  
  `cart_price` decimal(10, 2),  
  `sku_num` int,  
  `img_url` varchar(200),  
  `sku_name` varchar(200),  
  `is_checked` int,  
  `create_time` timestamp(0),  
  `operate_time` timestamp(0),  
  `is_ordered` bigint,  
  `order_time` timestamp(0),  
  `source_type` varchar(20),  
  `source_id` bigint,  
  `ts` timestamp(3),  
  `dt` string  
)  
PARTITIONED BY (`dt`)  
WITH (  
  'connector'='hudi',  
  'path'  
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_cart_info'  
,  
  'table.type'='MERGE_ON_READ',  
  'hoodie.datasource.write.recordkey.field' = 'id',  
  'hoodie.datasource.write.precombine.field' = 'ts',  
  'write.bucket_assign.tasks'='1',  
  'write.tasks' = '4',  
  'compaction.tasks' = '1',  
  'compaction.async.enabled' = 'true',  
  'compaction.schedule.enabled' = 'true',  
  'compaction.trigger.strategy' = 'num_commits',  
  'compaction.delta_commits' = '5',  
  'read.streaming.enabled' = 'true',  
  'changelog.enabled' = 'true',  
  'read.streaming.skip_compaction' = 'true',  
  'hive_sync.enable'='true',  
  'hive_sync.mode' = 'hms',  
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',  
  'hive_sync.db'='hive_ods',  
  'hive_sync.table'='ods_cart_info');
```

9.2.11 优惠券信息表 ods_coupon_info

```
CREATE TABLE `hudi_ods`.`ods_coupon_info` (  
  `id` bigint,  
  `coupon_name` varchar(100),  
  `coupon_type` varchar(10),  
  `condition_amount` decimal(10, 2),
```

```
`condition_num` bigint,
`activity_id` bigint,
`benefit_amount` decimal(16, 2),
`benefit_discount` decimal(10, 2),
`create_time` timestamp(0),
`range_type` varchar(10),
`limit_num` int,
`taken_count` int,
`start_time` timestamp(0),
`end_time` timestamp(0),
`operate_time` timestamp(0),
`expire_time` timestamp(0),
`range_desc` varchar(500),
`ts` timestamp(3),
`dt` string
)
PARTITIONED BY (`dt`)
WITH (
    'connector'='hudi',
    'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_coupon_info',
    'table.type'='MERGE_ON_READ',
    'hoodie.datasource.write.recordkey.field' = 'id',
    'hoodie.datasource.write.precombine.field' = 'ts',
    'write.bucket_assign.tasks'='1',
    'write.tasks' = '4',
    'compaction.tasks' = '1',
    'compaction.async.enabled' = 'true',
    'compaction.schedule.enabled' = 'true',
    'compaction.trigger.strategy' = 'num_commits',
    'compaction.delta_commits' = '5',
    'read.streaming.enabled' = 'true',
    'changelog.enabled' = 'true',
    'read.streaming.skip_compaction' = 'true',
    'hive_sync.enable'='true',
    'hive_sync.mode' = 'hms',
    'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
    'hive_sync.db'='hive_ods',
    'hive_sync.table'='ods_coupon_info');
```

9.2.12 商品平台属性表 ods_sku_attr_value

```
CREATE TABLE `hudi_ods`.`ods_sku_attr_value` (
    `id` bigint,
    `attr_id` bigint,
    `value_id` bigint,
    `sku_id` bigint,
    `attr_name` varchar(30),
    `value_name` varchar(30),
    `ts` timestamp(3),
    `dt` string
)
PARTITIONED BY (`dt`)
WITH (
    'connector'='hudi',
    'path'
```

```
= 'hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_sku_attr_value',
  'table.type' = 'MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks' = '1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable' = 'true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db' = 'hive_ods',
  'hive_sync.table' = 'ods_sku_attr_value');
```

9.2.13 商品表 ods_sku_info

```
CREATE TABLE `hudi_ods`.`ods_sku_info` (
  `id` bigint,
  `spu_id` bigint,
  `price` decimal(10, 0),
  `sku_name` varchar(200),
  `sku_desc` varchar(2000),
  `weight` decimal(10, 2),
  `tm_id` bigint,
  `category3_id` bigint,
  `sku_default_img` varchar(300),
  `is_sale` boolean,
  `create_time` timestamp(0),
  `ts` timestamp(3),
  `dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector' = 'hudi',
  'path'
= 'hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_sku_info',
  'table.type' = 'MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks' = '1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable' = 'true',
```

```
'hive_sync.mode' = 'hms',  
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',  
'hive_sync.db'='hive_ods',  
'hive_sync.table'='ods_sku_info');
```

9.2.14 商品销售属性值表 ods_sku_sale_attr_value

```
CREATE TABLE `hudi_ods`.`ods_sku_sale_attr_value` (  
  `id` bigint,  
  `sku_id` bigint,  
  `spu_id` int,  
  `sale_attr_value_id` bigint,  
  `sale_attr_id` bigint,  
  `sale_attr_name` varchar(30),  
  `sale_attr_value_name` varchar(30),  
  `ts` timestamp(3),  
  `dt` string  
)  
PARTITIONED BY (`dt`)  
WITH (  
  'connector'='hudi',  
  'path'  
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_sku_sale_attr_value',  
  'table.type'='MERGE_ON_READ',  
  'hoodie.datasource.write.recordkey.field' = 'id',  
  'hoodie.datasource.write.precombine.field' = 'ts',  
  'write.bucket_assign.tasks'='1',  
  'write.tasks' = '4',  
  'compaction.tasks' = '1',  
  'compaction.async.enabled' = 'true',  
  'compaction.schedule.enabled' = 'true',  
  'compaction.trigger.strategy' = 'num_commits',  
  'compaction.delta_commits' = '5',  
  'read.streaming.enabled' = 'true',  
  'changelog.enabled' = 'true',  
  'read.streaming.skip_compaction' = 'true',  
  'hive_sync.enable'='true',  
  'hive_sync.mode' = 'hms',  
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',  
  'hive_sync.db'='hive_ods',  
  'hive_sync.table'='ods_sku_sale_attr_value');
```

9.2.15 SPU 表 ods_spu_info

```
CREATE TABLE `hudi_ods`.`ods_spu_info` (  
  `id` bigint,  
  `spu_name` varchar(200),  
  `description` varchar(1000),  
  `category3_id` bigint,  
  `tm_id` bigint,  
  `ts` timestamp(3),  
  `dt` string  
)  
PARTITIONED BY (`dt`)  
WITH (  
  'connector'='hudi',  
  'path'
```

```
= 'hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_spu_info',  
  'table.type' = 'MERGE_ON_READ',  
  'hoodie.datasource.write.recordkey.field' = 'id',  
  'hoodie.datasource.write.precombine.field' = 'ts',  
  'write.bucket_assign.tasks' = '1',  
  'write.tasks' = '4',  
  'compaction.tasks' = '1',  
  'compaction.async.enabled' = 'true',  
  'compaction.schedule.enabled' = 'true',  
  'compaction.trigger.strategy' = 'num_commits',  
  'compaction.delta_commits' = '5',  
  'read.streaming.enabled' = 'true',  
  'changelog.enabled' = 'true',  
  'read.streaming.skip_compaction' = 'true',  
  'hive_sync.enable' = 'true',  
  'hive_sync.mode' = 'hms',  
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',  
  'hive_sync.db' = 'hive_ods',  
  'hive_sync.table' = 'ods_spu_info');
```

9.2.16 购物车表 ods_cart_info

```
CREATE TABLE `hudi_ods`.`ods_cart_info` (  
  `id` bigint,  
  `user_id` varchar(200),  
  `sku_id` bigint,  
  `cart_price` decimal(10, 2),  
  `sku_num` int,  
  `img_url` varchar(200),  
  `sku_name` varchar(200),  
  `is_checked` int,  
  `create_time` timestamp(0),  
  `operate_time` timestamp(0),  
  `is_ordered` bigint,  
  `order_time` timestamp(0),  
  `source_type` varchar(20),  
  `source_id` bigint,  
  `ts` timestamp(3),  
  `dt` string  
)  
PARTITIONED BY (`dt`)  
WITH (  
  'connector' = 'hudi',  
  'path' =  
= 'hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_cart_info'  
,  
  'table.type' = 'MERGE_ON_READ',  
  'hoodie.datasource.write.recordkey.field' = 'id',  
  'hoodie.datasource.write.precombine.field' = 'ts',  
  'write.bucket_assign.tasks' = '1',  
  'write.tasks' = '4',  
  'compaction.tasks' = '1',  
  'compaction.async.enabled' = 'true',  
  'compaction.schedule.enabled' = 'true',  
  'compaction.trigger.strategy' = 'num_commits',  
  'compaction.delta_commits' = '5',  
  'read.streaming.enabled' = 'true',
```

```
'changelog.enabled' = 'true',  
'read.streaming.skip_compaction' = 'true',  
'hive_sync.enable'='true',  
'hive_sync.mode' = 'hms',  
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',  
'hive_sync.db'='hive_ods',  
'hive_sync.table'='ods_cart_info');
```

9.2.17 评论表 ods_comment_info

```
CREATE TABLE `hudi_ods`.`ods_comment_info` (  
  `id` bigint,  
  `user_id` bigint,  
  `nick_name` varchar(20),  
  `head_img` varchar(200),  
  `sku_id` bigint,  
  `spu_id` bigint,  
  `order_id` bigint,  
  `appraise` varchar(10),  
  `comment_txt` varchar(2000),  
  `create_time` timestamp(0),  
  `operate_time` timestamp(0),  
  `ts` timestamp(3),  
  `dt` string  
)  
PARTITIONED BY (`dt`)  
WITH (  
  'connector'='hudi',  
  'path'  
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_comment_in  
fo',  
  'table.type'='MERGE_ON_READ',  
  'hoodie.datasource.write.recordkey.field' = 'id',  
  'hoodie.datasource.write.precombine.field' = 'ts',  
  'write.bucket_assign.tasks'='1',  
  'write.tasks' = '4',  
  'compaction.tasks' = '1',  
  'compaction.async.enabled' = 'true',  
  'compaction.schedule.enabled' = 'true',  
  'compaction.trigger.strategy' = 'num_commits',  
  'compaction.delta_commits' = '5',  
  'read.streaming.enabled' = 'true',  
  'changelog.enabled' = 'true',  
  'read.streaming.skip_compaction' = 'true',  
  'hive_sync.enable'='true',  
  'hive_sync.mode' = 'hms',  
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',  
  'hive_sync.db'='hive_ods',  
  'hive_sync.table'='ods_comment_info');
```

9.2.18 优惠券领用表 ods_coupon_use

```
CREATE TABLE `hudi_ods`.`ods_coupon_use` (  
  `id` bigint,  
  `coupon_id` bigint,  
  `user_id` bigint,  
  `order_id` bigint,  
  `coupon_status` varchar(10),
```



```
`create_time` timestamp(0),
`get_time` timestamp(0),
`using_time` timestamp(0),
`used_time` timestamp(0),
`expire_time` timestamp(0),
`ts` timestamp(3),
`dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_coupon_use',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_ods',
  'hive_sync.table'='ods_coupon_use');
```

9.2.19 收藏表 ods_favor_info

```
CREATE TABLE `hudi_ods`.`ods_order_detail` (
  `id` bigint,
  `order_id` bigint,
  `sku_id` bigint,
  `sku_name` varchar(200),
  `img_url` varchar(200),
  `order_price` decimal(10, 2),
  `sku_num` bigint,
  `create_time` timestamp(0),
  `source_type` varchar(20),
  `source_id` bigint,
  `split_total_amount` decimal(16, 2),
  `split_activity_amount` decimal(16, 2),
  `split_coupon_amount` decimal(16, 2),
  `ts` timestamp(3),
  `dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_order_deta
```

```
il',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_ods',
  'hive_sync.table'='ods_order_detail');
```

9.2.20 订单明细表 ods_order_detail

```
CREATE TABLE `hudi_ods`.`ods_order_detail` (
  `id` bigint,
  `order_id` bigint,
  `sku_id` bigint,
  `sku_name` varchar(200),
  `img_url` varchar(200),
  `order_price` decimal(10, 2),
  `sku_num` bigint,
  `create_time` timestamp(0),
  `source_type` varchar(20),
  `source_id` bigint,
  `split_total_amount` decimal(16, 2),
  `split_activity_amount` decimal(16, 2),
  `split_coupon_amount` decimal(16, 2),
  `ts` timestamp(3),
  `dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_order_detail',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
```

```
'read.streaming.skip_compaction' = 'true',  
'hive_sync.enable'='true',  
'hive_sync.mode' = 'hms',  
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',  
'hive_sync.db'='hive_ods',  
'hive_sync.table'='ods_order_detail');
```

9.2.21 订单明细活动关联表 ods_order_detail_activity

```
CREATE TABLE `hudi_ods`.`ods_order_detail_activity` (  
  `id` bigint,  
  `order_id` bigint,  
  `order_detail_id` bigint,  
  `activity_id` bigint,  
  `activity_rule_id` bigint,  
  `sku_id` bigint,  
  `create_time` timestamp(0),  
  `ts` timestamp(3),  
  `dt` string  
)  
PARTITIONED BY (`dt`)  
WITH (  
  'connector'='hudi',  
  'path'  
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_order_detail_activity',  
  'table.type'='MERGE_ON_READ',  
  'hoodie.datasource.write.recordkey.field' = 'id',  
  'hoodie.datasource.write.precombine.field' = 'ts',  
  'write.bucket_assign.tasks'='1',  
  'write.tasks' = '4',  
  'compaction.tasks' = '1',  
  'compaction.async.enabled' = 'true',  
  'compaction.schedule.enabled' = 'true',  
  'compaction.trigger.strategy' = 'num_commits',  
  'compaction.delta_commits' = '5',  
  'read.streaming.enabled' = 'true',  
  'changelog.enabled' = 'true',  
  'read.streaming.skip_compaction' = 'true',  
  'hive_sync.enable'='true',  
  'hive_sync.mode' = 'hms',  
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',  
  'hive_sync.db'='hive_ods',  
  'hive_sync.table'='ods_order_detail_activity');
```

9.2.22 订单明细优惠券关联表 ods_order_detail_coupon

```
CREATE TABLE `hudi_ods`.`ods_order_detail_coupon` (  
  `id` bigint,  
  `order_id` bigint,  
  `order_detail_id` bigint,  
  `coupon_id` bigint,  
  `coupon_use_id` bigint,  
  `sku_id` bigint,  
  `create_time` timestamp(0),  
  `ts` timestamp(3),  
  `dt` string  
)
```

```
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_order_detail_coupon',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_ods',
  'hive_sync.table'='ods_order_detail_coupon');
```

9.2.23 订单表 ods_order_info

```
CREATE TABLE `hudi_ods`.`ods_order_info` (
  `id` bigint,
  `consignee` varchar(100),
  `consignee_tel` varchar(20),
  `total_amount` decimal(10, 2),
  `order_status` varchar(20),
  `user_id` bigint,
  `payment_way` varchar(20),
  `delivery_address` varchar(1000),
  `order_comment` varchar(200),
  `out_trade_no` varchar(50),
  `trade_body` varchar(200),
  `create_time` timestamp(0),
  `operate_time` timestamp(0),
  `expire_time` timestamp(0),
  `process_status` varchar(20),
  `tracking_no` varchar(100),
  `parent_order_id` bigint,
  `img_url` varchar(200),
  `province_id` int,
  `activity_reduce_amount` decimal(16, 2),
  `coupon_reduce_amount` decimal(16, 2),
  `original_total_amount` decimal(16, 2),
  `feight_fee` decimal(16, 2),
  `feight_fee_reduce` decimal(16, 2),
  `refundable_time` timestamp(0),
  `ts` timestamp(3),
  `dt` string
)
PARTITIONED BY (`dt`)
```

```
WITH (  
    'connector'='hudi',  
    'path'  
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_order_info'  
,  
    'table.type'='MERGE_ON_READ',  
    'hoodie.datasource.write.recordkey.field' = 'id',  
    'hoodie.datasource.write.precombine.field' = 'ts',  
    'write.bucket_assign.tasks'='1',  
    'write.tasks' = '4',  
    'compaction.tasks' = '1',  
    'compaction.async.enabled' = 'true',  
    'compaction.schedule.enabled' = 'true',  
    'compaction.trigger.strategy' = 'num_commits',  
    'compaction.delta_commits' = '5',  
    'read.streaming.enabled' = 'true',  
    'changelog.enabled' = 'true',  
    'read.streaming.skip_compaction' = 'true',  
    'hive_sync.enable'='true',  
    'hive_sync.mode' = 'hms',  
    'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',  
    'hive_sync.db'='hive_ods',  
    'hive_sync.table'='ods_order_info');
```

9.2.24 退单表 ods_order_refund_info

```
CREATE TABLE `hudi_ods`.`ods_order_refund_info` (  
    `id` bigint,  
    `user_id` bigint,  
    `order_id` bigint,  
    `sku_id` bigint,  
    `refund_type` varchar(20),  
    `refund_num` bigint,  
    `refund_amount` decimal(16, 2),  
    `refund_reason_type` varchar(200),  
    `refund_reason_txt` varchar(20),  
    `refund_status` varchar(10),  
    `create_time` timestamp(0),  
    `ts` timestamp(3),  
    `dt` string  
)  
PARTITIONED BY (`dt`)  
WITH (  
    'connector'='hudi',  
    'path'  
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_order_refund_info',  
    'table.type'='MERGE_ON_READ',  
    'hoodie.datasource.write.recordkey.field' = 'id',  
    'hoodie.datasource.write.precombine.field' = 'ts',  
    'write.bucket_assign.tasks'='1',  
    'write.tasks' = '4',  
    'compaction.tasks' = '1',  
    'compaction.async.enabled' = 'true',  
    'compaction.schedule.enabled' = 'true',  
    'compaction.trigger.strategy' = 'num_commits',  
    'compaction.delta_commits' = '5',
```

```
'read.streaming.enabled' = 'true',
'changelog.enabled' = 'true',
'read.streaming.skip_compaction' = 'true',
'hive_sync.enable'='true',
'hive_sync.mode' = 'hms',
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
'hive_sync.db'='hive_ods',
'hive_sync.table'='ods_order_refund_info');
```

9.2.25 订单状态流水表 ods_order_status_log

```
CREATE TABLE `hudi_ods`.`ods_order_status_log` (
  `id` bigint,
  `order_id` bigint,
  `order_status` varchar(11),
  `operate_time` timestamp(0),
  `ts` timestamp(3),
  `dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_order_status_log',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_ods',
  'hive_sync.table'='ods_order_status_log');
```

9.2.26 支付表 ods_payment_info

```
CREATE TABLE `hudi_ods`.`ods_payment_info` (
  `id` int,
  `out_trade_no` varchar(50),
  `order_id` bigint,
  `user_id` bigint,
  `payment_type` varchar(20),
  `trade_no` varchar(50),
  `total_amount` decimal(10, 2),
  `subject` varchar(200),
  `payment_status` varchar(20),
  `create_time` timestamp(0),
  `callback_time` timestamp(0),
```

```
`callback_content` string,
`ts` timestamp(3),
`dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_payment_in
fo',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_ods',
  'hive_sync.table'='ods_payment_info');
```

9.2.27 退款表 ods_refund_payment

```
CREATE TABLE `hudi_ods`.`ods_refund_payment` (
  `id` int,
  `out_trade_no` varchar(50),
  `order_id` bigint,
  `sku_id` bigint,
  `payment_type` varchar(20),
  `trade_no` varchar(50),
  `total_amount` decimal(10, 2),
  `subject` varchar(200),
  `refund_status` varchar(30),
  `create_time` timestamp(0),
  `callback_time` timestamp(0),
  `callback_content` string,
  `ts` timestamp(3),
  `dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_refund_pay
ment',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
```



```
'write.tasks' = '4',
'compaction.tasks' = '1',
'compaction.async.enabled' = 'true',
'compaction.schedule.enabled' = 'true',
'compaction.trigger.strategy' = 'num_commits',
'compaction.delta_commits' = '5',
'read.streaming.enabled' = 'true',
'changelog.enabled' = 'true',
'read.streaming.skip_compaction' = 'true',
'hive_sync.enable'='true',
'hive_sync.mode' = 'hms',
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
'hive_sync.db'='hive_ods',
'hive_sync.table'='ods_refund_payment');
```

9.2.28 用户表 ods_user_info

```
CREATE TABLE `hudi_ods`.`ods_user_info` (
  `id` bigint,
  `login_name` varchar(200),
  `nick_name` varchar(200),
  `passwd` varchar(200),
  `name` varchar(200),
  `phone_num` varchar(200),
  `email` varchar(200),
  `head_img` varchar(200),
  `user_level` varchar(200),
  `birthday` date,
  `gender` varchar(1),
  `create_time` timestamp(0),
  `operate_time` timestamp(0),
  `status` varchar(200),
  `ts` timestamp(3),
  `dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/ods/ods_user_info'
,
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
```

```
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',  
'hive_sync.db'='hive_ods',  
'hive_sync.table'='ods_user_info');
```

9.2.29 数据装载

(1) 我们将 Flink CDC 拿到的数据，分别对应 hudi_ods 库中的每张表进行插入操作。因为对每张表设置了 hive_sync，所以每张表在 hive 的 hive_ods 数据库中同样有一份数据映射。

(2) 在 Flink SQL 中执行插入语句，每个语句都会启动一个 Flink SQL 程序，数据将会实时地进入 ODS 层中。

```
-- ods_activity_info 插入数据  
INSERT INTO hudi_ods.ods_activity_info  
select *  
    ,LOCALTIMESTAMP as ts  
    ,cast(CURRENT_DATE as string) as dt  
from flink_cdc.activity_info_cdc;  
-- select * from hudi_ods.ods_activity_info;  
  
-- ods_activity_rule 插入数据  
INSERT INTO hudi_ods.ods_activity_rule  
select *  
    ,LOCALTIMESTAMP as ts  
    ,cast(CURRENT_DATE as string) as dt  
from flink_cdc.activity_rule_cdc;  
-- select * from hudi_ods.ods_activity_rule;  
  
-- ods_base_category1 插入数据  
INSERT INTO hudi_ods.ods_base_category1  
select *  
    ,LOCALTIMESTAMP as ts  
    ,cast(CURRENT_DATE as string) as dt  
from flink_cdc.base_category1_cdc;  
-- select * from hudi_ods.ods_base_category1;  
  
-- ods_base_category2 插入数据  
INSERT INTO hudi_ods.ods_base_category2  
select *  
    ,LOCALTIMESTAMP as ts  
    ,cast(CURRENT_DATE as string) as dt  
from flink_cdc.base_category2_cdc;  
-- select * from hudi_ods.ods_base_category2;  
  
-- ods_base_category3 插入数据  
INSERT INTO hudi_ods.ods_base_category3  
select *  
    ,LOCALTIMESTAMP as ts  
    ,cast(CURRENT_DATE as string) as dt  
from flink_cdc.base_category3_cdc;  
-- select * from hudi_ods.ods_base_category3;
```

```
-- ods_base_dic 插入数据
INSERT INTO hudi_ods.ods_base_dic
select *
      ,LOCALTIMESTAMP as ts
      ,cast(CURRENT_DATE as string) as dt
from flink_cdc.base_dic_cdc;
-- select * from hudi_ods.ods_base_dic;

-- ods_base_province 插入数据
INSERT INTO hudi_ods.ods_base_province
select *
      ,LOCALTIMESTAMP as ts
      ,cast(CURRENT_DATE as string) as dt
from flink_cdc.base_province_cdc;
-- select * from hudi_ods.ods_base_province;

-- ods_base_region 插入数据
INSERT INTO hudi_ods.ods_base_region
select *
      ,LOCALTIMESTAMP as ts
      ,cast(CURRENT_DATE as string) as dt
from flink_cdc.base_region_cdc;
-- select * from hudi_ods.ods_base_region;

-- ods_base_trademark 插入数据
INSERT INTO hudi_ods.ods_base_trademark
select *
      ,LOCALTIMESTAMP as ts
      ,cast(CURRENT_DATE as string) as dt
from flink_cdc.base_trademark_cdc;
-- select * from hudi_ods.ods_base_trademark;

-- ods_cart_info 插入数据
INSERT INTO hudi_ods.ods_cart_info
select *
      ,LOCALTIMESTAMP as ts
      ,cast(CURRENT_DATE as string) as dt
from flink_cdc.cart_info_cdc;
-- select * from hudi_ods.ods_cart_info;

-- ods_comment_info 插入数据
INSERT INTO hudi_ods.ods_comment_info
select *
      ,LOCALTIMESTAMP as ts
      ,cast(CURRENT_DATE as string) as dt
from flink_cdc.comment_info_cdc;
-- select * from hudi_ods.ods_comment_info;
```

```
-- ods_coupon_info 插入数据
INSERT INTO hudi_ods.ods_coupon_info
select *
    ,LOCALTIMESTAMP as ts
    ,cast(CURRENT_DATE as string) as dt
from flink_cdc.coupon_info_cdc;
-- select * from hudi_ods.ods_coupon_info;

-- ods_coupon_use 插入数据
INSERT INTO hudi_ods.ods_coupon_use
select *
    ,LOCALTIMESTAMP as ts
    ,cast(CURRENT_DATE as string) as dt
from flink_cdc.coupon_use_cdc;
-- select * from hudi_ods.ods_coupon_use;

-- ods_favor_info 插入数据
INSERT INTO hudi_ods.ods_favor_info
select *
    ,LOCALTIMESTAMP as ts
    ,cast(CURRENT_DATE as string) as dt
from flink_cdc.favor_info_cdc;
-- select * from hudi_ods.ods_favor_info;

-- ods_order_detail 插入数据
INSERT INTO hudi_ods.ods_order_detail
select *
    ,LOCALTIMESTAMP as ts
    ,cast(CURRENT_DATE as string) as dt
from flink_cdc.order_detail_cdc;
-- select * from hudi_ods.ods_order_detail;

-- ods_order_detail_activity 插入数据
INSERT INTO hudi_ods.ods_order_detail_activity
select *
    ,LOCALTIMESTAMP as ts
    ,cast(CURRENT_DATE as string) as dt
from flink_cdc.order_detail_activity_cdc;
-- select * from hudi_ods.ods_order_detail_activity;

-- ods_order_detail_coupon 插入数据
INSERT INTO hudi_ods.ods_order_detail_coupon
select *
    ,LOCALTIMESTAMP as ts
    ,cast(CURRENT_DATE as string) as dt
from flink_cdc.order_detail_coupon_cdc;
-- select * from hudi_ods.ods_order_detail_coupon;

-- ods_order_info 插入数据
INSERT INTO hudi_ods.ods_order_info
```

```
select *
    ,LOCALTIMESTAMP as ts
    ,cast(CURRENT_DATE as string) as dt
from flink_cdc.order_info_cdc;
-- select * from hudi_ods.ods_order_info;

-- ods_order_refund_info 插入数据
INSERT INTO hudi_ods.ods_order_refund_info
select *
    ,LOCALTIMESTAMP as ts
    ,cast(CURRENT_DATE as string) as dt
from flink_cdc.order_refund_info_cdc;
-- select * from hudi_ods.ods_order_refund_info;

-- ods_order_status_log 插入数据
INSERT INTO hudi_ods.ods_order_status_log
select *
    ,LOCALTIMESTAMP as ts
    ,cast(CURRENT_DATE as string) as dt
from flink_cdc.order_status_log_cdc;
-- select * from hudi_ods.ods_order_status_log;

-- ods_payment_info 插入数据
INSERT INTO hudi_ods.ods_payment_info
select *
    ,LOCALTIMESTAMP as ts
    ,cast(CURRENT_DATE as string) as dt
from flink_cdc.payment_info_cdc;
-- select * from hudi_ods.ods_payment_info;

-- ods_refund_payment 插入数据
INSERT INTO hudi_ods.ods_refund_payment
select *
    ,LOCALTIMESTAMP as ts
    ,cast(CURRENT_DATE as string) as dt
from flink_cdc.refund_payment_cdc;
-- select * from hudi_ods.ods_refund_payment;

-- ods_sku_attr_value 插入数据
INSERT INTO hudi_ods.ods_sku_attr_value
select *
    ,LOCALTIMESTAMP as ts
    ,cast(CURRENT_DATE as string) as dt
from flink_cdc.sku_attr_value_cdc;
-- select * from hudi_ods.ods_sku_attr_value;

-- ods_sku_info 插入数据
INSERT INTO hudi_ods.ods_sku_info
select *
    ,LOCALTIMESTAMP as ts
```

```
        ,cast(CURRENT_DATE as string) as dt
from flink_cdc.skus_attr_value_cdc;
-- select * from hudi_ods.ods_sku_info;

-- ods_sku_sale_attr_value 插入数据
INSERT INTO hudi_ods.ods_sku_sale_attr_value
select *
        ,LOCALTIMESTAMP as ts
        ,cast(CURRENT_DATE as string) as dt
from flink_cdc.skus_sale_attr_value_cdc;
-- select * from hudi_ods.ods_sku_sale_attr_value;

-- ods_spu_info 插入数据
INSERT INTO hudi_ods.ods_spu_info
select *
        ,LOCALTIMESTAMP as ts
        ,cast(CURRENT_DATE as string) as dt
from flink_cdc.spu_info_cdc;
-- select * from hudi_ods.ods_spu_info;

-- ods_user_info 插入数据
INSERT INTO hudi_ods.ods_user_info
select *
        ,LOCALTIMESTAMP as ts
        ,cast(CURRENT_DATE as string) as dt
from flink_cdc.user_info_cdc;
-- select * from hudi_ods.ods_user_info;
c
```

第 10 章 湖仓一体之 DIM 层

DIM 层设计要点：

- 1) DIM 层的设计依据是维度建模理论，该层存储维度模型的维度表。
- 2) 日期维度采用文件导入

注意：读取 hudi 表格装载数据 如果不添加参数`read.start-commit='earliest'` 只会读取当前没有合并的数据，如果是当时生成的数据 没有触发合并机制仍然能读取到数据。等待一段时间之后需要添加参数才能读取到完整的数据。

10.1 商品维度表 dim_sku

10.1.1 建表语句

```
CREATE TABLE hudi_dim.dim_sku
(
  `id` BIGINT,
  `price` DECIMAL(10, 0),
  `sku_name` VARCHAR(200),
  `sku_desc` VARCHAR(2000),
  `weight` DECIMAL(10,2),
  `is_sale` BOOLEAN,
  `spu_id` BIGINT,
  `spu_name` VARCHAR(200),
  `category3_id` BIGINT,
  `category3_name` VARCHAR(200),
  `category2_id` BIGINT,
  `category2_name` VARCHAR(200),
  `category1_id` BIGINT,
  `category1_name` VARCHAR(10),
  `tm_id` BIGINT,
  `tm_name` VARCHAR(100),
  `sku_attr_values` MULTISET<string>,
  `sku_sale_attr_values` MULTISET<string>,
  `create_time` TIMESTAMP(0),
  `ts` timestamp(3),
  `dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dim/dim_sku',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '1',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
```

```
'compaction.trigger.strategy' = 'num_commits',
'compaction.delta_commits' = '5',
'read.streaming.enabled' = 'true',
'changelog.enabled' = 'true',
'read.streaming.skip_compaction' = 'true',
'hive_sync.enable'='true',
'hive_sync.mode' = 'hms',
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
'hive_sync.db'='hive_dim',
'hive_sync.table'='dim_sku');
```

10.1.2 数据装载

```
insert into hudi_dim.dim_sku
select
    sku.id,
    sku.price,
    sku.sku_name,
    sku.sku_desc,
    sku.weight,
    sku.is_sale,
    sku.spu_id,
    spu.spu_name,
    sku.category3_id,
    c3.name,
    c3.category2_id,
    c2.name,
    c2.category1_id,
    c1.name,
    sku.tm_id,
    tm.tm_name,
    attr.attrs,
    sale_attr.sale_attrs,
    sku.create_time,
    LOCALTIMESTAMP as ts,
    sku.dt dt
from hudi_ods.ods_sku_info sku
left join hudi_ods.ods_spu_info spu on sku.spu_id=spu.id
left      join      hudi_ods.ods_base_category3      c3      on
sku.category3_id=c3.id
left join hudi_ods.ods_base_category2 c2 on c3.category2_id=c2.id
left join hudi_ods.ods_base_category1 c1 on c2.category1_id=c1.id
left join hudi_ods.ods_base_trademark tm on sku.tm_id=tm.id
left join
(
    select
        sku_id,
        collect(concat_ws('||',cast(attr_id as
string),cast(value_id as string),attr_name,value_name)) attrs
        from hudi_ods.ods_sku_attr_value
        group by sku_id
) attr on sku.id=attr.sku_id
left join
(
    select
        sku_id,
        collect(concat_ws('||',cast(sale_attr_id as
```



```
string), cast(sale_attr_value_id as
string), sale_attr_name, sale_attr_value_name)) sale_attrs
  from hudi_ods.ods_sku_sale_attr_value
  group by sku_id
) sale_attr on sku.id=sale_attr.sku_id;
```

10.2 优惠券维度表 dim_coupon

10.2.1 建表语句

```
CREATE TABLE hudi_dim.dim_coupon
(
  `id`          BIGINT,
  `coupon_name` VARCHAR(100),
  `coupon_type_code` VARCHAR(10),
  `coupon_type_name` VARCHAR(100),
  `condition_amount` DECIMAL(10, 2),
  `condition_num`   BIGINT,
  `activity_id`     BIGINT,
  `benefit_amount`  DECIMAL(16, 2),
  `benefit_discount` DECIMAL(10, 2),
  `benefit_rule`    STRING,
  `create_time`     TIMESTAMP(0),
  `range_type_code` VARCHAR(10),
  `range_type_name` VARCHAR(100),
  `limit_num`       INT,
  `taken_count`     INT,
  `start_time`      TIMESTAMP(0),
  `end_time`        TIMESTAMP(0),
  `operate_time`    TIMESTAMP(0),
  `expire_time`     TIMESTAMP(0),
  `ts` timestamp(3),
  `dt` string
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dim/dim_coupo
n',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '1',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_dim',
```

```
'hive_sync.table'='dim_coupon');
```

10.2.2 数据装载

```
insert into hudi_dim.dim_coupon
select
    id,
    coupon_name,
    coupon_type,
    coupon_dic.dic_name,
    condition_amount,
    condition_num,
    activity_id,
    benefit_amount,
    benefit_discount,
    case coupon_type
        when '3201' then concat('满 ',cast(condition_amount as string),'元减',cast(benefit_amount as string),'元')
        when '3202' then concat('满 ',cast(condition_num as string),'件打',cast(10*(1-benefit_discount) as string),'折')
        when '3203' then concat('减 ',cast(benefit_amount as string),'元')
    end benefit_rule,
    create_time,
    range_type,
    range_dic.dic_name,
    limit_num,
    taken_count,
    start_time,
    end_time,
    operate_time,
    expire_time,
    LOCALTIMESTAMP as ts,
    ci.dt as dt
from
(
    select
        id,
        coupon_name,
        coupon_type,
        condition_amount,
        condition_num,
        activity_id,
        benefit_amount,
        benefit_discount,
        create_time,
        range_type,
        limit_num,
        taken_count,
        start_time,
        end_time,
        operate_time,
        expire_time,
        dt
    from hudi_ods.ods_coupon_info
)ci
left join
```

```
(
    select
        dic_code,
        dic_name
    from hudi_ods.ods_base_dic
    where parent_code='32'
) coupon_dic
on ci.coupon_type=coupon_dic.dic_code
left join
(
    select
        dic_code,
        dic_name
    from hudi_ods.ods_base_dic
    where parent_code='33'
) range_dic
on ci.range_type=range_dic.dic_code;
```

10.3 活动维度表 dim_activity

10.3.1 建表语句

```
CREATE TABLE hudi_dim.dim_activity
(
    `activity_rule_id`    int,
    `activity_id`        bigint,
    `activity_name`      varchar(200),
    `activity_type_code` varchar(20),
    `activity_type_name` varchar(100),
    `activity_desc`      varchar(2000),
    `start_time`         timestamp(0),
    `end_time`           timestamp(0),
    `create_time`        timestamp(0),
    `condition_amount`   DECIMAL(16, 2),
    `condition_num`      BIGINT,
    `benefit_amount`     DECIMAL(16, 2),
    `benefit_discount`   DECIMAL(10, 2),
    `benefit_rule`       STRING,
    `benefit_level`      bigint,
    `ts`                 timestamp(3),
    `dt`                 string
)
PARTITIONED BY (`dt`)
WITH (
    'connector'='hudi',
    'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dim/dim_activ
ity',
    'table.type'='MERGE_ON_READ',
    'hoodie.datasource.write.recordkey.field' = 'activity_id',
    'hoodie.datasource.write.precombine.field' = 'ts',
    'write.bucket_assign.tasks'='1',
    'write.tasks' = '1',
    'compaction.tasks' = '1',
    'compaction.async.enabled' = 'true',
    'compaction.schedule.enabled' = 'true',
    'compaction.trigger.strategy' = 'num_commits',
```

```
'compaction.delta_commits' = '5',
'read.streaming.enabled' = 'true',
'changelog.enabled' = 'true',
'read.streaming.skip_compaction' = 'true',
'hive_sync.enable'='true',
'hive_sync.mode' = 'hms',
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
'hive_sync.db'='hive_dim',
'hive_sync.table'='dim_activity');
```

10.3.2 数据装载

```
insert into hudi_dim.dim_activity
select
    rule.id,
    info.id,
    activity_name,
    rule.activity_type,
    dic.dic_name,
    activity_desc,
    start_time,
    end_time,
    create_time,
    condition_amount,
    condition_num,
    benefit_amount,
    benefit_discount,
    case rule.activity_type
        when '3101' then concat('满',cast(condition_amount as
string),'元减',cast(benefit_amount as string),'元')
        when '3102' then concat('满',cast(condition_amount as
string),'件打',cast(10*(1-benefit_discount) as string),'折')
        when '3103' then concat('打',cast(10*(1-benefit_discount)
as string),'折')
    end benefit_rule,
    benefit_level,
    LOCALTIMESTAMP as ts,
    info.dt
from hudi_ods.ods_activity_rule rule
left join hudi_ods.ods_activity_info info
on rule.activity_id=info.id
left join
(
    select
        dic_code,
        dic_name
    from hudi_ods.ods_base_dic
    where parent_code='31'
) dic
on rule.activity_type=dic.dic_code;
```

10.4 地区维度表 dim_province

10.4.1 建表语句

```
CREATE TABLE hudi_dim.dim_province
```

```
(
  `id`          bigint,
  `province_name` varchar(20),
  `area_code`   varchar(20),
  `iso_code`     varchar(20),
  `iso_3166_2`  varchar(20),
  `region_id`   varchar(20),
  `region_name` varchar(20),
  `ts` timestamp(3)
) WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dim/dim_province',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '1',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_dim',
  'hive_sync.table'='dim_province');
```

10.4.2 数据装载

```
insert into hudi_dim.dim_province
select
  province.id,
  province.name,
  province.area_code,
  province.iso_code,
  province.iso_3166_2,
  region_id,
  region_name,
  LOCALTIMESTAMP as ts
from hudi_ods.ods_base_province province
left join hudi_ods.ods_base_region region
on province.region_id=region.id;
```

10.5 日期维度表 dim_date

10.5.1 建表语句

```
CREATE TABLE hudi_dim.dim_date
(
  `date_id`    STRING,
  `week_id`    STRING,
```

```
`week_day`    STRING,
`day`         STRING,
`month`       STRING,
`quarter`     STRING,
`year`        STRING,
`is_workday`  STRING,
`holiday_id`  STRING,
`ts`          TIMESTAMP(3)
) WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dim/dim_date'
,
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'date_id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '1',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '1',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_dim',
  'hive_sync.table'='dim_date');
```

10.5.2 数据装载

通常情况下，时间维度表的数据并不是来自于业务系统，而是手动写入，并且由于时间维度表数据的可预见性，无须每日导入，一般可一次性导入一年的数据。

(1) 将 HDFS 的文件映射为一张表

```
CREATE TABLE myflink.date_file_source (
  `date_id`    STRING,
  `week_id`    STRING,
  `week_day`   STRING,
  `day`        STRING,
  `month`      STRING,
  `quarter`    STRING,
  `year`       STRING,
  `is_workday` STRING,
  `holiday_id` STRING
) WITH (
  'connector' = 'filesystem',
  'path' = 'hdfs://hadoop102:8020/tmp_data/date_info.csv',
  'format' = 'csv'
);
```

(2) 将数据文件上传到 HDFS 上临时表路径/tmp_data/date_info.csv



date_info.csv

(3) 执行以下语句将其导入时间维度表

```
insert into hudi_dim.dim_date
select
    *,
    LOCALTIMESTAMP as ts
from myflink.date_file_source;
```

10.6 用户维度表 dim_user_info

10.6.1 建表语句

```
CREATE TABLE hudi_dim.dim_user_info
(
    `id` bigint,
    `login_name` varchar(200),
    `nick_name` varchar(200),
    `name` varchar(200),
    `phone_num` varchar(200),
    `email` varchar(200),
    `user_level` varchar(200),
    `birthday` date,
    `gender` varchar(1),
    `create_time` timestamp(0),
    `operate_time` timestamp(0),
    `start_time` timestamp(0),
    `ts` timestamp(3),
    `dt` string
)
PARTITIONED BY (`dt`)
WITH (
    'connector'='hudi',
    'path'
    ='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dim/dim_user_
    info',
    'table.type'='MERGE_ON_READ',
    'hoodie.datasource.write.recordkey.field' = 'id',
    'hoodie.datasource.write.precombine.field' = 'ts',
    'write.bucket_assign.tasks'='1',
    'write.tasks' = '1',
    'compaction.tasks' = '1',
    'compaction.async.enabled' = 'true',
    'compaction.schedule.enabled' = 'true',
    'compaction.trigger.strategy' = 'num_commits',
    'compaction.delta_commits' = '5',
    'read.streaming.enabled' = 'true',
    'changelog.enabled' = 'true',
    'read.streaming.skip_compaction' = 'true',
    'hive_sync.enable'='true',
    'hive_sync.mode' = 'hms',
```

```
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',  
'hive_sync.db'='hive_dim',  
'hive_sync.table'='dim_user_info');
```

10.6.2 数据装载

```
insert into hudi_dim.dim_user_info  
select  
    id,  
    login_name,  
    nick_name,  
    md5(name) name,  
  
    md5(if(regexexp(phone_num, '^(13[0-9]|14[01456879]|15[0-35-9]|16[2567]|17[0-8]|18[0-9]|19[0-35-9])\\d{8}$'), phone_num, CAST(NULL AS STRING))) phone_num,  
  
    md5(if(regexexp(email, '^([a-zA-Z0-9_-])+@[a-zA-Z0-9_-]+(\\.([a-zA-Z0-9_-]+)+)$'), email, CAST(NULL AS STRING))) email,  
    user_level,  
    birthday,  
    gender,  
    create_time,  
    operate_time,  
    nvl(operate_time, create_time) as start_time,  
    LOCALTIMESTAMP as ts,  
    date_format(create_time, 'yyyy-MM-dd') as dt  
from hudi_ods.ods_user_info;
```

第 11 章 湖仓一体之 DWD 层

DWD 层设计要点:

- 1) DWD 层的设计依据是维度建模理论, 该层存储维度模型的事实表。
- 2) DWD 层的数据存储格式为 orc 列式存储+snappy 压缩。
- 3) DWD 层表名的命名规范为 `dwd_数据域_表名_单分区增量全量标识 (inc/full)`

注意: 如果插入数据的 sql 中有聚合操作, hudi 会读取撤回流之前的数据, 导致存在部分空值, 如果后续使用到 `groupBy` 等语法, 需要手动过滤分组字段不为空。

11.1 交易域加购事务事实表 `dwd_trade_cart_add`

11.1.1 建表语句

```
CREATE TABLE hudi_dwd.dwd_trade_cart_add  
(  
    `id` BIGINT,  
    `user_id` VARCHAR(200),  
    `sku_id` BIGINT,  
    `date_id` STRING,  
    `create_time` TIMESTAMP(0),
```



```
`source_id`          BIGINT,
`source_type_code`   VARCHAR(20),
`source_type_name`   varchar(100),
`sku_num`            INT,
`ts`                  TIMESTAMP(3),
`dt`                  STRING
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dwd/dwd_trade
_cart_add',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '1',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_dwd',
  'hive_sync.table'='dwd_trade_cart_add'
);
```

11.1.2 数据装载

```
insert into hudi_dwd.dwd_trade_cart_add
select
  cart.id,
  cart.user_id,
  cart.sku_id,
  cart.dt_date_id,
  cart.create_time,
  cart.source_id,
  cart.source_type,
  dic.source_type_name,
  cart.sku_num,
  cart.ts,
  cart.dt
from hudi_ods.ods_cart_info cart
left join
(
  select
    dic_code,
    dic_name source_type_name
  from hudi_ods.ods_base_dic
  where parent_code='24'
) dic
```

```
on cart.source_type=dic.dic_code;
```

11.2 交易域下单事务事实表 `dwd_trade_order_detail`

11.2.1 建表语句

```
CREATE TABLE hudi_dwd.dwd_trade_order_detail
(
  `id` BIGINT,
  `order_id` BIGINT,
  `user_id` BIGINT,
  `sku_id` BIGINT,
  `province_id` INT,
  `activity_id` BIGINT,
  `activity_rule_id` BIGINT,
  `coupon_id` BIGINT,
  `date_id` STRING,
  `create_time` TIMESTAMP(0),
  `source_id` BIGINT,
  `source_type_code` VARCHAR(20),
  `source_type_name` VARCHAR(100),
  `sku_num` BIGINT,
  `split_original_amount` DECIMAL(16, 2),
  `split_activity_amount` DECIMAL(16, 2),
  `split_coupon_amount` DECIMAL(16, 2),
  `split_total_amount` DECIMAL(16, 2),
  `ts` TIMESTAMP(3),
  `dt` STRING
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dwd/dwd_trade_order_detail',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '1',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_dwd',
  'hive_sync.table'='dwd_trade_order_detail'
);
```

11.2.2 数据装载

```
insert into hudi_dwd.dwd_trade_order_detail
select
    od.id,
    od.order_id,
    oi.user_id,
    od.sku_id,
    oi.province_id,
    act.activity_id,
    act.activity_rule_id,
    cou.coupon_id,
    od.dt date_id,
    od.create_time,
    od.source_id,
    od.source_type,
    dic.dic_name,
    od.sku_num,
    od.sku_num * od.order_price split_original_amount,
    nvl(od.split_activity_amount,0.0) split_activity_amount,
    nvl(od.split_coupon_amount,0.0) split_coupon_amount,
    od.split_total_amount,
    od.ts,
    od.dt
from hudi_ods.ods_order_detail od
left join
(
    select
        id,
        user_id,
        province_id
    from hudi_ods.ods_order_info
) oi
on od.order_id = oi.id
left join
(
    select
        order_detail_id,
        activity_id,
        activity_rule_id
    from hudi_ods.ods_order_detail_activity
) act
on od.id = act.order_detail_id
left join
(
    select
        order_detail_id,
        coupon_id
    from hudi_ods.ods_order_detail_coupon
) cou
on od.id = cou.order_detail_id
left join
(
    select
        dic_code,
        dic_name
```

```
from hudi_ods.ods_base_dic
where parent_code='24'
)dic
on od.source_type=dic.dic_code;
```

11.3 交易域支付成功事务事实表 dwd_trade_pay_detail_suc

11.3.1 建表语句

```
CREATE TABLE hudi_dwd.dwd_trade_pay_detail_suc
(
  `id` BIGINT,
  `order_id` BIGINT,
  `user_id` BIGINT,
  `sku_id` BIGINT,
  `province_id` INT,
  `activity_id` BIGINT,
  `activity_rule_id` BIGINT,
  `coupon_id` BIGINT,
  `payment_type_code` VARCHAR(20),
  `payment_type_name` VARCHAR(100),
  `date_id` STRING,
  `callback_time` TIMESTAMP(0),
  `source_id` BIGINT,
  `source_type_code` VARCHAR(20),
  `source_type_name` VARCHAR(100),
  `sku_num` BIGINT,
  `split_original_amount` DECIMAL(16, 2),
  `split_activity_amount` DECIMAL(16, 2),
  `split_coupon_amount` DECIMAL(16, 2),
  `split_payment_amount` DECIMAL(16, 2),
  `ts` TIMESTAMP(3),
  `dt` STRING
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dwd/dwd_trade_pay_detail_suc',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '1',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_dwd',
```

```
'hive_sync.table'='dwd_trade_pay_detail_suc'
);
```

11.3.2 数据装载

```
insert into hudi_dwd.dwd_trade_pay_detail_suc
select
    od.id,
    od.order_id,
    pi.user_id,
    od.sku_id,
    oi.province_id,
    act.activity_id,
    act.activity_rule_id,
    cou.coupon_id,
    pi.payment_type,
    pay_dic.dic_name,
    date_format(pi.callback_time,'yyyy-MM-dd') date_id,
    pi.callback_time,
    od.source_id,
    od.source_type,
    src_dic.dic_name,
    od.sku_num,
    od.sku_num * od.order_price split_original_amount,
    nvl(od.split_activity_amount,0.0) split_activity_amount,
    nvl(od.split_coupon_amount,0.0) split_coupon_amount,
    od.split_total_amount,
    od.ts,
    od.dt
from hudi_ods.ods_order_detail od
join
(
    select
        user_id,
        order_id,
        payment_type,
        callback_time
    from hudi_ods.ods_payment_info
    where payment_status='1602'
) pi
on od.order_id=pi.order_id
left join
(
    select
        id,
        province_id
    from hudi_ods.ods_order_info
) oi
on od.order_id = oi.id
left join
(
    select
        order_detail_id,
        activity_id,
        activity_rule_id
    from hudi_ods.ods_order_detail_activity
) act
```

```
on od.id = act.order_detail_id
left join
(
    select
        order_detail_id,
        coupon_id
    from hudi_ods.ods_order_detail_coupon
) cou
on od.id = cou.order_detail_id
left join
(
    select
        dic_code,
        dic_name
    from hudi_ods.ods_base_dic
    where parent_code='11'
) pay_dic
on pi.payment_type=pay_dic.dic_code
left join
(
    select
        dic_code,
        dic_name
    from hudi_ods.ods_base_dic
    where parent_code='24'
) src_dic
on od.source_type=src_dic.dic_code;
```

11.4 交易域交易流程累积快照事实表 dwd_trade_trade_flow_acc

11.4.1 建表语句

```
CREATE TABLE hudi_dwd.dwd_trade_trade_flow_acc
(
    `order_id`          BIGINT,
    `user_id`           BIGINT,
    `province_id`       INT,
    `order_date_id`     STRING,
    `order_time`        TIMESTAMP(0),
    `payment_date_id`   STRING,
    `payment_time`      TIMESTAMP(0),
    `finish_date_id`    STRING,
    `finish_time`       TIMESTAMP(0),
    `order_original_amount` DECIMAL(16, 2),
    `order_activity_amount` DECIMAL(16, 2),
    `order_coupon_amount` DECIMAL(16, 2),
    `order_total_amount` DECIMAL(10, 2),
    `payment_amount`    DECIMAL(10, 2),
    `ts`                TIMESTAMP(3),
    `dt`                STRING
)
PARTITIONED BY (`dt`)
WITH (
    'connector'='hudi',
    'path'
    ='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dwd/dwd_trade_trade_flow_acc',
```

```
'table.type'='MERGE_ON_READ',
'hoodie.datasource.write.recordkey.field' = 'order_id',
'hoodie.datasource.write.precombine.field' = 'ts',
'write.bucket_assign.tasks'='1',
'write.tasks' = '1',
'compaction.tasks' = '1',
'compaction.async.enabled' = 'true',
'compaction.schedule.enabled' = 'true',
'compaction.trigger.strategy' = 'num_commits',
'compaction.delta_commits' = '5',
'read.streaming.enabled' = 'true',
'changelog.enabled' = 'true',
'read.streaming.skip_compaction' = 'true',
'hive_sync.enable'='true',
'hive_sync.mode' = 'hms',
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
'hive_sync.db'='hive_dwd',
'hive_sync.table'='dwd_trade_trade_flow_acc'
);
```

11.4.2 数据装载

```
insert into hudi_dwd.dwd_trade_trade_flow_acc
select
    oi.id,
    oi.user_id,
    oi.province_id,
    date_format(oi.create_time,'yyyy-MM-dd'),
    oi.create_time,
    date_format(pi.callback_time,'yyyy-MM-dd'),
    pi.callback_time,
    date_format(log.operate_time,'yyyy-MM-dd'),
    log.operate_time,
    oi.original_total_amount,
    oi.activity_reduce_amount,
    oi.coupon_reduce_amount,
    oi.total_amount,
    nvl(pi.payment_amount,0.0),
    oi.ts,
    oi.dt
from hudi_ods.ods_order_info oi
left join
(
    select
        order_id,
        callback_time,
        total_amount payment_amount
    from hudi_ods.ods_payment_info
    where payment_status='1602'
)pi
on oi.id=pi.order_id
left join
(
    select
        order_id,
        operate_time
    from hudi_ods.ods_order_status_log
```

```
    where order_status='1004'
)log
on oi.id=log.order_id;
```

11.5 工具域优惠券使用(支付)事务事实表 dwd_tool_coupon_used

11.5.1 建表语句

```
CREATE TABLE hudi_dwd.dwd_tool_coupon_used
(
  `id`          BIGINT,
  `coupon_id`   BIGINT,
  `user_id`     BIGINT,
  `order_id`    BIGINT,
  `date_id`     STRING,
  `payment_time` TIMESTAMP(0),
  `ts`          TIMESTAMP(3),
  `dt`          STRING
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/dwd/dwd_tool_coupon_used',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_dwd',
  'hive_sync.table'='dwd_tool_coupon_used'
);
```

11.5.2 数据装载

```
insert into hudi_dwd.dwd_tool_coupon_used
select
  id,
  coupon_id,
  user_id,
  order_id,
  date_format(used_time,'yyyy-MM-dd') date_id,
  used_time,
  ts,
  dt
```



```
from hudi_ods.ods_coupon_use
where used_time is not null;
```

11.6 互动域收藏商品事务事实表 `dwd_interaction_favor_add`

11.6.1 建表语句

```
CREATE TABLE hudi_dwd.dwd_interaction_favor_add
(
  `id`          BIGINT,
  `user_id`     BIGINT,
  `sku_id`      BIGINT,
  `date_id`     STRING,
  `create_time` TIMESTAMP(0),
  `ts`          TIMESTAMP(3),
  `dt`          STRING
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dwd/dwd_inter
action_favor_add',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '1',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_dwd',
  'hive_sync.table'='dwd_interaction_favor_add'
);
```

11.6.2 数据装载

```
insert into hudi_dwd.dwd_interaction_favor_add
select
  id,
  user_id,
  sku_id,
  date_format(create_time,'yyyy-MM-dd') date_id,
  create_time,
  ts,
  dt
from hudi_ods.ods_favor_info;
```

11.7 流量域页面浏览事务事实表 `dwd_traffic_page_view`

11.7.1 建表语句

```
CREATE TABLE hudi_dwd.dwd_traffic_page_view
(
  `uuid`          STRING,
  `province_id`   BIGINT,
  `brand`         STRING,
  `channel`       STRING,
  `is_new`        STRING,
  `model`         STRING,
  `mid_id`        STRING,
  `operate_system` STRING,
  `user_id`       STRING,
  `version_code`  STRING,
  `page_item`     STRING,
  `page_item_type` STRING,
  `last_page_id`  STRING,
  `page_id`       STRING,
  `source_type`   STRING,
  `date_id`       STRING,
  `view_time`     STRING,
  `session_id`    STRING,
  `during_time`   STRING,
  `ts`            BIGINT,
  `dt`            STRING
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/dwd/dwd_traffic_page_view',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'uuid',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_dwd',
  'hive_sync.table'='dwd_traffic_page_view'
);
```

11.7.2 数据装载

```
insert into hudi_dwd.dwd_traffic_page_view
select
    uuid,
    province_id,
    brand,
    channel,
    is_new,
    model,
    mid_id,
    operate_system,
    user_id,
    version_code,
    page_item,
    page_item_type,
    last_page_id,
    page_id,
    source_type,
    date_format(from_utc_timestamp(ts, 'GMT+8'), 'yyyy-MM-dd')
date_id,
    date_format(from_utc_timestamp(ts, 'GMT+8'), 'yyyy-MM-dd
HH:mm:ss') view_time,
    session_id,
    during_time,
    ts,
    dt
from
(
    select
        uuid,
        common.ar area_code,
        common.ba brand,
        common.ch channel,
        common.is_new is_new,
        common.md model,
        common.mid mid_id,
        common.os operate_system,
        common.uid user_id,
        common.vc version_code,
        page.during_time,
        page.item page_item,
        page.item_type page_item_type,
        page.last_page_id,
        page.page_id,
        page.source_type,
        ts,
        concat(common.mid, '-', last_value(if(page.last_page_id is
null, ts, cast(null as string))) over (partition by common.mid order
by t)) session_id,
        dt
    from
        hudi_ods.ods_log
    OPTIONS('read.tasks'='1', 'read.streaming.check-interval'
=
'4')*/
    where page is not null
)log
```

```
left join
(
  select
    id province_id,
    area_code
  from      hudi_ods.ods_base_province
/*+
OPTIONS('read.tasks'='1','read.streaming.check-interval'
'4')*/
)bp
on log.area_code=bp.area_code;
```

11.8 用户域用户注册事务事实表 dwd_user_register

11.8.1 建表语句

```
CREATE TABLE hudi_dwd.dwd_user_register
(
  `user_id`      BIGINT,
  `date_id`      STRING,
  `create_time`  TIMESTAMP(0),
  `channel`      STRING,
  `province_id`  BIGINT,
  `version_code` STRING,
  `mid_id`       STRING,
  `brand`        STRING,
  `model`        STRING,
  `operate_system` STRING,
  `ts`           BIGINT,
  `dt`           STRING
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/dwd/dwd_user_register',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'user_id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '4',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_dwd',
  'hive_sync.table'='dwd_user_register'
);
```

11.8.2 数据装载

```
insert into hudi_dwd.dwd_user_register
select
    ui.user_id,
    date_format(ui.create_time,'yyyy-MM-dd') date_id,
    ui.create_time,
    log.channel,
    bp.province_id,
    log.version_code,
    log.mid_id,
    log.brand,
    log.model,
    log.operate_system,
    ui.ts,
    date_format(ui.create_time,'yyyy-MM-dd') dt
from
    (
        select
            id user_id,
            create_time
        from hudi_ods.ods_user_info
    )ui
left join
    (
        select
            common.ar area_code,
            common.ba brand,
            common.ch channel,
            common.md model,
            common.mid mid_id,
            common.os operate_system,
            common.uid user_id,
            common.vc version_code,
            ts
        from hudi_ods.ods_log
        where page.page_id='register'
        and common.uid is not null
    )log
on cast(ui.user_id as string)=log.user_id
left join
    (
        select
            id province_id,
            area_code
        from hudi_ods.ods_base_province
    )bp
on log.area_code=bp.area_code;
```

11.9 用户域用户登录事务事实表 dwd_user_login

11.9.1 建表语句

```
CREATE TABLE hudi_dwd.dwd_user_login
(
    `user_id`          STRING,
```

```
`date_id`      STRING,
`login_time`   STRING,
`channel`      STRING,
`province_id`  BIGINT,
`version_code` STRING,
`mid_id`       STRING,
`brand`        STRING,
`model`        STRING,
`operate_system` STRING,
`ts`           TIMESTAMP(3),
`dt`           STRING
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/dwd/dwd_user_login',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'user_id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '1',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_dwd',
  'hive_sync.table'='dwd_user_login'
);
```

11.9.2 数据装载

```
insert into hudi_dwd.dwd_user_login
select
  user_id,
  date_format(from_utc_timestamp(ts,'GMT+8'),'yyyy-MM-dd')
date_id,
  date_format(from_utc_timestamp(ts,'GMT+8'),'yyyy-MM-dd
HH:mm:ss') login_time,
  channel,
  province_id,
  version_code,
  mid_id,
  brand,
  model,
  operate_system,
  ts,
  dt
from
```

```
(
    select
        user_id,
        channel,
        area_code,
        version_code,
        mid_id,
        brand,
        model,
        operate_system,
        ts,
        dt
    from
        (
            select
                user_id,
                channel,
                area_code,
                version_code,
                mid_id,
                brand,
                model,
                operate_system,
                ts,
                dt,
                row_number() over (partition by session_id order by ts)
            rn
        from
            (
                select
                    common.uid user_id,
                    common.ch channel,
                    common.ar area_code,
                    common.vc version_code,
                    common.mid mid_id,
                    common.ba brand,
                    common.md model,
                    common.os operate_system,
                    ts,
                    concat(common.mid, '-', last_value(if(page.last_page_id is
                    null,ts,cast(null as string))) over (partition by common.mid order
                    by t)) session_id,
                    dt
                from hudi_ods.ods_log
                where page is not null
            )t1
            where user_id is not null
        )t2
        where rn=1
    )t3
left join
(
    select
        id province_id,
        area_code
    from hudi_ods.ods_base_province
```

```
)bp  
on t3.area_code=bp.area_code;
```

第 12 章 湖仓一体之 DWS 层

设计要点:

- 1) DWS 层的设计参考指标体系。
 - 2) DWS 层表名的命名规范为 dws_数据域_统计粒度_业务过程_统计周期 (1d/nd/td)
- 注: 1d 表示最近 1 日, nd 表示最近 n 日, td 表示历史至今。
- 3) 使用 flink 的 sum()函数才可以回滚更新

```
use modules core,hive;
```

12.1 最近 1 日汇总表

12.1.1 交易域用户商品粒度订单最近 1 日汇总表

dws_trade_user_sku_order_1d

1) 建表语句

```
CREATE TABLE hudi_dws.dws_trade_user_sku_order_1d  
(  
  `user_id`          BIGINT,  
  `sku_id`           BIGINT,  
  `sku_name`         VARCHAR(200),  
  `category1_id`     BIGINT,  
  `category1_name`   VARCHAR(10),  
  `category2_id`     BIGINT,  
  `category2_name`   VARCHAR(200),  
  `category3_id`     BIGINT,  
  `category3_name`   VARCHAR(200),  
  `tm_id`            BIGINT,  
  `tm_name`          VARCHAR(100),  
  `order_count_1d`   BIGINT,  
  `order_num_1d`     BIGINT,  
  `order_original_amount_1d` DECIMAL(16, 2),  
  `activity_reduce_amount_1d` DECIMAL(16, 2),  
  `coupon_reduce_amount_1d` DECIMAL(16, 2),  
  `order_total_amount_1d` DECIMAL(16, 2),  
  `ts`               TIMESTAMP(3),  
  `dt`               STRING  
)  
PARTITIONED BY (`dt`)  
WITH (  
  'connector'='hudi',  
  'path'  
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dws/dws_trade  
_user_sku_order_1d',  
  'table.type'='MERGE_ON_READ',
```



```
'hoodie.datasource.write.recordkey.field' =  
'user_id,sku_id',  
'hoodie.datasource.write.precombine.field' = 'ts',  
'write.bucket_assign.tasks'='1',  
'write.tasks' = '1',  
'compaction.tasks' = '1',  
'compaction.async.enabled' = 'true',  
'compaction.schedule.enabled' = 'true',  
'compaction.trigger.strategy' = 'num_commits',  
'compaction.delta_commits' = '5',  
'read.streaming.enabled' = 'true',  
'changelog.enabled' = 'true',  
'read.streaming.skip_compaction' = 'true',  
'hive_sync.enable'='true',  
'hive_sync.mode' = 'hms',  
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',  
'hive_sync.db'='hive_dws',  
'hive_sync.table'='dws_trade_user_sku_order_1d'  
);
```

2) 数据装载

```
insert into hudi_dws.dws_trade_user_sku_order_1d  
select  
    user_id,  
    id,  
    sku_name,  
    category1_id,  
    category1_name,  
    category2_id,  
    category2_name,  
    category3_id,  
    category3_name,  
    tm_id,  
    tm_name,  
    order_count_1d,  
    order_num_1d,  
    order_original_amount_1d,  
    activity_reduce_amount_1d,  
    coupon_reduce_amount_1d,  
    order_total_amount_1d,  
    LOCALTIMESTAMP as ts,  
    dt  
from  
(  
    select  
        dt,  
        user_id,  
        sku_id,  
        count(*) order_count_1d,  
        sum(sku_num) order_num_1d,  
        sum(split_original_amount) order_original_amount_1d,  
        sum(nvl(split_activity_amount,0.0))  
activity_reduce_amount_1d,  
        sum(nvl(split_coupon_amount,0.0))  
coupon_reduce_amount_1d,  
        sum(split_total_amount) order_total_amount_1d  
    from  
        hudi_dwd.dwd_trade_order_detail/*+
```

```
OPTIONS('read.tasks'='1')*/
  where user_id is not null and sku_id is not null
  group by dt,user_id,sku_id
)od
left join
(
  select
    id,
    sku_name,
    category1_id,
    category1_name,
    category2_id,
    category2_name,
    category3_id,
    category3_name,
    tm_id,
    tm_name
  from hudi_dim.dim_sku/*+ OPTIONS('read.tasks'='1')*/
)sku
on od.sku_id=sku.id;
```

12.1.2 交易域用户粒度订单最近 1 日汇总表

dws_trade_user_order_1d

1) 建表语句

```
CREATE TABLE hudi_dws.dws_trade_user_order_1d
(
  `user_id`          BIGINT,
  `order_count_1d`    BIGINT,
  `order_num_1d`      BIGINT,
  `order_original_amount_1d` DECIMAL(16, 2),
  `activity_reduce_amount_1d` DECIMAL(16, 2),
  `coupon_reduce_amount_1d` DECIMAL(16, 2),
  `order_total_amount_1d` DECIMAL(16, 2),
  `ts`               TIMESTAMP(3),
  `dt`               STRING
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dws/dws_trade_user_order_1d',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'user_id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '1',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
```

```
'read.streaming.skip_compaction' = 'true',
'hive_sync.enable'='true',
'hive_sync.mode' = 'hms',
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
'hive_sync.db'='hive_dws',
'hive_sync.table'='dws_trade_user_order_1d'
);
```

2) 数据装载

```
insert into hudi_dws.dws_trade_user_order_1d
select
    user_id,
    count(distinct(order_id)),
    sum(sku_num),
    sum(split_original_amount),
    sum(nvl(split_activity_amount,0)),
    sum(nvl(split_coupon_amount,0)),
    sum(split_total_amount),
    LOCALTIMESTAMP as ts,
    dt
from
    hudi_dwd.dwd_trade_order_detail/*+
OPTIONS('read.tasks'='1')*/
group by user_id,dt;
```

12.1.3 交易域用户粒度加购最近 1 日汇总表

dws_trade_user_cart_add_1d

1) 建表语句

```
CREATE TABLE hudi_dws.dws_trade_user_cart_add_1d
(
    `user_id`          VARCHAR(200),
    `cart_add_count_1d` BIGINT,
    `cart_add_num_1d`  BIGINT,
    `ts`               TIMESTAMP(3),
    `dt`               STRING
)
PARTITIONED BY (`dt`)
WITH (
    'connector'='hudi',
    'path'
    ='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dws/dws_trade
    _user_cart_add_1d',
    'table.type'='MERGE_ON_READ',
    'hoodie.datasource.write.recordkey.field' = 'user_id',
    'hoodie.datasource.write.precombine.field' = 'ts',
    'write.bucket_assign.tasks'='1',
    'write.tasks' = '1',
    'compaction.tasks' = '1',
    'compaction.async.enabled' = 'true',
    'compaction.schedule.enabled' = 'true',
    'compaction.trigger.strategy' = 'num_commits',
    'compaction.delta_commits' = '5',
    'read.streaming.enabled' = 'true',
    'changelog.enabled' = 'true',
    'read.streaming.skip_compaction' = 'true',
```

```
'hive_sync.enable'='true',  
'hive_sync.mode' = 'hms',  
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',  
'hive_sync.db'='hive_dws',  
'hive_sync.table'='dws_trade_user_cart_add_1d'  
);
```

2) 数据装载

```
insert into hudi_dws.dws_trade_user_cart_add_1d  
select  
    user_id,  
    count(*),  
    sum(sku_num),  
    LOCALTIMESTAMP as ts,  
    dt  
from hudi_dwd.dwd_trade_cart_add/*+ OPTIONS('read.tasks'='1')*/  
group by user_id,dt;
```

12.1.4 交易域用户粒度支付最近 1 日汇总表

dws_trade_user_payment_1d

1) 建表语句

```
CREATE TABLE hudi_dws.dws_trade_user_payment_1d  
(  
    `user_id`          BIGINT,  
    `payment_count_1d` BIGINT,  
    `payment_num_1d`   BIGINT,  
    `payment_amount_1d` DECIMAL(16, 2),  
    `ts`               TIMESTAMP(3),  
    `dt`               STRING  
)  
PARTITIONED BY (`dt`)  
WITH (  
    'connector'='hudi',  
    'path'  
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dws/dws_trade_user_payment_1d',  
    'table.type'='MERGE_ON_READ',  
    'hoodie.datasource.write.recordkey.field' = 'user_id',  
    'hoodie.datasource.write.precombine.field' = 'ts',  
    'write.bucket_assign.tasks'='1',  
    'write.tasks' = '1',  
    'compaction.tasks' = '1',  
    'compaction.async.enabled' = 'true',  
    'compaction.schedule.enabled' = 'true',  
    'compaction.trigger.strategy' = 'num_commits',  
    'compaction.delta_commits' = '5',  
    'read.streaming.enabled' = 'true',  
    'changelog.enabled' = 'true',  
    'read.streaming.skip_compaction' = 'true',  
    'hive_sync.enable'='true',  
    'hive_sync.mode' = 'hms',  
    'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',  
    'hive_sync.db'='hive_dws',  
    'hive_sync.table'='dws_trade_user_payment_1d'
```

```
);
```

2) 数据装载

```
insert into hudi_dws.dws_trade_user_payment_1d
select
    user_id,
    count(distinct(order_id)),
    sum(sku_num),
    sum(split_payment_amount),
    LOCALTIMESTAMP as ts,
    dt
from
    hudi_dwd.dwd_trade_pay_detail_suc/*+
OPTIONS('read.tasks'='1')*/
group by user_id,dt;
```

12.1.5 交易域省份粒度订单最近 1 日汇总表

dws_trade_province_order_1d

1) 建表语句

```
CREATE TABLE hudi_dws.dws_trade_province_order_1d
(
    `province_id`          INT,
    `province_name`        varchar(20),
    `area_code`            varchar(20),
    `iso_code`             varchar(20),
    `iso_3166_2`           varchar(20),
    `order_count_1d`       BIGINT,
    `order_original_amount_1d` DECIMAL(16, 2),
    `activity_reduce_amount_1d` DECIMAL(16, 2),
    `coupon_reduce_amount_1d` DECIMAL(16, 2),
    `order_total_amount_1d` DECIMAL(16, 2),
    `ts`                   TIMESTAMP(3),
    `dt`                   STRING
)
PARTITIONED BY (`dt`)
WITH (
    'connector'='hudi',
    'path'
    ='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dws/dws_trade_province_order_1d',
    'table.type'='MERGE_ON_READ',
    'hoodie.datasource.write.recordkey.field' = 'province_id',
    'hoodie.datasource.write.precombine.field' = 'ts',
    'write.bucket_assign.tasks'='1',
    'write.tasks' = '1',
    'compaction.tasks' = '1',
    'compaction.async.enabled' = 'true',
    'compaction.schedule.enabled' = 'true',
    'compaction.trigger.strategy' = 'num_commits',
    'compaction.delta_commits' = '5',
    'read.streaming.enabled' = 'true',
    'changelog.enabled' = 'true',
    'read.streaming.skip_compaction' = 'true',
    'hive_sync.enable'='true',
    'hive_sync.mode' = 'hms',
```

```
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
'hive_sync.db'='hive_dws',
'hive_sync.table'='dws_trade_province_order_1d'
);
```

2) 数据装载

```
insert into hudi_dws.dws_trade_province_order_1d
select
    province_id,
    province_name,
    area_code,
    iso_code,
    iso_3166_2,
    order_count_1d,
    order_original_amount_1d,
    activity_reduce_amount_1d,
    coupon_reduce_amount_1d,
    order_total_amount_1d,
    LOCALTIMESTAMP as ts,
    dt
from
(
    select
        province_id,
        count(distinct(order_id)) order_count_1d,
        sum(split_original_amount) order_original_amount_1d,
        sum(nvl(split_activity_amount,0))
activity_reduce_amount_1d,
        sum(nvl(split_coupon_amount,0)) coupon_reduce_amount_1d,
        sum(split_total_amount) order_total_amount_1d,
        dt
    from
        hudi_dwd.dwd_trade_order_detail/*+
OPTIONS('read.tasks'='1')*/
    group by province_id,dt
)o
left join
(
    select
        id,
        province_name,
        area_code,
        iso_code,
        iso_3166_2
    from hudi_dim.dim_province/*+ OPTIONS('read.tasks'='1')*/
)p
on o.province_id=p.id;
```

12.1.6 工具域用户优惠券粒度优惠券使用(支付)最近 1 日汇总表

dws_tool_user_coupon_coupon_used_1d

1) 建表语句

```
CREATE TABLE hudi_dws.dws_tool_user_coupon_coupon_used_1d
(
    `user_id`          BIGINT,
    `coupon_id`        BIGINT,
```

```
`coupon_name`      VARCHAR(100),
`coupon_type_code` VARCHAR(10),
`coupon_type_name` VARCHAR(100),
`benefit_rule`      STRING,
`used_count_ld`     BIGINT,
`ts`                 TIMESTAMP(3),
`dt`                 STRING
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dws/dws_tool_
user_coupon_coupon_used_ld',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field'
='user_id,coupon_id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '1',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_dws',
  'hive_sync.table'='dws_tool_user_coupon_coupon_used_ld'
);
```

2) 数据装载

```
insert into hudi_dws.dws_tool_user_coupon_coupon_used_ld
select
  user_id,
  coupon_id,
  coupon_name,
  coupon_type_code,
  coupon_type_name,
  benefit_rule,
  used_count,
  LOCALTIMESTAMP as ts,
  dt
from
(
  select
    dt,
    user_id,
    coupon_id,
    count(*) used_count
  from
    hudi_dwd.dwd_tool_coupon_used/*+
OPTIONS('read.tasks'='1')*/
  group by dt,user_id,coupon_id
```

```
)t1
left join
(
  select
    id,
    coupon_name,
    coupon_type_code,
    coupon_type_name,
    benefit_rule
  from hudi_dim.dim_coupon/*+ OPTIONS('read.tasks'='1')*/
)t2
on t1.coupon_id=t2.id;
```

12.1.7 互动域商品粒度收藏商品最近 1 日汇总表

dws_interaction_sku_favor_add_1d

1) 建表语句

```
CREATE TABLE hudi_dws.dws_interaction_sku_favor_add_1d
(
  `sku_id`          BIGINT,
  `sku_name`        VARCHAR(200),
  `category1_id`    BIGINT,
  `category1_name`  VARCHAR(10),
  `category2_id`    BIGINT,
  `category2_name`  VARCHAR(200),
  `category3_id`    BIGINT,
  `category3_name`  VARCHAR(200),
  `tm_id`           BIGINT,
  `tm_name`         VARCHAR(100),
  `favor_add_count_1d` BIGINT,
  `ts`              TIMESTAMP(3),
  `dt`              STRING
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dws/dws_inter
action_sku_favor_add_1d',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'sku_id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '1',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
```



```
'hive_sync.db'='hive_dws',  
'hive_sync.table'='dws_interaction_sku_favor_add_1d'  
);
```

2) 数据装载

```
insert into hudi_dws.dws_interaction_sku_favor_add_1d  
select  
    sku_id,  
    sku_name,  
    category1_id,  
    category1_name,  
    category2_id,  
    category2_name,  
    category3_id,  
    category3_name,  
    tm_id,  
    tm_name,  
    favor_add_count,  
    LOCALTIMESTAMP as ts,  
    dt  
from  
(  
    select  
        dt,  
        sku_id,  
        count(*) favor_add_count  
    from  
        hudi_dwd.dwd_interaction_favor_add/*+  
OPTIONS('read.tasks'='1')*/  
    group by dt,sku_id  
)favor  
left join  
(  
    select  
        id,  
        sku_name,  
        category1_id,  
        category1_name,  
        category2_id,  
        category2_name,  
        category3_id,  
        category3_name,  
        tm_id,  
        tm_name  
    from hudi_dim.dim_sku/*+ OPTIONS('read.tasks'='1')*/  
)sku  
on favor.sku_id=sku.id;
```

12.1.8 流量域会话粒度页面浏览最近 1 日汇总表

dws_traffic_session_page_view_1d

1) 建表语句

```
CREATE TABLE hudi_dws.dws_traffic_session_page_view_1d  
(  
    `session_id`    STRING,  
    `mid_id`        STRING,
```

```
`brand`          STRING,
`model`          STRING,
`operate_system` STRING,
`version_code`   STRING,
`channel`        STRING,
`during_time_1d` BIGINT,
`page_count_1d`  BIGINT,
`ts`             TIMESTAMP(3),
`dt`            STRING
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dws/dws_traffic_session_page_view_1d',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field'
='session_id,mid_id,brand,model,operate_system,version_code,channel',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '1',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_dws',
  'hive_sync.table'='dws_traffic_session_page_view_1d'
);
```

2) 数据装载

```
insert into hudi_dws.dws_traffic_session_page_view_1d
select
  session_id,
  mid_id,
  brand,
  model,
  operate_system,
  version_code,
  channel,
  sum(cast(during_time as bigint)),
  count(*),
  LOCALTIMESTAMP as ts,
  dt
from          hudi_dwd.dwd_traffic_page_view/*+
OPTIONS('read.tasks'='1')*/
group by
dt,session_id,mid_id,brand,model,operate_system,version_code,channel;
```

12.1.9 流量域访客页面粒度页面浏览最近 1 日汇总表

dws_traffic_page_visitor_page_view_1d

1) 建表语句

```
CREATE TABLE hudi_dws.dws_traffic_page_visitor_page_view_1d
(
  `mid_id`          STRING,
  `brand`           STRING,
  `model`           STRING,
  `operate_system`  STRING,
  `page_id`         STRING,
  `during_time_1d`  BIGINT,
  `view_count_1d`   BIGINT,
  `ts`              TIMESTAMP(3),
  `dt`              STRING
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dws/dws_traffic_page_visitor_page_view_1d',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field'
='mid_id,brand,model,operate_system,page_id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '1',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_dws',
  'hive_sync.table'='dws_traffic_page_visitor_page_view_1d'
);
```

2) 数据装载

```
insert into hudi_dws.dws_traffic_page_visitor_page_view_1d
select
  mid_id,
  brand,
  model,
  operate_system,
  page_id,
  sum(cast(during_time as bigint)),
  count(*),
  LOCALTIMESTAMP as ts,
```

```
dt
from                                hudi_dwd.dwd_traffic_page_view/*+
OPTIONS('read.tasks'='1')*/
group by dt,mid_id,brand,model,operate_system,page_id;
```

12.2 最近 n 日汇总表

12.2.1 交易域用户商品粒度订单最近 n 日汇总表

dws_trade_user_sku_order_nd

1) 建表语句

```
CREATE TABLE hudi_dws.dws_trade_user_sku_order_nd
(
  `user_id`          BIGINT,
  `sku_id`           BIGINT,
  `sku_name`         VARCHAR(200),
  `category1_id`     BIGINT,
  `category1_name`   VARCHAR(10),
  `category2_id`     BIGINT,
  `category2_name`   VARCHAR(200),
  `category3_id`     BIGINT,
  `category3_name`   VARCHAR(200),
  `tm_id`            BIGINT,
  `tm_name`          VARCHAR(100),
  `order_count_7d`   BIGINT,
  `order_num_7d`     BIGINT,
  `order_original_amount_7d` DECIMAL(38, 2),
  `activity_reduce_amount_7d` DECIMAL(38, 2),
  `coupon_reduce_amount_7d` DECIMAL(38, 2),
  `order_total_amount_7d` DECIMAL(38, 2),
  `order_count_30d`  BIGINT,
  `order_num_30d`    BIGINT,
  `order_original_amount_30d` DECIMAL(38, 2),
  `activity_reduce_amount_30d` DECIMAL(38, 2),
  `coupon_reduce_amount_30d` DECIMAL(38, 2),
  `order_total_amount_30d` DECIMAL(38, 2),
  `ts`               TIMESTAMP(3),
  `dt`               STRING
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dws/dws_trade
_user_sku_order_nd',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field'
='user_id,sku_id,sku_name,category1_id,category1_name,category2
_id,category2_name,category3_id,category3_name,tm_id,tm_name',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '1',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
```

```
'compaction.schedule.enabled' = 'true',
'compaction.trigger.strategy' = 'num_commits',
'compaction.delta_commits' = '5',
'read.streaming.enabled' = 'true',
'changelog.enabled' = 'true',
'read.streaming.skip_compaction' = 'true',
'hive_sync.enable'='true',
'hive_sync.mode' = 'hms',
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
'hive_sync.db'='hive_dws',
'hive_sync.table'='dws_trade_user_sku_order_nd'
);
```

2) 数据装载

```
insert into hudi_dws.dws_trade_user_sku_order_nd
select
  user_id,
  sku_id,
  sku_name,
  category1_id,
  category1_name,
  category2_id,
  category2_name,
  category3_id,
  category3_name,
  tm_id,
  tm_name,
  sum(if(dt>=date_add(CURRENT_DATE,-6),order_count_1d,0)),
  sum(if(dt>=date_add(CURRENT_DATE,-6),order_num_1d,0)),

  sum(if(dt>=date_add(CURRENT_DATE,-6),order_original_amount_1d,
0)),

  sum(if(dt>=date_add(CURRENT_DATE,-6),activity_reduce_amount_1d,
0)),

  sum(if(dt>=date_add(CURRENT_DATE,-6),coupon_reduce_amount_1d,0)
),

  sum(if(dt>=date_add(CURRENT_DATE,-6),order_total_amount_1d,0))
,
  sum(order_count_1d),
  sum(order_num_1d),
  sum(order_original_amount_1d),
  sum(activity_reduce_amount_1d),
  sum(coupon_reduce_amount_1d),
  sum(order_total_amount_1d),
  LOCALTIMESTAMP as ts,
  cast(CURRENT_DATE as STRING) dt
from      hudi_dws.dws_trade_user_sku_order_1d/*+
OPTIONS('read.tasks'='1')*/
where dt >= date_add(CURRENT_DATE,-29)
group by
user_id,sku_id,sku_name,category1_id,category1_name,category2_
id,category2_name,category3_id,category3_name,tm_id,tm_name;
```

12.2.2 交易域省份粒度订单最近 n 日汇总表

dws_trade_province_order_nd

1) 建表语句

```
CREATE TABLE hudi_dws.dws_trade_province_order_nd
(
  `province_id`          INT,
  `province_name`        varchar(20),
  `area_code`            varchar(20),
  `iso_code`             varchar(20),
  `iso_3166_2`           varchar(20),
  `order_count_7d`       BIGINT,
  `order_original_amount_7d` DECIMAL(38, 2),
  `activity_reduce_amount_7d` DECIMAL(38, 2),
  `coupon_reduce_amount_7d` DECIMAL(38, 2),
  `order_total_amount_7d` DECIMAL(38, 2),
  `order_count_30d`      BIGINT,
  `order_original_amount_30d` DECIMAL(38, 2),
  `activity_reduce_amount_30d` DECIMAL(38, 2),
  `coupon_reduce_amount_30d` DECIMAL(38, 2),
  `order_total_amount_30d` DECIMAL(38, 2),
  `ts`                   TIMESTAMP(3),
  `dt`                   STRING
)
PARTITIONED BY (`dt`)
WITH (
  'connector'='hudi',
  'path'
='hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dws/dws_trade_province_order_nd',
  'table.type'='MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field'
='province_id,province_name,area_code,iso_code,iso_3166_2',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks'='1',
  'write.tasks' = '1',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable'='true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db'='hive_dws',
  'hive_sync.table'='dws_trade_province_order_nd'
);
```

2) 数据装载

```
insert into hudi_dws.dws_trade_province_order_nd
select
```

```
province_id,
province_name,
area_code,
iso_code,
iso_3166_2,
sum(if(dt>=date_add(CURRENT_DATE,-6),order_count_1d,0)),

sum(if(dt>=date_add(CURRENT_DATE,-6),order_original_amount_1d,
0)),

sum(if(dt>=date_add(CURRENT_DATE,-6),activity_reduce_amount_1d,
0)),

sum(if(dt>=date_add(CURRENT_DATE,-6),coupon_reduce_amount_1d,0)
),

sum(if(dt>=date_add(CURRENT_DATE,-6),order_total_amount_1d,0))
,
sum(order_count_1d),
sum(order_original_amount_1d),
sum(activity_reduce_amount_1d),
sum(coupon_reduce_amount_1d),
sum(order_total_amount_1d),
LOCALTIMESTAMP as ts,
cast(CURRENT_DATE as STRING) dt
from hudi_dws.dws_trade_province_order_1d/*+
OPTIONS('read.tasks'='1')*/
where dt>=date_add(CURRENT_DATE,-29)
and dt<=CURRENT_DATE
group by
province_id,province_name,area_code,iso_code,iso_3166_2;
```

12.3 历史至今汇总表

12.3.1 交易域用户粒度订单历史至今汇总表

dws_trade_user_order_td

1) 建表语句

```
CREATE TABLE hudi_dws.dws_trade_user_order_td
(
  `user_id` BIGINT,
  `order_date_first` STRING,
  `order_date_last` STRING,
  `order_count_td` BIGINT,
  `order_num_td` BIGINT,
  `original_amount_td` DECIMAL(38, 2),
  `activity_reduce_amount_td` DECIMAL(38, 2),
  `coupon_reduce_amount_td` DECIMAL(38, 2),
  `total_amount_td` DECIMAL(38, 2),
  `ts` TIMESTAMP(3),
  `dt` STRING
) WITH (
  'connector'='hudi',
  'path'
```

```
= 'hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dws/dws_trade_user_order_td',
  'table.type' = 'MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'user_id',
  'hoodie.datasource.write.precombine.field' = 'ts',
  'write.bucket_assign.tasks' = '1',
  'write.tasks' = '1',
  'compaction.tasks' = '1',
  'compaction.async.enabled' = 'true',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '5',
  'read.streaming.enabled' = 'true',
  'changelog.enabled' = 'true',
  'read.streaming.skip_compaction' = 'true',
  'hive_sync.enable' = 'true',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
  'hive_sync.db' = 'hive_dws',
  'hive_sync.table' = 'dws_trade_user_order_td'
);
```

2) 数据装载

```
insert into hudi_dws.dws_trade_user_order_td
select
  user_id,
  min(dt) login_date_first,
  max(dt) login_date_last,
  sum(order_count_1d) order_count,
  sum(order_num_1d) order_num,
  sum(order_original_amount_1d) original_amount,
  sum(activity_reduce_amount_1d) activity_reduce_amount,
  sum(coupon_reduce_amount_1d) coupon_reduce_amount,
  sum(order_total_amount_1d) total_amount,
  LOCALTIMESTAMP as ts,
  cast(CURRENT_DATE as STRING) dt
from
  hudi_dws.dws_trade_user_order_1d/*+
  OPTIONS('read.tasks'='1')*/
group by user_id;
```

12.3.2 用户域用户粒度登录历史至今汇总表 dws_user_user_login_td

1) 建表语句

```
CREATE TABLE hudi_dws.dws_user_user_login_td
(
  `user_id`          bigint,
  `login_date_last`  STRING,
  `login_count_td`   BIGINT,
  `ts`               TIMESTAMP(3),
  `dt`               STRING
) WITH (
  'connector' = 'hudi',
  'path' = 'hdfs://hadoop102:8020/user/hudi/warehouse/hudi_dws/dws_user_user_login_td',
  'table.type' = 'MERGE_ON_READ',
  'hoodie.datasource.write.recordkey.field' = 'user_id',
```



```
'hoodie.datasource.write.precombine.field' = 'ts',
'write.bucket_assign.tasks'='1',
'write.tasks' = '1',
'compaction.tasks' = '1',
'compaction.async.enabled' = 'true',
'compaction.schedule.enabled' = 'true',
'compaction.trigger.strategy' = 'num_commits',
'compaction.delta_commits' = '5',
'read.streaming.enabled' = 'true',
'changelog.enabled' = 'true',
'read.streaming.skip_compaction' = 'true',
'hive_sync.enable'='true',
'hive_sync.mode' = 'hms',
'hive_sync.metastore.uris' = 'thrift://hadoop102:9083',
'hive_sync.db'='hive_dws',
'hive_sync.table'='dws_user_user_login_td'
);
```

2) 数据装载

```
insert into hudi_dws.dws_user_user_login_td
select
    u.id,
    nvl(login_date_last,u.dt) login_date_last,
    nvl(login_count_td,1) login_count_td,
    LOCALTIMESTAMP as ts,
    cast(CURRENT_DATE as STRING) dt
from
    (
        select
            id,
            dt
        from hudi_dim.dim_user_info/*+ OPTIONS('read.tasks'='1')*/
    )u
left join
    (
        select
            user_id,
            max(dt) login_date_last,
            count(*) login_count_td
        from hudi_dwd.dwd_user_login/*+ OPTIONS('read.tasks'='1')*/
        group by user_id
    )l
on cast(u.id as string)=l.user_id;
```

第 13 章 湖仓一体之 ADS 层

使用 flink 的 mysql-connector 连接器, 将结果直接导入到 mysql 中, 方便后续进行可视化展示。

13.1 MySQL 建库建表

首先需要在 mysql 中创建出存储数据的表格。

13.1.1 创建数据库

```
CREATE DATABASE IF NOT EXISTS gmall_report DEFAULT CHARSET utf8  
COLLATE utf8_general_ci;  
USE gmall_report;  
set table.dynamic-table-options.enabled=true;
```

13.1.2 创建表

1) 各渠道流量统计

```
DROP TABLE IF EXISTS `ads_traffic_stats_by_channel`;  
CREATE TABLE `ads_traffic_stats_by_channel` (  
  `dt` date NOT NULL COMMENT '统计日期',  
  `recent_days` bigint(20) NOT NULL COMMENT '最近天数,1:最近 1 天,7:  
最近 7 天,30:最近 30 天',  
  `channel` varchar(16) CHARACTER SET utf8 COLLATE utf8_general_ci  
NOT NULL COMMENT '渠道',  
  `uv_count` bigint(20) NULL DEFAULT NULL COMMENT '访客人数',  
  `avg_duration_sec` bigint(20) NULL DEFAULT NULL COMMENT '会话  
平均停留时长, 单位为秒',  
  `avg_page_count` bigint(20) NULL DEFAULT NULL COMMENT '会话平均  
浏览页面数',  
  `sv_count` bigint(20) NULL DEFAULT NULL COMMENT '会话数',  
  `bounce_rate` decimal(16, 2) NULL DEFAULT NULL COMMENT '跳出率',  
  PRIMARY KEY (`dt`, `recent_days`, `channel`) USING BTREE  
) ENGINE = InnoDB CHARACTER SET = utf8 COLLATE = utf8_general_ci  
COMMENT = '各渠道流量统计' ROW_FORMAT = DYNAMIC;
```

2) 用户变动统计

```
DROP TABLE IF EXISTS `ads_user_change`;  
CREATE TABLE `ads_user_change` (  
  `dt` varchar(16) CHARACTER SET utf8 COLLATE utf8_general_ci NOT  
NULL COMMENT '统计日期',  
  `user_churn_count` varchar(16) CHARACTER SET utf8 COLLATE  
utf8_general_ci NULL DEFAULT NULL COMMENT '流失用户数',  
  `user_back_count` varchar(16) CHARACTER SET utf8 COLLATE  
utf8_general_ci NULL DEFAULT NULL COMMENT '回流用户数',  
  PRIMARY KEY (`dt`) USING BTREE  
) ENGINE = InnoDB CHARACTER SET = utf8 COLLATE = utf8_general_ci  
COMMENT = '用户变动统计' ROW_FORMAT = DYNAMIC;
```

3) 用户留存率

```
DROP TABLE IF EXISTS `ads_user_retention`;
```

```
CREATE TABLE `ads_user_retention` (  
  `dt` date NOT NULL COMMENT '统计日期',  
  `create_date` varchar(16) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL COMMENT '用户新增日期',  
  `retention_day` int(20) NOT NULL COMMENT '截至当前日期留存天数',  
  `retention_count` bigint(20) NULL DEFAULT NULL COMMENT '留存用户数量',  
  `new_user_count` bigint(20) NULL DEFAULT NULL COMMENT '新增用户数量',  
  `retention_rate` decimal(16, 2) NULL DEFAULT NULL COMMENT '留存率',  
  PRIMARY KEY (`dt`, `create_date`, `retention_day`) USING BTREE  
) ENGINE = InnoDB CHARACTER SET = utf8 COLLATE = utf8_general_ci  
COMMENT = '留存率' ROW_FORMAT = DYNAMIC;
```

4) 用户新增活跃统计

```
DROP TABLE IF EXISTS `ads_user_stats`;  
CREATE TABLE `ads_user_stats` (  
  `dt` date NOT NULL COMMENT '统计日期',  
  `recent_days` bigint(20) NOT NULL COMMENT '最近 n 日, 1:最近 1 日, 7:最近 7 日, 30:最近 30 日',  
  `new_user_count` bigint(20) NULL DEFAULT NULL COMMENT '新增用户数',  
  `active_user_count` bigint(20) NULL DEFAULT NULL COMMENT '活跃用户数',  
  PRIMARY KEY (`dt`, `recent_days`) USING BTREE  
) ENGINE = InnoDB CHARACTER SET = utf8 COLLATE = utf8_general_ci  
COMMENT = '用户新增活跃统计' ROW_FORMAT = DYNAMIC;
```

5) 用户行为漏斗分析

```
DROP TABLE IF EXISTS `ads_user_action`;  
CREATE TABLE `ads_user_action` (  
  `dt` date NOT NULL COMMENT '统计日期',  
  `home_count` bigint(20) NULL DEFAULT NULL COMMENT '浏览首页人数',  
  `good_detail_count` bigint(20) NULL DEFAULT NULL COMMENT '浏览商品详情页人数',  
  `cart_count` bigint(20) NULL DEFAULT NULL COMMENT '加入购物车人数',  
  `order_count` bigint(20) NULL DEFAULT NULL COMMENT '下单人数',  
  `payment_count` bigint(20) NULL DEFAULT NULL COMMENT '支付人数',  
  PRIMARY KEY (`dt`) USING BTREE  
) ENGINE = InnoDB CHARACTER SET = utf8 COLLATE = utf8_general_ci  
COMMENT = '漏斗分析' ROW_FORMAT = DYNAMIC;
```

6) 新增下单用户统计

```
DROP TABLE IF EXISTS `ads_new_order_user_stats`;  
CREATE TABLE `ads_new_order_user_stats` (  
  `dt` varchar(20) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL,  
  `recent_days` bigint(20) NOT NULL,  
  `new_order_user_count` bigint(20) NULL DEFAULT NULL,
```

```
PRIMARY KEY (`recent_days`, `dt`) USING BTREE
) ENGINE = InnoDB CHARACTER SET = utf8 COLLATE = utf8_general_ci
ROW_FORMAT = Dynamic;
```

7) 最近 30 日各品牌复购率

```
DROP TABLE IF EXISTS `ads_repeat_purchase_by_tm`;
CREATE TABLE `ads_repeat_purchase_by_tm` (
  `dt` date NOT NULL COMMENT '统计日期',
  `recent_days` bigint(20) NOT NULL COMMENT '最近天数,30:最近 30 天',
  `tm_id` varchar(16) CHARACTER SET utf8 COLLATE utf8_general_ci
NOT NULL COMMENT '品牌 ID',
  `tm_name` varchar(32) CHARACTER SET utf8 COLLATE utf8_general_ci
NULL DEFAULT NULL COMMENT '品牌名称',
  `order_repeat_rate` decimal(16, 2) NULL DEFAULT NULL COMMENT '复购率',
  PRIMARY KEY (`dt`, `recent_days`, `tm_id`) USING BTREE
) ENGINE = InnoDB CHARACTER SET = utf8 COLLATE = utf8_general_ci
COMMENT = '各品牌复购率统计' ROW_FORMAT = DYNAMIC;
```

8) 各品牌商品下单统计

```
DROP TABLE IF EXISTS `ads_order_stats_by_tm`;
CREATE TABLE `ads_order_stats_by_tm` (
  `dt` date NOT NULL COMMENT '统计日期',
  `recent_days` bigint(20) NOT NULL COMMENT '最近天数,1:最近 1 天,7:最近 7 天,30:最近 30 天',
  `tm_id` varchar(16) CHARACTER SET utf8 COLLATE utf8_general_ci
NOT NULL COMMENT '品牌 ID',
  `tm_name` varchar(32) CHARACTER SET utf8 COLLATE utf8_general_ci
NULL DEFAULT NULL COMMENT '品牌名称',
  `order_count` bigint(20) NULL DEFAULT NULL COMMENT '订单数',
  `order_user_count` bigint(20) NULL DEFAULT NULL COMMENT '订单人数',
  PRIMARY KEY (`dt`, `recent_days`, `tm_id`) USING BTREE
) ENGINE = InnoDB CHARACTER SET = utf8 COLLATE = utf8_general_ci
COMMENT = '各品牌商品交易统计' ROW_FORMAT = DYNAMIC;
```

9) 各分类商品下单统计

```
DROP TABLE IF EXISTS `ads_order_stats_by_cate`;
CREATE TABLE `ads_order_stats_by_cate` (
  `dt` date NOT NULL COMMENT '统计日期',
  `recent_days` bigint(20) NOT NULL COMMENT '最近天数,1:最近 1 天,7:最近 7 天,30:最近 30 天',
  `category1_id` varchar(16) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL COMMENT '一级分类 id',
  `category1_name` varchar(64) CHARACTER SET utf8 COLLATE utf8_general_ci NULL DEFAULT NULL COMMENT '一级分类名称',
  `category2_id` varchar(16) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL COMMENT '二级分类 id',
  `category2_name` varchar(64) CHARACTER SET utf8 COLLATE utf8_general_ci NULL DEFAULT NULL COMMENT '二级分类名称',
  `category3_id` varchar(16) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL COMMENT '三级分类 id',
  `category3_name` varchar(64) CHARACTER SET utf8 COLLATE
```

```
utf8_general_ci NULL DEFAULT NULL COMMENT '三级分类名称',
`order_count` bigint(20) NULL DEFAULT NULL COMMENT '订单数',
`order_user_count` bigint(20) NULL DEFAULT NULL COMMENT '订单
人数',
PRIMARY KEY (`dt`, `recent_days`, `category1_id`,
`category2_id`, `category3_id`) USING BTREE
) ENGINE = InnoDB CHARACTER SET = utf8 COLLATE = utf8_general_ci
COMMENT = '各分类商品交易统计' ROW_FORMAT = DYNAMIC;
```

10) 各分类商品购物车存量 Top3

```
DROP TABLE IF EXISTS `ads_sku_cart_num_top3_by_cate`;
CREATE TABLE `ads_sku_cart_num_top3_by_cate` (
`dt` date NOT NULL COMMENT '统计日期',
`category1_id` varchar(16) CHARACTER SET utf8 COLLATE
utf8_general_ci NOT NULL COMMENT '一级分类 ID',
`category1_name` varchar(64) CHARACTER SET utf8 COLLATE
utf8_general_ci NULL DEFAULT NULL COMMENT '一级分类名称',
`category2_id` varchar(16) CHARACTER SET utf8 COLLATE
utf8_general_ci NOT NULL COMMENT '二级分类 ID',
`category2_name` varchar(64) CHARACTER SET utf8 COLLATE
utf8_general_ci NULL DEFAULT NULL COMMENT '二级分类名称',
`category3_id` varchar(16) CHARACTER SET utf8 COLLATE
utf8_general_ci NOT NULL COMMENT '三级分类 ID',
`category3_name` varchar(64) CHARACTER SET utf8 COLLATE
utf8_general_ci NULL DEFAULT NULL COMMENT '三级分类名称',
`sku_id` varchar(16) CHARACTER SET utf8 COLLATE utf8_general_ci
NOT NULL COMMENT '商品 id',
`sku_name` varchar(128) CHARACTER SET utf8 COLLATE
utf8_general_ci NULL DEFAULT NULL COMMENT '商品名称',
`cart_num` bigint(20) NULL DEFAULT NULL COMMENT '购物车中商品数
量',
`rk` bigint(20) NULL DEFAULT NULL COMMENT '排名',
PRIMARY KEY (`dt`, `sku_id`, `category1_id`, `category2_id`,
`category3_id`) USING BTREE
) ENGINE = InnoDB CHARACTER SET = utf8 COLLATE = utf8_general_ci
COMMENT = '各分类商品购物车存量 Top10' ROW_FORMAT = DYNAMIC;
```

11) 各品牌商品收藏次数 Top3

```
DROP TABLE IF EXISTS `ads_sku_favor_count_top3_by_tm`;
CREATE TABLE `ads_sku_favor_count_top3_by_tm` (
`dt` varchar(20) CHARACTER SET utf8 COLLATE utf8_general_ci NOT
NULL,
`tm_id` varchar(20) CHARACTER SET utf8 COLLATE utf8_general_ci
NOT NULL,
`tm_name` varchar(128) CHARACTER SET utf8 COLLATE
utf8_general_ci NULL DEFAULT NULL,
`sku_id` varchar(20) CHARACTER SET utf8 COLLATE utf8_general_ci
NOT NULL,
`sku_name` varchar(128) CHARACTER SET utf8 COLLATE
utf8_general_ci NULL DEFAULT NULL,
`favor_count` bigint(20) NULL DEFAULT NULL,
`rk` bigint(20) NULL DEFAULT NULL,
PRIMARY KEY (`dt`, `tm_id`, `sku_id`) USING BTREE
) ENGINE = InnoDB CHARACTER SET = utf8 COLLATE = utf8_general_ci
```

```
ROW_FORMAT = Dynamic;
```

12) 下单到支付时间间隔平均值

```
DROP TABLE IF EXISTS `ads_order_to_pay_interval_avg`;
CREATE TABLE `ads_order_to_pay_interval_avg` (
  `dt` varchar(20) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL,
  `order_to_pay_interval_avg` bigint(20) NULL DEFAULT NULL,
  PRIMARY KEY (`dt`) USING BTREE
) ENGINE = InnoDB CHARACTER SET = utf8 COLLATE = utf8_general_ci
ROW_FORMAT = Dynamic;
```

13) 各省份交易统计

```
DROP TABLE IF EXISTS `ads_order_by_province`;
CREATE TABLE `ads_order_by_province` (
  `dt` date NOT NULL COMMENT '统计日期',
  `recent_days` bigint(20) NOT NULL COMMENT '最近天数,1:最近 1 天,7:最近 7 天,30:最近 30 天',
  `province_id` varchar(16) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL COMMENT '省份 ID',
  `province_name` varchar(16) CHARACTER SET utf8 COLLATE utf8_general_ci NULL DEFAULT NULL COMMENT '省份名称',
  `area_code` varchar(16) CHARACTER SET utf8 COLLATE utf8_general_ci NULL DEFAULT NULL COMMENT '地区编码',
  `iso_code` varchar(16) CHARACTER SET utf8 COLLATE utf8_general_ci NULL DEFAULT NULL COMMENT '国际标准地区编码',
  `iso_code_3166_2` varchar(16) CHARACTER SET utf8 COLLATE utf8_general_ci NULL DEFAULT NULL COMMENT '国际标准地区编码',
  `order_count` bigint(20) NULL DEFAULT NULL COMMENT '订单数',
  `order_total_amount` decimal(16, 2) NULL DEFAULT NULL COMMENT '订单金额',
  PRIMARY KEY (`dt`, `recent_days`, `province_id`) USING BTREE
) ENGINE = InnoDB CHARACTER SET = utf8 COLLATE = utf8_general_ci
COMMENT = '各地区订单统计' ROW_FORMAT = DYNAMIC;
```

14) 优惠券使用情况统计

```
DROP TABLE IF EXISTS `ads_coupon_stats`;
CREATE TABLE `ads_coupon_stats` (
  `dt` varchar(20) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL,
  `coupon_id` varchar(20) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL,
  `coupon_name` varchar(128) CHARACTER SET utf8 COLLATE utf8_general_ci NOT NULL,
  `used_count` bigint(20) NULL DEFAULT NULL,
  `userd_user_count` bigint(20) NULL DEFAULT NULL,
  PRIMARY KEY (`dt`, `coupon_id`) USING BTREE
) ENGINE = InnoDB CHARACTER SET = utf8 COLLATE = utf8_general_ci
ROW_FORMAT = Dynamic;
```

13.1 流量主题

之后再 sql-client 中创建对 mysql 的映射，最终将数据导出。

13.1.1 各渠道流量统计 ads_traffic_stats_by_channel

需求说明如下

统计周期	统计粒度	指标	说明
最近 1/7/30 日	渠道	访客数	统计访问人数
最近 1/7/30 日	渠道	会话平均停留时长	统计每个会话平均停留时长
最近 1/7/30 日	渠道	会话平均浏览页面数	统计每个会话平均浏览页面数
最近 1/7/30 日	渠道	会话总数	统计会话总数
最近 1/7/30 日	渠道	跳出率	只有一个页面的会话的比例

1) 建表语句

```
CREATE TABLE hudi_ads.ads_traffic_stats_by_channel
(
  `dt` STRING,
  `recent_days` INT,
  `channel` STRING,
  `uv_count` BIGINT,
  `avg_duration_sec` BIGINT,
  `avg_page_count` BIGINT,
  `sv_count` BIGINT,
  `bounce_rate` DECIMAL(16, 2),
  PRIMARY KEY (`dt`, `recent_days`, `channel`) NOT ENFORCED
) WITH (
  'connector'='jdbc',
  'url'='jdbc:mysql://hadoop102:3306/gmall_report?useUnicode=true&characterEncoding=UTF-8',
  'username'='root',
  'password'='000000',
  'connection.max-retry-timeout'='60s',
  'table-name'='ads_traffic_stats_by_channel',
  'sink.buffer-flush.max-rows'='500',
  'sink.buffer-flush.interval'='5s',
  'sink.max-retries'='3',
  'sink.parallelism'='1'
);
```

2) 数据装载

```
insert into hudi_ads.ads_traffic_stats_by_channel
select
  dt,
  recent_days,
  channel,
  cast(count(distinct(mid_id)) as bigint) uv_count,
  cast(avg(during_time_1d)/1000 as bigint) avg_duration_sec,
  cast(avg(page_count_1d) as bigint) avg_page_count,
  cast(count(*) as bigint) sv_count,
  cast(sum(if(page_count_1d=1,1,0))/count(*) as decimal(16,2))
  bounce_rate
```



```
from
(
select
dt,
channel,
mid_id,
during_time_ld,
page_count_ld,
Array[1,7,30] days
from hudi_dws.dws_traffic_session_page_view_ld/*+
OPTIONS('read.tasks'='1')*/
) t
CROSS JOIN UNNEST(days) tmp(recent_days)
where dt>=date_add(dt,-recent_days+1)
group by dt,recent_days,channel;
```

13.2 用户主题

13.2.1 用户变动统计 ads_user_change

该需求包括两个指标，分别为流失用户数和回流用户数，以下为对两个指标的解释说明。

指标	说明
流失用户数	之前活跃过的用户，最近一段时间未活跃，就称为流失用户。 此处要求统计 7 日前（只包含 7 日前当天）活跃，但最近 7 日未活跃的用户总数。
回流用户数	之前的活跃用户，一段时间未活跃（流失），今日又活跃了，就称为回流用户。此处要求统计回流用户总数。

1) 建表语句

```
CREATE TABLE hudi_ads.ads_user_change
(
`dt` STRING,
`user_churn_count` BIGINT,
`user_back_count` BIGINT,
PRIMARY KEY (`dt`) NOT ENFORCED
) WITH (
'connector'='jdbc',
'url' = 'jdbc:mysql://hadoop102:3306/gmall_report?useUnicode=true&characterEncoding=UTF-8',
'username' = 'root',
'password' = '123456',
'connection.max-retry-timeout' = '60s',
'table-name' = 'ads_user_change',
'sink.buffer-flush.max-rows' = '500',
'sink.buffer-flush.interval' = '5s',
'sink.max-retries' = '3',
'sink.parallelism' = '1'
);
```


2) 数据装载

```
insert into hudi_ads.ads_user_change
select
    churn.dt,
    user_churn_count,
    user_back_count
from
    (
        select
            dt,
            count(*) user_churn_count
        from
            hudi_dws.dws_user_user_login_td/*+
OPTIONS('read.tasks'='1')*/
        where login_date_last=date_add(dt,-7)
        group by dt
    ) churn
join
    (
        select
            dt,
            count(*) user_back_count
        from
            (
                select
                    dt,
                    user_id,
                    login_date_last
                from
                    hudi_dws.dws_user_user_login_td/*+
OPTIONS('read.tasks'='1')*/

            ) t1
        join
            (
                select
                    user_id,
                    login_date_last login_date_previous
                from
                    hudi_dws.dws_user_user_login_td/*+
OPTIONS('read.tasks'='1')*/
                where dt=date_add(dt,-1)
            ) t2
        on t1.user_id=t2.user_id
        where datediff(login_date_last,login_date_previous)>=8
        group by dt
    ) back
on churn.dt=back.d;
```

13.2.2 用户留存率 ads_user_retention

留存分析一般包含新增留存和活跃留存分析。

新增留存分析是分析某天的新增用户中，有多少人后续的活跃行为。活跃留存分析是分析某天的活跃用户中，有多少人后续的活跃行为。

留存分析是衡量产品对用户价值高低的重要指标。

此处要求统计新增留存率，新增留存率具体是指留存用户数与新增用户数的比值，例如 2020-06-14 新增 100 个用户，1 日之后（2020-06-15）这 100 人中有 80 个人活跃了，那 2020-06-14 的 1 日留存数则为 80，2020-06-14 的 1 日留存率则为 80%。

要求统计每天的 1 至 7 日留存率，如下图所示。

时间	新增用户	1天后	2天后	3天后	4天后	5天后	6天后	7天后
2021-06-01	642	1.09%	0.93%	0.78%	0.47%	0.62%	0.78%	0.47%
2021-06-02	691	1.74%	1.74%	1.3%	0.87%	1.16%	0.87%	
2021-06-03	647	1.55%	1.24%	1.39%	1.24%	1.39%		
2021-06-04	629	2.38%	1.75%	1.59%	1.59%			
2021-06-05	247	1.21%	1.21%	2.02%				
2021-06-06	241	2.49%	1.66%					
2021-06-07	562	1.07%						

1) 建表语句

```
CREATE TABLE hudi_ads.ads_user_retention
(
  `dt` STRING,
  `create_date` STRING,
  `retention_day` INT,
  `retention_count` BIGINT,
  `new_user_count` BIGINT,
  `retention_rate` DECIMAL(16, 2),
  PRIMARY KEY (`dt`, `create_date`, `retention_day`) NOT
ENFORCED
) WITH (
  'connector'='jdbc',
  'url'='
=jdbc:mysql://hadoop102:3306/gmall_report?useUnicode=true&characterEncoding=UTF-8',
  'username'='root',
  'password'='123456',
  'connection.max-retry-timeout'='60s',
  'table-name'='ads_user_retention',
  'sink.buffer-flush.max-rows'='500',
  'sink.buffer-flush.interval'='5s',
  'sink.max-retries'='3',
  'sink.parallelism'='1'
);
```

2) 数据装载

```
insert into hudi_ads.ads_user_retention
select
  t2.dt,
  login_date_first create_date,
  datediff(t2.dt,login_date_first) retention_day,
  sum(if(login_date_last=t2.dt,1,0)) retention_count,
  count(*) new_user_count,
```

```

        cast(sum(if(login_date_last=t2.dt,1,0))/count(*)*100 as
decimal(16,2)) retention_rate
from
(
    select
        dt,
        user_id,
        date_id login_date_first
    from
        hudi_dwd.dwd_user_register/*+
OPTIONS('read.tasks'='1')*/
    where dt>=date_add(dt,-7)
    and dt<dt
)t1
join
(
    select
        dt,
        user_id,
        login_date_last
    from
        hudi_dws.dws_user_user_login_td/*+
OPTIONS('read.tasks'='1')*/
    where dt=dt
)t2
on t1.user_id=t2.user_id
and t1.dt=t2.dt
group by t2.dt,login_date_first;

```

13.2.3 用户新增活跃统计 ads_user_stats

需求说明如下

统计周期	指标	指标说明
最近 1、7、30 日	新增用户数	略
最近 1、7、30 日	活跃用户数	略

1) 建表语句

```

CREATE TABLE hudi_ads.ads_user_stats
(
    `dt` STRING,
    `recent_days` BIGINT,
    `new_user_count` BIGINT,
    `active_user_count` BIGINT,
    PRIMARY KEY (`dt`, `recent_days`) NOT ENFORCED
) WITH (
    'connector'='jdbc',
    'url' =
'jdbc:mysql://hadoop102:3306/gmall_report?useUnicode=true&characterEncoding=UTF-8',
    'username' = 'root',
    'password' = '123456',
    'connection.max-retry-timeout' = '60s',
    'table-name' = 'ads_user_stats',
    'sink.buffer-flush.max-rows' = '500',
    'sink.buffer-flush.interval' = '5s',

```

```
'sink.max-retries' = '3',  
'sink.parallelism' = '1'  
);
```

2) 数据装载

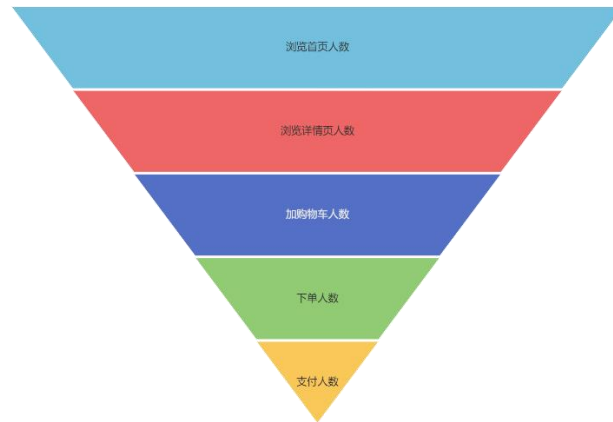
```
insert into hudi_ads.ads_user_stats  
select  
    t2.dt,  
    t2.recent_days,  
    new_user_count,  
    active_user_count  
from  
(  
    select  
        t1.dt dt,  
        recent_days,  
  
        sum(if(login_date_last>=date_add(t1.dt,-recent_days+1),1,0))  
        new_user_count  
    from  
        (  
            select  
                dt,  
                login_date_last,  
                Array[1,7,30] days  
            from  
                hudi_dws.dws_user_user_login_td/*+  
OPTIONS('read.tasks'='1')*/  
        ) t1  
        CROSS JOIN UNNEST(days) tmp(recent_days)  
    group by t1.dt,recent_days  
    ) t2  
join  
(  
    select  
        t3.dt,  
        recent_days,  
        sum(if(date_id>=date_add(t3.dt,-recent_days+1),1,0))  
        active_user_count  
    from  
        (  
            select  
                dt,  
                date_id,  
                Array[1,7,30] days  
            from  
                hudi_dwd.dwd_user_register/*+  
OPTIONS('read.tasks'='1')*/  
        ) t3  
        CROSS JOIN UNNEST(days) tmp(recent_days)  
    group by t3.dt,recent_days  
    ) t4  
on t2.recent_days=t4.recent_days  
and t2.dt=t4.dt;
```

13.2.4 用户行为漏斗分析 ads_user_action

漏斗分析是一个数据分析模型,它能够科学反映一个业务流程从起点到终点各阶段用户

转化情况。由于其能将各阶段环节都展示出来，故哪个阶段存在问题，就能一目了然。

漏斗图



该需求要求统计一个完整的购物流程各个阶段的人数，具体说明如下：

统计周期	指标	说明
最近 1 日	首页浏览人数	略
最近 1 日	商品详情页浏览人数	略
最近 1 日	加购人数	略
最近 1 日	下单人数	略
最近 1 日	支付人数	支付成功人数

1) 建表语句

```
CREATE TABLE hudi_ads.ads_user_action
(
  `dt` STRING,
  `home_count` BIGINT,
  `good_detail_count` BIGINT,
  `cart_count` BIGINT,
  `order_count` BIGINT,
  `payment_count` BIGINT,
  PRIMARY KEY (`dt`) NOT ENFORCED
) WITH (
  'connector'='jdbc',
  'url'='jdbc:mysql://hadoop102:3306/gmall_report?useUnicode=true&characterEncoding=UTF-8',
  'username'='root',
  'password'='123456',
  'connection.max-retry-timeout'='60s',
  'table-name'='ads_user_action',
  'sink.buffer-flush.max-rows'='500',
  'sink.buffer-flush.interval'='5s',
  'sink.max-retries'='3',
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
'sink.parallelism' = '1'
);
```

2) 数据装载

```
insert into hudi_ads.ads_user_action
select
    page.dt,
    home_count,
    good_detail_count,
    cart_count,
    order_count,
    payment_count
from
(
    select
        dt,
        1 recent_days,
        sum(if(page_id='home',1,0)) home_count,
        sum(if(page_id='good_detail',1,0)) good_detail_count
    from hudi_dws.dws_traffic_page_visitor_page_view_1d/*+
OPTIONS('read.tasks'='1')*/
    where page_id in ('home','good_detail')
    group by dt
)page
join
(
    select
        dt,
        1 recent_days,
        count(*) cart_count
    from hudi_dws.dws_trade_user_cart_add_1d/*+
OPTIONS('read.tasks'='1')*/
    group by dt
)cart
on page.recent_days=cart.recent_days and page.dt=cart.dt
join
(
    select
        dt,
        1 recent_days,
        count(*) order_count
    from hudi_dws.dws_trade_user_order_1d/*+
OPTIONS('read.tasks'='1')*/
    group by dt
)ord
on page.recent_days=ord.recent_days and page.dt=ord.dt
join
(
    select
        dt,
        1 recent_days,
        count(*) payment_count
    from hudi_dws.dws_trade_user_payment_1d/*+
OPTIONS('read.tasks'='1')*/
    group by dt
)pay
on page.recent_days=pay.recent_days and page.dt=pay.dt;
```

13.2.5 新增下单用户统计 ads_new_order_user_stats

需求说明如下

统计周期	指标	说明
最近 1、7、30 日	新增下单人数	略

1) 建表语句

```
CREATE TABLE hudi_ads.ads_new_order_user_stats
(
  `dt` STRING,
  `recent_days` BIGINT,
  `new_order_user_count` BIGINT,
  PRIMARY KEY (`recent_days`, `dt`) NOT ENFORCED
) WITH (
  'connector'='jdbc',
  'url' = 'jdbc:mysql://hadoop102:3306/gmall_report?useUnicode=true&characterEncoding=UTF-8',
  'username' = 'root',
  'password' = '123456',
  'connection.max-retry-timeout' = '60s',
  'table-name' = 'ads_new_order_user_stats',
  'sink.buffer-flush.max-rows' = '500',
  'sink.buffer-flush.interval' = '5s',
  'sink.max-retries' = '3',
  'sink.parallelism' = '1'
);
```

2) 数据装载

```
insert into hudi_ads.ads_new_order_user_stats
select
  dt,
  recent_days,
  sum(if(order_date_first>=date_add(dt,-recent_days+1),1,0))
new_order_user_count
from
(
  select
    dt,
    order_date_first,
    Array[1,7,30] days
  from hudi_dws.dws_trade_user_order_td/*+
OPTIONS('read.tasks'='1')*/
) t
CROSS JOIN UNNEST(days) tmp(recent_days)
group by dt,recent_days;
```

13.3 商品主题

13.3.1 最近 30 日各品牌复购率 ads_repeat_purchase_by_tm

需求说明如下

统计周期	统计粒度	指标	说明
最近 30 日	品牌	复购率	重复购买人数占购买人数比例

1) 建表语句

```
CREATE TABLE hudi_ads.ads_repeat_purchase_by_tm
(
  `dt`          STRING,
  `recent_days` INT,
  `tm_id`       BIGINT,
  `tm_name`     VARCHAR(100),
  `order_repeat_rate` DECIMAL(16, 2),
  PRIMARY KEY (`dt`, `recent_days`, `tm_id`) NOT ENFORCED
) WITH (
  'connector'='jdbc',
  'url'='jdbc:mysql://hadoop102:3306/gmall_report?useUnicode=true&characterEncoding=UTF-8',
  'username' = 'root',
  'password' = '123456',
  'connection.max-retry-timeout' = '60s',
  'table-name' = 'ads_repeat_purchase_by_tm',
  'sink.buffer-flush.max-rows' = '500',
  'sink.buffer-flush.interval' = '5s',
  'sink.max-retries' = '3',
  'sink.parallelism' = '1'
);
```

2) 数据装载

```
insert into hudi_ads.ads_repeat_purchase_by_tm
select
  dt,
  30,
  tm_id,
  tm_name,
  cast(sum(if(order_count>=2,1,0))/sum(if(order_count>=1,1,0))
as decimal(16,2))
from
(
  select
    dt,
    user_id,
    tm_id,
    tm_name,
    sum(order_count_30d) order_count
  from hudi_dws.dws_trade_user_sku_order_nd/*+
OPTIONS('read.tasks'='1')*/
  group by dt,user_id, tm_id,tm_name
)t1
group by dt,tm_id,tm_name;
```

13.3.2 各品牌商品下单统计 ads_order_stats_by_tm

需求说明如下

统计周期	统计粒度	指标	说明
最近 1、7、30 日	品牌	订单数	略
最近 1、7、30 日	品牌	订单人数	略

1) 建表语句

```
CREATE TABLE hudi_ads.ads_order_stats_by_tm
(
  `dt` STRING,
  `recent_days` BIGINT,
  `tm_id` BIGINT,
  `tm_name` VARCHAR(100),
  `order_count` BIGINT,
  `order_user_count` BIGINT,
  PRIMARY KEY (`dt`, `recent_days`, `tm_id`) NOT ENFORCED
) WITH (
  'connector'='jdbc',
  'url' = 'jdbc:mysql://hadoop102:3306/gmall_report?useUnicode=true&characterEncoding=UTF-8',
  'username' = 'root',
  'password' = '123456',
  'connection.max-retry-timeout' = '60s',
  'table-name' = 'ads_order_stats_by_tm',
  'sink.buffer-flush.max-rows' = '500',
  'sink.buffer-flush.interval' = '5s',
  'sink.max-retries' = '3',
  'sink.parallelism' = '1'
);
```

2) 数据装载

```
insert into hudi_ads.ads_order_stats_by_tm
select
  dt,
  recent_days,
  tm_id,
  tm_name,
  order_count,
  order_user_count
from
(
  select
    dt,
    1 recent_days,
    tm_id,
    tm_name,
    sum(order_count_1d) order_count,
    count(distinct(user_id)) order_user_count
  from
    hudi_dws.dws_trade_user_sku_order_1d/*+
    OPTIONS('read.tasks'='1')*/
  group by dt,tm_id,tm_name
  union all
  select
    dt,
    recent_days,
```

```

        tm_id,
        tm_name,
        sum(order_count),
        count(distinct(if(order_count>0,user_id,cast(null as
string))))
    from
    (
        select
            dt,
            recent_days,
            user_id,
            tm_id,
            tm_name,
            case recent_days
                when 7 then order_count_7d
                when 30 then order_count_30d
            end order_count
        from
        (
            select
                dt,
                user_id,
                tm_id,
                tm_name,
                order_count_7d,
                order_count_30d,
                Array[7,30] days
            from
                hudi_dws.dws_trade_user_sku_order_nd/*+
OPTIONS('read.tasks'='1')*/
            )t1
        CROSS JOIN UNNEST(days) tmp(recent_days)
        )t2
    group by dt,recent_days,tm_id,tm_name
)odr;

```

13.3.3 各品类商品下单统计 ads_order_stats_by_cate

需求说明如下

统计周期	统计粒度	指标	说明
最近 1、7、30 日	品类	订单数	略
最近 1、7、30 日	品类	订单人数	略

1) 建表语句

```

CREATE TABLE hudi_ads.ads_order_stats_by_cate
(
    `dt` STRING,
    `recent_days` INT,
    `category1_id` BIGINT,
    `category1_name` VARCHAR(10),
    `category2_id` BIGINT,
    `category2_name` VARCHAR(200),
    `category3_id` BIGINT,
    `category3_name` VARCHAR(200),

```

```
`order_count`          BIGINT,
`order_user_count`      BIGINT,
PRIMARY KEY (`dt`, `recent_days`, `category1_id`,
`category2_id`, `category3_id`) NOT ENFORCED
) WITH (
  'connector'='jdbc',
  'url' =
'jdbc:mysql://hadoop102:3306/gmall_report?useUnicode=true&characterEncoding=UTF-8',
  'username' = 'root',
  'password' = '123456',
  'connection.max-retry-timeout' = '60s',
  'table-name' = 'ads_order_stats_by_cate',
  'sink.buffer-flush.max-rows' = '500',
  'sink.buffer-flush.interval' = '5s',
  'sink.max-retries' = '3',
  'sink.parallelism' = '1'
);
```

2) 数据装载

```
insert into hudi_ads.ads_order_stats_by_cate
select
  dt,
  recent_days,
  category1_id,
  category1_name,
  category2_id,
  category2_name,
  category3_id,
  category3_name,
  order_count,
  order_user_count
from
(
  select
    dt,
    1 recent_days,
    category1_id,
    category1_name,
    category2_id,
    category2_name,
    category3_id,
    category3_name,
    sum(order_count_1d) order_count,
    count(distinct(user_id)) order_user_count
  from
    hudi_dws.dws_trade_user_sku_order_1d/*+
OPTIONS('read.tasks'='1')*/
  group by
    dt,category1_id,category1_name,category2_id,category2_name,category3_id,category3_name
  union all
  select
    dt,
    recent_days,
    category1_id,
    category1_name,
    category2_id,
```

```
        category2_name,
        category3_id,
        category3_name,
        sum(order_count),
        count(distinct(if(order_count>0,user_id,cast(null as
string))))
    from
    (
        select
            dt,
            recent_days,
            user_id,
            category1_id,
            category1_name,
            category2_id,
            category2_name,
            category3_id,
            category3_name,
            case recent_days
                when 7 then order_count_7d
                when 30 then order_count_30d
            end order_count
        from
        (
            select
                dt,
                user_id,
                category1_id,
                category1_name,
                category2_id,
                category2_name,
                category3_id,
                category3_name,
                order_count_7d,
                order_count_30d,
                Array[7,30] days
            from
                hudi_dws.dws_trade_user_sku_order_nd/*+
OPTIONS('read.tasks'='1')*/
            )
            CROSS JOIN UNNEST(days) tmp(recent_days)
        )t1
    group
        by
        dt,recent_days,category1_id,category1_name,category2_id,catego
ry2_name,category3_id,category3_name
    )odr;
```

13.3.4 各分类商品购物车存量 Top3ads_sku_cart_num_top3_by_cate

1) 建表语句

```
CREATE TABLE hudi_ads.ads_sku_cart_num_top3_by_cate
(
    `dt` STRING,
    `category1_id` BIGINT,
    `category1_name` VARCHAR(10),
    `category2_id` BIGINT,
    `category2_name` VARCHAR(200),
    `category3_id` BIGINT,
```

```
`category3_name` VARCHAR(200),
`sku_id`          BIGINT,
`sku_name`        VARCHAR(200),
`cart_num`        INT,
`rk`              BIGINT,
PRIMARY KEY (`dt`, `sku_id`, `category1_id`, `category2_id`,
`category3_id`) NOT ENFORCED
) WITH (
  'connector'='jdbc',
  'url'      =
'jdbc:mysql://hadoop102:3306/gmall_report?useUnicode=true&characterEncoding=UTF-8',
  'username' = 'root',
  'password' = '123456',
  'connection.max-retry-timeout' = '60s',
  'table-name' = 'ads_sku_cart_num_top3_by_cate',
  'sink.buffer-flush.max-rows' = '500',
  'sink.buffer-flush.interval' = '5s',
  'sink.max-retries' = '3',
  'sink.parallelism' = '1'
);
```

2) 数据装载

```
insert into hudi_ads.ads_sku_cart_num_top3_by_cate
select
  dt,
  category1_id,
  category1_name,
  category2_id,
  category2_name,
  category3_id,
  category3_name,
  sku_id,
  sku_name,
  cart_num,
  rk
from
(
  select
    cart.dt,
    sku_id,
    sku_name,
    category1_id,
    category1_name,
    category2_id,
    category2_name,
    category3_id,
    category3_name,
    cart_num,
    ROW_NUMBER() over (partition by
category1_id,category2_id,category3_id order by cart_num desc) rk
  from
  (
    select
      dt,
      sku_id,
```

```

        sum(sku_num) cart_num
    from hudi_dwd.dwd_trade_cart_add/*+
OPTIONS('read.tasks'='1')*/
    group by dt,sku_id
)cart
left join
(
    select
        dt,
        id,
        sku_name,
        category1_id,
        category1_name,
        category2_id,
        category2_name,
        category3_id,
        category3_name
    from hudi_dim.dim_sku/*+ OPTIONS('read.tasks'='1')*/
)sku
on cart.sku_id=sku.id and cart.dt=sku.dt
)t1
where rk<=3;

```

13.3.5 各品牌商品收藏次数 Top3ads_sku_favor_count_top3_by_tm

1) 建表语句

```

CREATE TABLE hudi_ads.ads_sku_favor_count_top3_by_tm
(
    `dt` STRING,
    `tm_id` BIGINT,
    `tm_name` VARCHAR(100),
    `sku_id` BIGINT,
    `sku_name` VARCHAR(200),
    `favor_count` BIGINT,
    `rk` BIGINT,
    PRIMARY KEY (`dt`, `tm_id`, `sku_id`) NOT ENFORCED
) WITH (
    'connector'='jdbc',
    'url' =
'jdbc:mysql://hadoop102:3306/gmall_report?useUnicode=true&characterEncoding=UTF-8',
    'username' = 'root',
    'password' = '123456',
    'connection.max-retry-timeout' = '60s',
    'table-name' = 'ads_sku_favor_count_top3_by_tm',
    'sink.buffer-flush.max-rows' = '500',
    'sink.buffer-flush.interval' = '5s',
    'sink.max-retries' = '3',
    'sink.parallelism' = '1'
);

```

2) 数据装载

```

insert into hudi_ads.ads_sku_favor_count_top3_by_tm
select
    dt,
    tm_id,
    tm_name,

```

```

        sku_id,
        sku_name,
        favor_add_count_1d,
        rk
    from
    (
        select
            dt,
            tm_id,
            tm_name,
            sku_id,
            sku_name,
            favor_add_count_1d,
            ROW_NUMBER() over (partition by tm_id order by
            favor_add_count_1d desc) rk
        from
            hudi_dws.dws_interaction_sku_favor_add_1d/*+
        OPTIONS('read.tasks'='1')*/
    )t1
    where rk<=3;

```

13.4 交易主题

13.4.1 下单到支付时间间隔平均值 ads_order_to_pay_interval_avg

具体要求：最近 1 日完成支付的订单的下单时间到支付时间的时间间隔的平均值。

1) 建表语句

```

CREATE TABLE hudi_ads.ads_order_to_pay_interval_avg
(
    `dt` STRING,
    `order_to_pay_interval_avg` BIGINT,
    PRIMARY KEY (`dt`) NOT ENFORCED
) WITH (
    'connector'='jdbc',
    'url' =
    'jdbc:mysql://hadoop102:3306/gmall_report?useUnicode=true&characterEncoding=UTF-8',
    'username' = 'root',
    'password' = '123456',
    'connection.max-retry-timeout' = '60s',
    'table-name' = 'ads_order_to_pay_interval_avg',
    'sink.buffer-flush.max-rows' = '500',
    'sink.buffer-flush.interval' = '5s',
    'sink.max-retries' = '3',
    'sink.parallelism' = '1'
);

```

2) 数据装载

```

insert into hudi_ads.ads_order_to_pay_interval_avg
select
    dt,

    cast(avg(to_unix_timestamp(payment_time)-to_unix_timestamp(order_time)) as bigint)
from
    hudi_dwd.dwd_trade_trade_flow_acc/*+
    OPTIONS('read.tasks'='1')*/
where payment_date_id is not null

```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
group by dt;
```

13.4.2 各省份交易统计 ads_order_by_province

需求说明如下

统计周期	统计粒度	指标	说明
最近 1、7、30 日	省份	订单数	略
最近 1、7、30 日	省份	订单金额	略

1) 建表语句

```
CREATE TABLE hudi_ads.ads_order_by_province
(
  `dt` STRING,
  `recent_days` INT,
  `province_id` INT,
  `province_name` VARCHAR(20),
  `area_code` VARCHAR(20),
  `iso_code` VARCHAR(20),
  `iso_code_3166_2` VARCHAR(20),
  `order_count` BIGINT,
  `order_total_amount` DECIMAL(38, 2),
  PRIMARY KEY (`dt`, `recent_days`, `province_id`) NOT ENFORCED
) WITH (
  'connector'='jdbc',
  'url' =
  'jdbc:mysql://hadoop102:3306/gmall_report?useUnicode=true&characterEncoding=UTF-8',
  'username' = 'root',
  'password' = '123456',
  'connection.max-retry-timeout' = '60s',
  'table-name' = 'ads_order_by_province',
  'sink.buffer-flush.max-rows' = '500',
  'sink.buffer-flush.interval' = '5s',
  'sink.max-retries' = '3',
  'sink.parallelism' = '1'
);
```

2) 数据装载

```
insert into hudi_ads.ads_order_by_province
select
  dt,
  1 recent_days,
  province_id,
  province_name,
  area_code,
  iso_code,
  iso_3166_2,
  order_count_1d,
  order_total_amount_1d
from hudi_dws.dws_trade_province_order_1d/*+
OPTIONS('read.tasks'='1')*/
union
select
  dt,
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)


```
recent_days,
province_id,
province_name,
area_code,
iso_code,
iso_3166_2,
case recent_days
  when 7 then order_count_7d
  when 30 then order_count_30d
end order_count,
case recent_days
  when 7 then order_total_amount_7d
  when 30 then order_total_amount_30d
end order_total_amount,
LOCALTIMESTAMP as ts
from
(
  select
    dt,
    province_id,
    province_name,
    area_code,
    iso_code,
    iso_3166_2,
    order_count_7d,
    order_count_30d,
    order_total_amount_7d,
    order_total_amount_30d,
    Array[7,30] days
  from
    hudi_dws.dws_trade_province_order_nd/*+
OPTIONS('read.tasks'='1')*/
) t1
CROSS JOIN UNNEST(days) tmp(recent_days);
```

13.5 优惠券主题

13.5.1 优惠券使用统计 ads_coupon_stats

需求说明如下

统计周期	统计粒度	指标	说明
最近1日	优惠券	使用次数	支付才算使用
最近1日	优惠券	使用人数	支付才算使用

1) 建表语句

```
CREATE TABLE hudi_ads.ads_coupon_stats
(
  `dt` STRING,
  `coupon_id` BIGINT,
  `coupon_name` VARCHAR(100),
  `used_count` BIGINT,
  `used_user_count` BIGINT,
  PRIMARY KEY (`dt`) NOT ENFORCED
) WITH (
```

```
'connector'='jdbc',
'url'
'jdbc:mysql://hadoop102:3306/gmall_report?useUnicode=true&characterEncoding=UTF-8',
'username' = 'root',
'password' = '123456',
'connection.max-retry-timeout' = '60s',
'table-name' = 'ads_coupon_stats',
'sink.buffer-flush.max-rows' = '500',
'sink.buffer-flush.interval' = '5s',
'sink.max-retries' = '3',
'sink.parallelism' = '1'
);
```

2) 数据装载

```
insert into hudi_ads.ads_coupon_stats
select
    dt,
    coupon_id,
    coupon_name,
    cast(sum(used_count_1d) as bigint),
    cast(count(*) as bigint)
from
    hudi_dws.dws_tool_user_coupon_coupon_used_1d/*+
OPTIONS('read.tasks'='1')*/
group by dt,coupon_id,coupon_name;
```